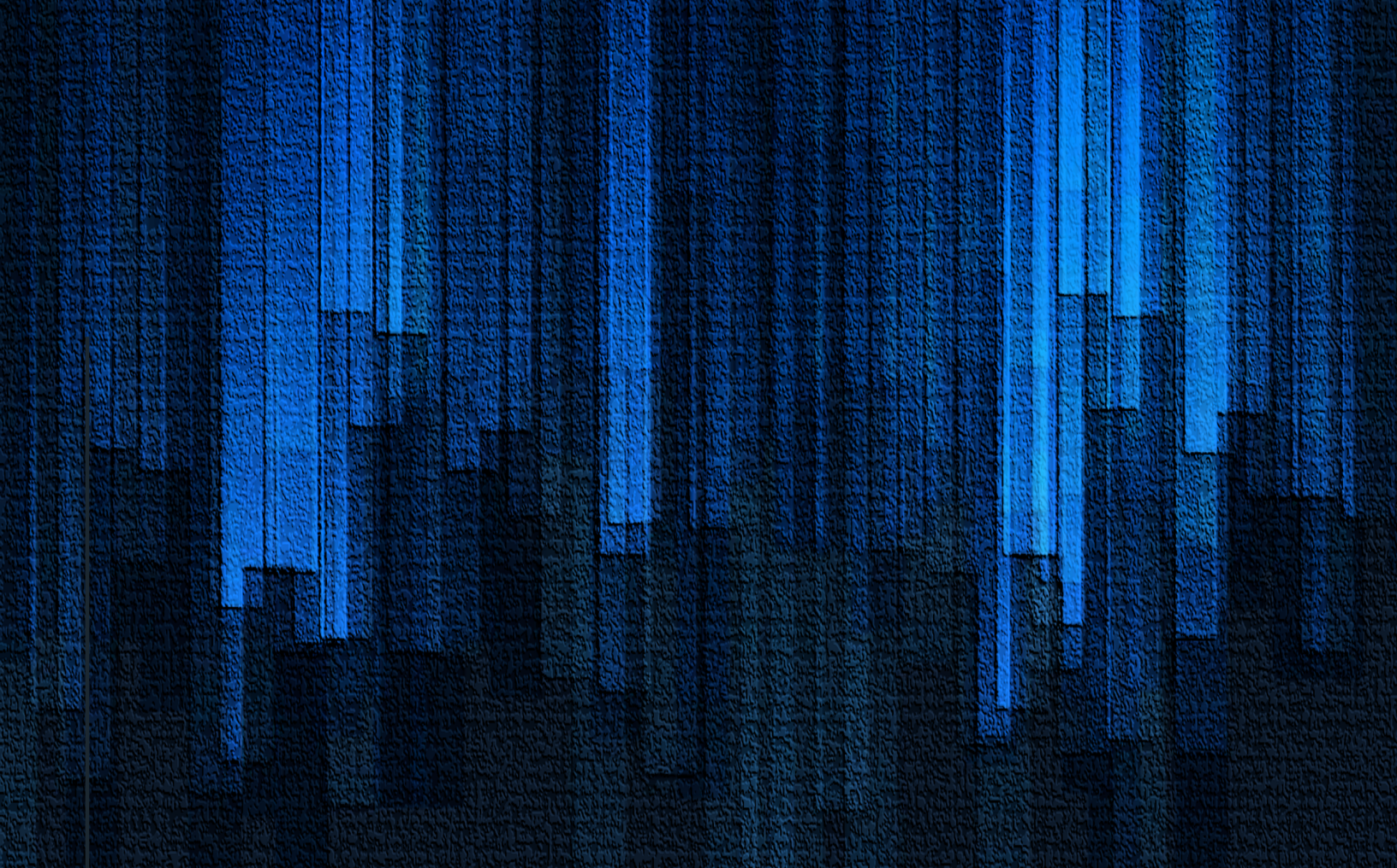




The Managed Runtime Environment

Diving into the JVM with Kotlin

Miguel Gamboa

The Managed Runtime Environment: Diving into the JVM with Kotlin

Miguel Gamboa

This book is available at <https://leanpub.com/kotlinonjvm>

This version was published on 2026-05-02 ISBN 978-989-33-6322-5



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Miguel Gamboa

Contents

Preface	i
Introduction to the JVM	1
Managed Runtime Environment	1
Java Ecosystem	2
JVM	3
Class Loader, CLASSPATH and Interoperability	5
Central Repository and Dependency Management	8
Exercises	10
Type Basics	12
JVM Type Fundamentals	12
Primitive Types	12
Reference Types	12
Members	14
Nested Types	19
Kotlin to Java type system	22
Kotlin Class Members	22
Conversions and Casting	24
Primitive Numeric Conversions	24
Reference Type Casting	25
Boxing and Unboxing	27
Abstract and Base Types	29
Anonymous classes	34
Kotlin Object declarations	36
Kotlin Function types	39
Lambdas Translation in former Kotlin 1.x	41
Exercises	43
Reflection	46
Kotlin Reflection	46
A Simple Logger Using Kotlin Reflection	50
Java Reflection	52
Checking Type Compatibility	54

CONTENTS

NaiveMapper: automatic object to object mapper	57
NaiveMapper Motivation	57
NaiveMapper Implementation	58
Annotations	61
Reflection of generic types	64
Reflection Overhead and the Enhancement of NaiveMapper	69
Reflection Overhead	69
Enhancing the NaiveMapper	71
Exercises	75
Performance testing and Microbenchmarking	79
Performance Measurement Pitfalls	79
JMH	86
Exercises	91
Bytecode and the Just-in-time Compiler	94
Evaluation Stack	94
Constant Pool and Descriptors	98
Bytecode for Comparison and Control Flow	100
Managing Instances and Members	101
Creating Objects	101
Accessing Fields	102
Invoking Methods	104
Method Dispatch Internals	105
Metaprogramming	111
Introduction	111
Class-File API	113
Dynamic Mapper	120
Cojen Maker	132
Cojen Maker Features	132
Cojen Maker Examples	133
Dynamic Mapper with Cojen Maker	136
Sequences, Generators and Suspending functions	141
Sequence Alternatives	141
Operation names	143
Composability: <i>method chaining</i> versus <i>nested functions</i>	143
<i>Eager</i> versus <i>lazy</i> evaluation	144
Extensibility	149
Access approach: <i>pull</i> versus <i>push</i>	151
Generators and yield pattern	152
Suspend functions	155
Deconstructing yield and generators	158

Translating a Suspend Function into a State Machine	158
sequence builder	169
Exercises	171
Garbage Collection and Cleanup Actions	174
The Managed Heap	174
Garbage Collection	175
Types Requiring Special Cleanup	179
Using closeable types	179
Finalization and Cleaners	182
Bibliography	187

Preface

This book reflects my journey through software development across academia and industry, as well as my occasional pursuit of an academic degree.

My first encounter with a **managed runtime environment** (MRE) was with the *msvbvm* DLL (*dynamic-link library*) in 1997, during my first job developing software in Visual Basic. This environment, known as the “*Microsoft Visual Basic virtual machine*,” provided a runtime for Visual Basic applications. Compared to the JVM, the *msvbvm* lacked some runtime services that are now considered essential in all modern MREs.

Today, these services are standard in runtimes such as .NET, JavaScript engines (e.g., V8), Ruby, Python, and many others. Yet, the JVM was perhaps the first widely accepted runtime to gather all essential features in a single platform, including dynamic loading, metadata, bytecode, verifier, just-in-time compiler, reflection, exception handling, garbage collection, and more.

I recall the significant impact of the JVM on the software development industry and the news when IBM first announced that the AS/400 (a family of midrange computers) would integrate a Java Virtual Machine, enabling it to run Java applications. This marked a major shift in midrange computer software development. Since then, many alternatives to the JVM have appeared, but none have achieved the same level of widespread acceptance. The fact that so many high-level programming languages now compile to JVM bytecode is a testament to its broad adoption.

Years later, I began my teaching career at ISEL (part of the Polytechnic Institute of Lisbon), where I had the opportunity to deepen my knowledge of MREs, particularly in .NET and JVM. From the beginning, I was fascinated by the concept, architecture, and principles governing MREs. Although my professional experience exposed me to these technologies, the pressures of deadlines and revenue goals often prevented me from fully exploring the intricacies of the JVM. Teaching at ISEL gave me the chance to explore the inner mechanics of the JVM, marking the start of my exciting journey into the world of MREs.

Later, I decided to pursue my PhD focusing on MREs. During this period, I explored various methods of interacting with the JVM. Specifically, I made custom modifications to the open-source JVM implementation, Jikes RVM. Subsequently, I shifted my approach to use instrumentation techniques with ASM to achieve a technology-independent solution that could be integrated with any JVM.

After completing my PhD, I contributed to a couple of open-source projects in Java, .NET, and JavaScript, which provided a foundation for starting my own project. I am the author and primary contributor of the HtmlFlow Java library, which has been my platform for exploring emerging trends. Since its launch, HtmlFlow has allowed me to experiment with Java 8 streams, lambdas, and *CompletableFuture*, as well as *progressive server-side rendering* (PSSR) and reactive data models such as *reactive streams*. More recently, it has been extended to include an idiomatic Kotlin DSL and

to explore alternative web templating asynchronous idioms, including CPS and Kotlin suspending functions.

The knowledge necessary for developing HtmlFlow is covered in this book, including topics such as metadata (Chapter 2), reflection (Chapter 3), performance analysis and microbenchmarking (Chapter 4), bytecode (Chapter 5), metaprogramming (Chapter 6), and sequence pipeline (Chapter 7). These subjects are integral components of the [Languages and Managed Runtimes](https://cc.isel.pt/academia/leic-en/lae-en/)¹ course in the Computer Science and Computer Engineering BSc at ISEL.

Author's Note

All the texts in this book are my original work, created entirely by me. However, I have **used AI assistance for proofreading**, correcting typos, and improving grammatical clarity. The content, ideas, and structure remain solely my own.

¹<https://cc.isel.pt/academia/leic-en/lae-en/>

Chapter 1: Introduction to the JVM

Simply put, the primary goal of this book is to deepen understanding of the software and **runtime services** that operate alongside our programs in a **Managed Runtime Environment** (MRE), such as the JVM. Secondly, it offers insights into the **underlying mechanisms** that power these environments. Finally, it equips programmers with the knowledge needed to interact with these runtime services and mechanisms, enabling them to develop their own utility micro-tools as alternatives to mainstream frameworks for logging, dependency injection, parsing, or other.

The JVM features discussed in this book are not exclusive to the JVM but are widely recognized as common characteristics of modern runtime environments. Many of these foundational concepts can also be applied to other MREs, such as .NET, or other. The choice of the JVM as the focus for this book is explained in Section 1.2 of this chapter.

This book is not specifically about the Kotlin programming language. In fact, any JVM-targeting language, such as Scala or Clojure, could be used to achieve the objectives outlined in each chapter. Here, Kotlin serves as the tool through which we delve into the inner workings of the JVM. This approach is similar to Jeffrey Richter’s notable book, *CLR via C#*, where the primary focus was on the Common Language Runtime (CLR)—the .NET platform’s equivalent to the JVM—with C# chosen as the preferred language for exploration.

1.1: Managed Runtime Environment

Developing software for a **Managed Runtime Environment** (MRE), such as JVM, .NET, Ruby, JavaScript, or other, can significantly enhance development productivity compared to *unmanaged environments*. In *unmanaged environments*, there is no reliance on **runtime services** to support software execution. These runtime services, such as *exception handling*, *garbage collection*, *sandboxing*, *verification*, etc, help prevent illegal states and violations within process execution, such as illegal memory access.

By handling tasks such as memory management, MREs significantly enhance and streamline the software development process for programmers. Additionally, offering runtime services like a **reflective API** unlocks new possibilities, making it easier for programmers to dynamically integrate features at runtime.

However, when running software in MREs, developers must consider not only the behavior of their programs but also the surrounding code provided by the MRE, which runs alongside and supports their software.

If we consider that, when building software, the amount of reused code from standard libraries and other dependencies far exceeds the size of our own codebase, then a significant portion of the running software—handling runtime services—is beyond the programmer’s direct control.

Consider a simple scenario of a web application backend. It is quite common to rely on various libraries and frameworks, such as an HTTP server (e.g., Tomcat, Jetty, or other), logging libraries (e.g., Log4J or SLF4J), unit testing frameworks (e.g., JUnit or TestNG), JSON parsers (e.g., Gson or Jackson), and dependency injection frameworks (e.g., Guice or Spring). These frameworks delve deeply into the JVM's internals to deliver their functionality.

For many programmers, these tools may seem like a “*kind of magic*” that somehow makes everything work. This book aims to demystify that magic, enabling programmers to better understand how these tools operate and, if necessary, manage and leverage them more effectively.

1.2: Java Ecosystem

The term **Java** can refer to either the **Programming Language** or the **Managed Runtime Environment** (MRE), commonly known as the **Java Virtual Machine (JVM)**, depending on the context in which it is used. While it is mostly accurate that, around its **inception in 1995** by **Sun Microsystems**, these two concepts were closely intertwined—since Java was initially the only high-level programming language targeting the JVM—the ecosystem quickly expanded with the emergence of numerous new programming languages designed for the JVM.

A **JVM programming language** refers to any high-level programming language that can be compiled into a valid class file (`.class`), which the JVM can execute. As of 2025, when this book is being written, there are several well-established programming languages, such as Clojure, Groovy, Scala, and Kotlin, which serve as mainstream alternatives to the Java programming language. For instance, at Google I/O in 2019, Google announced Kotlin as the preferred language for Android development, designating the platform as Kotlin-first.

After 30 years since the emergence of the JVM, it remains one of the most widely used runtimes for backend development and enterprise applications, if not the most utilized. While there are alternatives like .NET, JavaScript engines (e.g., V8), Ruby, and Python that provide comparable managed runtime services, the JVM continues to be a strong competitor, offering unmatched maturity and reliability in production environments.

The maturity and reliability of the JVM are among its greatest advantages, playing a crucial role in its widespread adoption by many organizations. However, this does not necessarily mean the JVM was inherently superior to its competition, as “superior” depends heavily on the quality metrics being considered. For instance, .NET may have outperformed Java in certain aspects when it first appeared, particularly in some well-known benchmarks. Nonetheless, the JVM's maturity, combined with its established user base and growing adoption, encouraged stakeholders to minimize risk and choose Java. This scenario is reminiscent of the famous 1980s marketing slogan, “*Nobody Gets Fired for Buying IBM*,” which significantly contributed to IBM's growth. Similarly, while alternative platforms with innovative features have emerged, stakeholders often prefer to take a more conservative approach and adopt the most widely used and trusted solution.

Still, the clear distinction between the JVM as a *reference implementation* and its various *distributions* has facilitated the proliferation of several mainstream JVM distributions. With **Oracle** holding

the *Java trademark* (since acquiring Sun Microsystems in 2009) and the **OpenJDK** serving as the **official reference implementation**, several new players have emerged, offering production-ready JVM distributions. These include organizations such as:

- **Amazon:** Corretto
- **Azul Systems:** Zulu
- **Eclipse:** Temurin
- **IBM:** Semeru

Oracle continues to maintain its commercial distribution of the Java platform, which includes **HotSpot**, the original JVM implementation from Sun Microsystems. Available in the **JRE** (Java Runtime Environment), it provides the JVM and core libraries required to execute Java programs. The **JDK** (Java Development Kit) extends the JRE by including development tools such as the Java compiler (i.e. `javac`), `javap` for bytecode inspection, `jmeter` for performance testing, and other utilities essential for application development.

1.3: JVM

A *software distribution unit* in the JVM can be a single class file (`.class`) or a collection of class files, typically packaged together in a JAR file (`.jar`). A class file is the result of compiling source code written in any JVM compatible programming language. It contains Java **bytecode** (JVM instructions), a symbol table, and other supplementary information. The class file format is **platform-independent**, as it relies on the JVM to dynamically translate the bytecode into machine-specific instructions for a given architecture. Today, nearly all major computing architectures have a JVM implementation capable of executing Java class files.

As an essential premise for understanding the JVM execution model, we must consider that each computer architecture can only run machine code specific to that architecture. This means that for example for a 64-bit Intel processor can only execute instructions from the x86-64 instruction set, while an Apple silicon processor is limited to executing its own ARM-based instruction set.

The idea that a `.class` file is **architecture-independent** and can run on any compatible platform is made possible by the JVM, which is responsible for **translating bytecode into machine code specific to the target architecture**. [Figure 1.1](#) illustrates this concept, showing how the same `.class` files can run on two different architectures and operating systems (e.g., Debian and macOS). Each JVM implementation handles the task of translating the bytecode into the appropriate machine code at run-time.

To enable this execution model in the most efficient way, the JVM is composed of various mechanisms that are part of the translation and execution process, including the **just-in-time (JIT)** compiler and the **class loader**, which we will detail further in the next section.

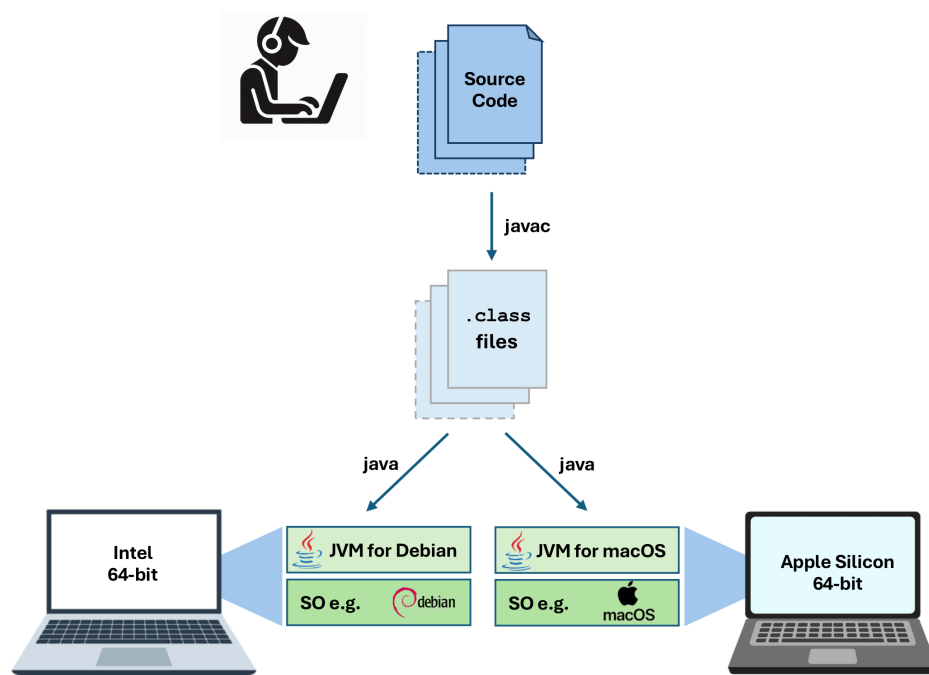


Figure 1.1. Compilation and execution model on JVM.

Bytecode is the cornerstone that enables hardware-independent software components in the Java ecosystem. However, it is the **metadata** within Java class files that serves as the “glue” connecting these components. Metadata allows a class file to discover, interact with, and reuse other class files seamlessly.

In a class file, **metadata** provides detailed information about the structure of a type, such as its fields, methods, constructors, and implemented interfaces. This includes method signatures, access modifiers, and annotations, which enable tools like the JVM, compilers, and frameworks to interpret and operate on the class at runtime or compile-time. The rules and principles governing metadata are defined by the **Type System**, which is examined in **Chapter 2**.

Metadata can be accessed at runtime through an API commonly referred to as **reflection**. In Java, this API is provided in the `java.lang.reflect` package. **Chapter 3** delves into leveraging reflection to develop custom micro-tools with dynamic behavior, automatically adaptable to the surrounding domain.

Although some runtime services, such as reflection, can enhance our applications with auto-customizable capabilities, they may also degrade the performance of certain operations compared to equivalent ad-hoc implementations. This means that while reflection can reduce redundant and repetitive source code, it introduces an inevitable performance overhead. **Chapter 4** explores state-of-the-art techniques for analyzing Java application performance, including observing and measuring the overhead caused by reflection.

Chapter 5 dives into the internal functioning of the JVM to understand the processing costs associated with different types of operations. To that end, we will explore the bytecode instruction

set in detail.

Chapter 6 explores complementary **meta-programming** techniques to reflection, such as dynamic code generation, which helps mitigate the performance overhead of reflection in certain operations. In this chapter, we will reimplement the same functionalities demonstrated in Chapter 3, but instead of relying on reflection, we will use automatic code generation to optimize the most performance-critical features. This approach enables us to reduce runtime overhead and improve the efficiency of our solutions while retaining the flexibility of dynamically adjusting behavior based on the context.

Chapter 7 explores **lazy sequences**, such as `Iterable` or `Stream`, and the `yield` operator, which is available in all modern programming environments. In this chapter, we will investigate this functionality through Kotlin’s **suspending functions** and examine how they are translated into the JVM.

The core principles and internals of the **Garbage Collector** (GC) are explored in **Chapter 8**, including the management of types that require special cleanup actions.

1.4: Class Loader, CLASSPATH and Interoperability

Regardless of the number of source code files, the compiler generates one `.class` file for each **type** defined in the source code. Remember that a `.class` file serves as a *unit of distribution* in the JVM.

“The venerable class file is the JVM’s fundamental unit of execution”—[Binstock, 2022]

The term **type** refers to any construct in the source code that defines a type, such as a `class`, `interface`, `enum`, `annotation`, or similar. Each of these constructs results in a separate `.class` file. For example, a file named `Foo.java` containing the following code will produce three separate files: `X.class`, `Y.class`, and `Z.class`, when compiled using `javac Foo.java`.

```
class X { public void print() { out.println("I am X"); }}
class Y { public void print() { out.println("I am Y"); }}
class Z { public void print() { out.println("I am Z"); }}
```

Another source file in the same directory, such as `App.java`, can reference and use any of the previously compiled classes (i.e. `X.class`, `Y.class`, or `Z.class`).

```
public class App {  
    public static void main(String[] args) {  
        System.out.println("Press ENTER to proceed.");  
        System.console().readLine();  
        new X().print();  
    }  
    public static void bar() {  
        new Y().print();  
    }  
}
```

Here, we refer to a **dependency** as the **usage** of any **type** defined in a different class file. Let's now examine how **dependency** resolution occurs at both **compile-time** and **run-time**.

When compiling Java source code, such as `App.java`, using `javac`, the compiler searches the folders specified in the `CLASSPATH` for `.class` files corresponding to the dependencies required by `App.java`, i.e. types `X` and `Y`. The `CLASSPATH` specifies the locations of user-defined `.class` files needed for compilation and execution and, by default, includes the current directory. Therefore, if the source files and their corresponding `.class` files are located in the same directory, no additional configuration is needed for `javac` to resolve the dependencies. Moreover, since the example in the `App` class only depends on the classes `X` and `Y`, we can safely delete the `Z.class` file from the folder and still compile `App.java` without any issues.

Now, **when running** `App.class`, a different dependency resolution occurs. Note that when executing `App.class` with `java App`, the execution flow does not pass through the `bar()` method. Since `bar()` is never invoked, and it is the only method that depends on class `Y`, the presence of the `Y.class` file is not required to execute `App`. We can easily verify that, once compiled, the `Y.class` file can be deleted, and `App` will still execute successfully.

The **class loader** is responsible for loading types from `.class` files into the JVM. It operates on the principle of **loading only what is necessary to ensure program execution**, delaying type loading until required. This is in contrast to a static linking approach, where all dependencies are bundled together into a resulting object file, regardless of whether they are invoked, or not. On the other hand, **delayed loading** not only accelerates program initialization but also minimizes resource consumption during execution.

Loading is typically triggered by the **just-in-time (JIT) compiler**. The JIT compiler operates on a method-by-method basis, and just before invoking a method, it must convert its bytecode into machine code. However, before this conversion, it requires access to the type definition of the declaring type. If the type is not already loaded, this will prompt the class loader to load it. This behavior can penalize the first execution of a method. However, for long-running applications, such as a web server, the initial execution cost is negligible compared to the numerous subsequent calls to the method.

The delayed loading behavior can be observed by deleting `X.class` and running the previously discussed `App` class. The program will initially run without any exceptions, displaying the message

"Press ENTER to proceed.". However, when the statement `new X().print();` is about to be executed, and the bytecode of the target methods (i.e., the `X` constructor and the `print` method) is being compiled into machine code by the JIT compiler, the JVM instructs the **class loader** to load the type `X`. At that point, the class loader will search for `X.class` in the locations specified in the `CLASSPATH`. If the file is not found, the JVM throws a `NoClassDefFoundError`. According to the Java API documentation regarding this error, we can read:

Thrown if the Java Virtual Machine or a `ClassLoader` instance tries to load in the definition of a class (as part of a normal method call or as part of creating a new instance using the new expression) and no definition of the class could be found. The searched-for class definition existed when the currently executing class was compiled, but the definition can no longer be found.

Delayed loading and dynamic linking enable types to be augmented without breaking backward compatibility. This means we can add new methods or even modify the body of an existing method, such as `print()`, without needing to rebuild the client code (i.e., the `App` class). We can observe this behavior by changing the body of the `print` method to print "I am X version 2", compiling `Foo.java` again, and running `App` to see the new message, all without the need to recompile `App`.

Any of the above classes can be used in Kotlin code just like in Java. However, the Kotlin compiler (`kotlinc`) does not include the current directory in the `CLASSPATH` by default. Therefore, to compile the `App.kt` file, which is equivalent to the previous `App.java`, we need to explicitly specify the `CLASSPATH`, for example: `kotlinc -cp . App.kt`.

Figure 1.2. `App.kt` equivalent to the `App.java`

```
fun main() {  
    println("Press ENTER to proceed.");  
    readLine();  
    X().print();  
}
```

Since we are not specifying a declaring class for the `main` function above, and because the JVM does not support top-level declarations (functions must be members of a class), the Kotlin compiler generates a class named `AppKt`. This class corresponds to the file name, `App`, with the suffix `Kt`. The generated class will contain the `main` function. As a result, we can run the application with the command `kotlin AppKt`.

Note that we are running the generated JVM class file `AppKt.class`. From the JVM's perspective, this is just a standard class file, regardless of the source programming language. In fact, `AppKt.class` depends on other class files, i.e. `X.class`, which has been compiled from a different programming language.

When we invoke `kotlin`, it simply runs the JVM with the `CLASSPATH` configured to include the Kotlin standard library. This means we can also run `AppKt.class` using `java` directly with the command: `java AppKt`. However, this will throw `NoClassDefFoundError: kotlin/io/ConsoleKt`.

This happens because, by default, `java` only includes the current directory and the Java standard libraries in the `CLASSPATH`. To run code that depends on Kotlin standard types, such as `kotlin.io.ConsoleKt`, we must include the Kotlin standard library in the `CLASSPATH`.

In my case, since Kotlin is installed in `/opt/homebrew/Cellar/kotlin`, I need to include the following path to the Kotlin standard library:

```
java -cp '.: /opt/homebrew/Cellar/kotlin/2.1.0/libexec/lib/kotlin-stdlib.jar' AppKt
```

For those using Windows, you must use the `;` character as the path separator instead of `:`.

1.5: Central Repository and Dependency Management

Although not directly provided by the JVM distribution but typically by a third party, the build and **dependency management** system is one of the core utilities in a modern MRE. Our programs may depend on auxiliary libraries beyond the Java standard libraries, and these libraries must be linked to our programs both at compile time to check for compilation errors and at runtime to provide the desired functionality.

Beyond many other advantages, build tools enable programmers to collaborate on the same project by sharing source code through a version control repository (such as GitHub, Bitbucket, or others) without needing to share dependent libraries directly. Instead, the build configuration file (such as `pom.xml` for Maven, `build.gradle` for Gradle, or `package.json` for npm) includes the **coordinates** for each dependent library, allowing the build tool to uniquely identify and download the required libraries as needed. Additionally, build tools typically maintain a **local cache** to avoid repeatedly downloading the same library, enabling efficient sharing across multiple projects on the same machine.

Libraries are typically hosted in specific **central repositories** for each technology. For example, in Java, the **Maven Central Repository** is one of the most widely used locations; in .NET, it is NuGet; and in JavaScript, it is NPM. While in some cases the name of the central repository matches the name of the build tool, this is not a requirement. In the Java ecosystem, for instance, you can use alternative build tools like Gradle, Ivy, or Grape to manage dependencies from the Maven Central Repository.

In the Maven Central Repository, a library is referred to as an **artifact**. Each artifact is identified by an `artifactId`, which is associated with a namespace or group defined by a `groupId`, along with a specific version. The combination of these three identifiers is called the **Maven coordinates**.

Let's analyze the outcome of creating a new Gradle project from scratch and how some of the key concepts introduced in the previous section are applied. For instance, in a folder named `demo`, you can create a new project by running the command `gradle init` and pressing Enter to select the default options for all choices, except for the implementation language, which should be set to Kotlin instead of Java. The selected options are as follows:

- Type of build to generate: Application
- Implementation language: Kotlin
- Target Java version: 21
- Project name: demo
- Application structure: Single application project
- Build script DSL: Kotlin
- Test framework: kotlin.test
- Build using new APIs and behavior: no

We will not delve deeply into the inner workings of Gradle but will briefly focus on aspects relevant to the subject of this book and the JVM. The `app/build.gradle.kts` file is one of the main configuration files for our application, and it contains three core sections:

- `plugins` that are used by the build process.
- `repositories` defines the external repositories that gradle will use to locate and fetch dependent libraries
- `dependencies` the required external libraries.

Within the dependencies section, we can distinguish between:

- `implementation` used when compiling main or test source code and when running the project or tests. These dependencies are added to both the compiler's and JVM classpath.
- `testImplementation` required only for compiling test source code and running tests.

When we build this project with `./gradlew build`, several outputs are generated in the `app/build` folder. For example, in the `app/build/distributions` directory, you will find archives such as `app.tar` or `app.zip`. These archives contain everything needed to run the resulting application on any environment, including the dependency libraries and a shell or batch script that configures the `CLASSPATH` and invokes `java` with the main class.

I challenge you to decompress one of these archives and inspect the `app` script. You'll notice how the `CLASSPATH` is set to include every JAR file from the `lib` folder, which is also bundled in the archive.

Some of the JAR files in the distribution are the result of the dependency `implementation(libs.guava)` configured in `app/build.gradle.kts`. Since our application does not use anything from that library, we can safely remove it from the `app/build.gradle.kts` file.

After rebuilding the project, you'll notice that the `lib` folder inside the archive `app/build/distributions/app.zip` now contains only 3 JAR files instead of the 9 seen previously. One of the mandatory dependency JAR files is `kotlin-stdlib.jar`, which, as mentioned in a previous section, is essential. Since we selected Kotlin as the programming language, the Kotlin standard library is required to both compile and run our application.

Summary

This chapter begins by exploring the **Managed Runtime Environment**, which is central to the Java platform's ability to provide platform independence, runtime safety, and optimized performance. The JVM serves as an abstraction layer, enabling Java applications to run seamlessly across different operating systems and architectures. This environment is supported by mechanisms such as garbage collection, Just-In-Time (JIT) compilation, and the Class Loader, among others, ensuring efficient execution at runtime.

The **Java Ecosystem** is presented as a comprehensive suite of tools, libraries, and frameworks that facilitate development. Its modular design empowers developers to build applications ranging from small utilities to large enterprise systems. The availability of a vast array of third-party libraries via Central Repositories and dependency management systems significantly enhances productivity, making it easier for developers to integrate and manage external resources.

In the section on the **JVM**, we provide an overview of key components such as the class loader and the JIT compiler. The **class loader** is responsible for dynamically loading `.class` files into the JVM only when needed, implementing a lazy loading strategy that optimizes runtime efficiency. The `CLASSPATH` plays a crucial role in locating these `.class` files, whether they are part of the application code, external libraries, or system dependencies.

1.6: Exercises

Exercise 1

1. How many class files are generated from a single Java source file?
2. Given that:
 - A class file is the executable unit in the JVM.
 - A class file contains exactly one type definition (class, interface, or other).

How does the Kotlin compiler determine the class in which to place top-level declarations, such as global functions?

Exercise 2

1. A Java program is compiled, producing `App.class`, which runs successfully using the command `java App`. Given that the Kotlin runtime for the JVM is installed on the same host, is it possible to run the same `App.class` using `kotlin App`?
2. A Kotlin program is compiled, producing `App.class`, which runs successfully using the command `kotlin App`. Given that the JVM runtime is installed on the same host, is it possible to run the same `App.class` using `java App`?

3. What is the difference between running a class file using `java` versus `kotlin`?
-

Exercise 3

1. Identify the class files produced by the compilation of `App.java`.
2. From the class files identified in the previous question, which ones can be removed without compromising the execution of `App.class`?

```
import static java.lang.System.out;
import static java.lang.System.console;

public class App {
    public static void main(String[] args) {
        out.println("Press ENTER to proceed.");
        console().readLine();
        new X().print();
    }

    public static void bar() {
        new Y().print();
    }
}

class X { public void print() { out.println("I am X version 2"); }}

class Y { public void print() { out.println("I am Y"); }}

interface Z { public void print(); }
```

Chapter 2: Type Basics

The primary focus of this chapter is the JVM **type system**, which is a fundamental part that extends beyond the boundaries of specific programming languages like Java or Kotlin. However, in practice, developers typically use a programming language to work with the type system. Therefore, this chapter serves as an illustration of the underlying type system principles and mechanisms by examining them through the perspectives of both the Java programming language and Kotlin.

We will also make use of the `javap` command-line utility included with the Java Development Kit (JDK) to disassemble and display bytecode and structural information about JVM classes. These observations will help confirm our expectations about the mapping of resulting JVM classes (i.e. `.class` files) and the target type system.

2.1: JVM Type Fundamentals

The Java Virtual Machine operates with two primary kinds of types: *primitive types* and *reference types*. Consequently, there are two corresponding kinds of values that can be stored in variables: *primitive values* and *references*.

Programming languages usually include a set of *primitive types*, also denoted as *built-in types*. The compiler's responsibility is to map these *primitive types* to corresponding JVM types.

Reference types are the building block of every JVM program. Programming languages such as Java and Kotlin may have several constructs for expressing types (e.g., classes, enums, annotations, etc.), but ultimately all of these constructs map down to a JVM *reference type* definition.

2.1.1: Primitive Types

Primitive types are special data types built into the runtime. They are **not objects** created from a class. A **literal** is the source code representation of a fixed value. Literals are represented directly in code (e.g. bytecode) without requiring computation.

In the Java Virtual Machine, the supported primitive data types encompass *numeric* types and the *boolean* type. The *numeric* types include integral types (`char`, `byte`, `short`, `int`, `long`) and floating-point types (`float` and `double`).

2.1.2: Reference Types

There are three primary kinds of **reference types** in the JVM: **class**, **array**, and **interface** types. These reference types represent references to dynamically created class instances, arrays, or instances of classes or arrays that implement interfaces, respectively.

In the context of the Java Virtual Machine, **an array is a data structure that allows you to store multiple values of the same data type in a single container.**

An array type comprises a *component type* with a single dimension, and the length of this dimension is not specified by the type itself. It's important to note that the *component type* of an array type can itself be another array type. If you start from any array type and consider its *component type*, and then continue this process for subsequent component types (if they are also array types), and so on, you will eventually reach a component type that is not an array type. This final component type is referred to as the *element type* of the array.

For example, in `int[][] arr`, the component type is `int[]`, indicating that `arr` is an array of arrays of integers. The element type is `int`, which means that each element inside the innermost arrays is an integer.

In summary, the *element type* of an array type is the fundamental data type of the individual elements stored within that array and can be a primitive type, a class type, or an interface type.

An instance of a *reference type* is essentially an object, and you can access it only through a reference. However, it's important to note that the actual type of the object being referenced may not necessarily match the declared type of the reference. This situation often occurs when an object reference is of an abstract type, such as an interface or an abstract class.

For instance, consider the Java function `reverse(Temporal milestone)`, where the variable `milestone` serves as a reference. This reference, `milestone`, may point to an instance of any class that either directly or indirectly inherits from the `Temporal` type.

Type names may consist of two parts: an optional *package* name and a *local* name. The package declaration in a source code file matches the name of the folder (or folders) containing that file. This organization helps to structure and manage code in larger projects and also helps prevent naming conflicts between classes and other elements in different packages.

The fully qualified name of a class includes its package name. For example, the Java code shown in [Figure 2.1](#) defines a type whose local name is `Person` and whose package is `pt.isel`.

Figure 2.1. class `Person` in Java

```
package pt.isel;  
class Person { }
```

The fully qualified name of the class is `pt.isel.Person` as denoted by the output of `javap Person.class` in [Figure 2.2](#).

Figure 2.2. Output of `javap` for class `Person`

```
class pt.isel.Person {  
    pt.isel.Person();  
}
```

The parameterless method `pt.isel.Person()` in the `Person` class is commonly known as the default *constructor* of the type. The default *constructor* is automatically generated by the compiler when a class does not explicitly define any constructors. A type can provide multiple overloaded constructors using the same guidelines as method overloading. **The constructor is automatically called by the JVM each time an object of the type is instantiated.**

Although `jvarkit` presents the name of the constructor matching the name of the owner class (e.g. `pt.isel.Person()`), in truth, this method has the distinguished name `<init>`. This name, `<init>`, is a special name used for any constructor.

2.1.3: Members

A JVM type definition can contain zero or more *members*. As shown earlier, for classes, it must include at least one member corresponding to a constructor.

The members within a type define how the type can be utilized and how it works. Each member of a type is associated with its own access modifier (e.g., `private`, `protected`), which determines how the member can be accessed. Collectively, the accessible members of a type are often referred to as the **type's contract**.

In addition to controlling access to a given member, developers can control whether an instance of the type is needed to access the member.

Members can be defined as either *per instance* or *per type*. A *per-instance* member requires an instance of the type in order to access it. A *per-type* member does not have this requirement. In Java programming language members default to per instance. One can change this to per type using the `static` keyword.

Considering the case of the `ExampleClass` of [Figure 2.3](#), the method `perTypeMethod()` may be called with `ExampleClass.perTypeMethod()`, whereas `perInstanceMethod()` requires an instance of `ExampleClass` as a **receiver**, e.g., `final var ec = new ExampleClass(); ec.perInstanceMethod();`.

A receiver object is the object upon which an instance method is being called.

Figure 2.3. `ExampleClass` in Java

```
public class ExampleClass {  
    public void perInstanceMethod() { ... }  
    public static void perTypeMethod() { ... }  
}
```

There are **two fundamental kinds of members**: *fields* and *methods*. A *field* is a named **unit of storage** that is associated with the declaring type. A *method* is a named **behavior** that can be invoked. All other type members (e.g., *properties*) are essentially methods that have been enhanced with additional metadata.

2.1.3.1: Fields

The fields of a type control how memory is allocated. The JVM uses the *instance field* declarations of a type to determine how much storage to allocate for an instance of that type. Memory for *static fields* is allocated **once** when the type is first loaded, whereas memory for non-static (*instance*) fields is allocated **each time** an instance of the type is created. All *static fields* are initialized to default values upon allocation. Similarly, the JVM initializes *instance fields* to default values. For numeric types, the default value is zero. For Booleans, the default value is false. For object references, the default value is `null`.

We can use the **JOL** (*Java Object Layout*) toolbox to analyze object layout schemes and estimate the size of objects in JVMs. JOL is provided as a separate JAR file and can also be integrated into an IDE. Once downloaded from the Maven Central repository, we can run it from the terminal using:

```
java -cp .:jol-cli-0.17-full.jar org.openjdk.jol.Main estimates <qualified name>
```

NOTE: For Windows-based systems, replace `:` with `;` in the `-cp` value.

Figure 2.4. SavingsAccount in Java

```
package pt.isel;

public class SavingsAccount {
    private static int accountsCount;
    private short accountCode;
    private String holderName;
    private long balance;
    private boolean isActive;
    private final double interestRate = 0.02;
}
```

For the example of the `SavingsAccount` class in [Figure 2.4](#), JOL may estimate the following layout for Hotspot JDK 15, 64-bit model, 8-byte alignment, and no compressed references. Note that the `accountsCount` field is not part of the object's layout because it is a static field. Static fields belong to the class itself and exist only **once** in the class's storage, rather than being associated with individual object instances.

Figure 2.5. JOL output showing the memory layout and instance size of a SavingsAccount instance.

```
pt.isel.SavingsAccount object internals:
```

OFF	SZ	TYPE	DESCRIPTION	VALUE
0	8		(object header: mark)	N/A
8	8		(object header: class)	N/A
16	8	long	SavingsAccount.balance	N/A
24	8	double	SavingsAccount.interestRate	N/A
32	2	short	SavingsAccount.accountCode	N/A
34	1	boolean	SavingsAccount.isActive	N/A
35	5		(alignment/padding gap)	
40	8	java.lang.String	SavingsAccount.holderName	N/A

Instance size: 48 bytes

Every object in the JVM begins with a fixed-size object header, which contains information about the object, such as metadata that is specific to the particular instance of a class (the “mark”) and class-specific metadata (the “class”). The object header is not directly accessible programmatically. The precise structure of this header may vary somewhat depending on the JVM implementation and version.

The `balance` field is of type `long`, and it occupies 8 bytes (bytes 16-23). The `interestRate` field is of type `double`, and it also occupies 8 bytes (bytes 24-31). The `accountCode` field is of type `short`, and it occupies 2 bytes (bytes 32-33). The `isActive` field is of type `boolean`, and it occupies 1 byte (byte 34).

There is a padding gap of 5 bytes (bytes 35-39) to ensure proper alignment. This padding is added to align the subsequent field (`holderName`) to an 8-byte boundary. This padding is used to avoid unaligned access to individual fields, which can negatively impact performance.

Finally, the `holderName` field is of type `java.lang.String`, and it occupies 8 bytes (bytes 40-47). In a 64-bit JVM model, object references are typically 8 bytes in size due to the larger address space provided. However, the JVM may also utilize compressed references, reducing the size to only 4 bytes.

Hence, the total size of the `SavingsAccount` instance is 48 bytes. This size accounts for all the fields and any necessary padding to align them properly in memory. The memory layout is determined by the JVM and may vary based on factors such as the JVM version, the target architecture (e.g., 32-bit or 64-bit), and alignment requirements.

The JVM often reorders fields within a type to optimize memory access and usage. If the JVM were to lay out the type’s fields in the order of their declaration, it would need to insert significant padding between fields to ensure proper alignment, e.g. 6 bytes between `accountCode`-`holderName` and 7 bytes between `isActive`-`interestRate`.

To avoid the need for excessive padding, the JVM reorders the fields in a way that allows for efficient alignment without wasted space. This optimized layout ensures that memory is used efficiently, resulting in better-aligned data and improved performance.

Sometimes, it's desirable to constrain a field to have a constant value that remains immutable throughout its lifetime. The JVM employs two techniques to support fields with such **immutable** values.

Constants

The first technique is used for fields whose **constant** value can be calculated at **compile time**, and it's the most efficient approach.

In this technique, the field's static value is stored as a literal in the metadata of the type's module. To use this approach, one must use the `final` keyword. Additionally, an initialization expression for the field is required, and its value must be calculable at **compile time**. Here's an example of such a field declaration:

```
class Constants { static final int BITS_OF_10KB = 8 * 10 * 1024; }
```

Next, you may see in the output from `javap -c -v Constants.class` of [Figure 2.6](#) that the resulting value of the expression `8 * 10 * 1024` is stored as the constant value `81920`.

Figure 2.6. Output of `javap` for class `Constants`

```
class Constants {
  static final int BITS_OF_10KB;
    descriptor: I
    flags: (0x0018) ACC_STATIC, ACC_FINAL
    ConstantValue: int 81920

  Constants();
}
```

The initial value of literal fields must be known at compile time, and any attempts to modify the field will be caught as compile-time errors. In Kotlin, values of this kind, which are immutable and known at compile time, are marked with the `const` modifier, indicating a **compile-time constant**. The previous example of the `Constants` class in Kotlin can be represented as follows:

```
const val BITS_OF_10KB: Int = 8*10*1024;
```

Immutable Values

In the second technique, the JVM permits programmers to declare fields as immutable yet **dynamically initializable**. Applying the `final` keyword to a field prohibits modification of the field's value once the constructor has finished execution. The initial value can be specified by using an initialization expression or by assigning a value to the field inside the constructor method. In either case, the value used can consider dynamic aspects of the program's execution state. The example of [Figure 2.7](#) demonstrates a final field in Java.

Figure 2.7. Account class in Java with an immutable field

```
public class Account {  
    public final long created = System.currentTimeMillis();  
}
```

The provided code of Figure 2.7 dynamically generates the initial value of the created field based on the current time. However, once the value of created is set, it cannot be changed after the new instance's constructor has completed execution.

Just like with methods, access to fields can be restricted using access modifiers such as private, protected, or public. Private fields are accessible only within the same class, whereas protected fields are accessible within the same package and by derived types, even if they are in different packages. Public fields are accessible from anywhere in the program. When there is no access modifier specified for a field or method in Java, it means that the **default access level** is applied, which means the field is accessible only within the same package.

Additionally, fields can be designated as per instance or per type (static). A static field can be accessed without an instance of the type, while a non-static field requires an instance to be accessed.

The Account class of Figure 2.8 defines some features related to tracking the number of instances created and recording the creation time of each instance.

Figure 2.8. Account class in Java

```
public class Account {  
    private static int nrOfAccounts = 0;  
    public static int getNumberOfAccounts() { return nrOfAccounts; }  
    private final long created;  
    public Account() {  
        nrOfAccounts++; // <=> Account.nrOfAccounts++;  
        created = currentTimeMillis(); // <=> this.created = currentTimeMillis();  
    }  
}
```

The private static integer field named nrOfAccounts is used to keep track of the number of Account instances created and it is initialized to 0. The getNumberOfAccounts() static method provides access to the nrOfAccounts field and returns the current count of Account instances. The constructor for the Account class increments the nrOfAccounts static field every time a new Account instance is created. This effectively keeps track of how many Account objects have been created.

The method's body, including constructors, enjoys unrestricted access to the members of the declaring type. Similarly, it can access protected or public members of the declaring type's base type. In most programming languages, methods can access their declaring type's members without explicit qualification, e.g., nrOfAccounts++; or created = However, explicit qualification

is allowed and can be written as `Account.nrofAccounts++`; or `this.created = ....`. For static members, you can qualify with the type name. When it comes to instance member names, each language provides a keyword that corresponds to the instance used to invoke the method (known as the **receiver**). In Java and Kotlin, this keyword is `this`. It's important to note that static methods do not possess a `this` reference and, therefore, cannot access non-static members without first acquiring a valid instance.

Some programming languages (e.g., Java programming language) allow programmers to access static members using either an instance or the type name as a qualification. In contrast, other programming languages (e.g., C#) do not permit programmers to use the instance name when accessing static members.

In [Figure 2.9](#), we have an example demonstrating different ways of accessing the same static member, either with an object reference (e.g., `a1` and `a2`) or the owner class name (e.g., `Account`).

Figure 2.9. Accessing a static member, through an object reference or the owner class.

```
Account a1 = new Account();
Account a2 = new Account();
System.out.println(a1.getNumberOfAccounts());
System.out.println(a2.getNumberOfAccounts());
System.out.println(Account.getNumberOfAccounts());
```

Although all of these are legal accesses, a compilation warning is generated when we use an object reference: “*The static method `getNumberOfAccounts()` from the type `Account` should be accessed in a static way.*”

2.1.4: Nested Types

The Java programming language permits the definition of a class within another class, known as a nested class. These nested classes can be categorized into two types: *static* and *non-static*. In the context of this book, we will refer to static nested classes simply as “*nested classes*” and non-static nested classes as “*inner classes*”.

In the following example of [Figure 2.10](#), the `NestedClass` class is declared within the `OuterClass` class. [Figure 2.11](#) presents its equivalent implementation in Kotlin. Unlike Java, which requires the `static` keyword to define a nested class, Kotlin assumes this behavior by default for nested class declarations.

Figure 2.10. Example of a nested class in Java.

```
class OuterClass {
    static class NestedClass {
    }
}
```

Figure 2.11. Example of a nested class in Kotlin.

```
class OuterClass {
    class NestedClass
}
```

Despite `NestedClass` being declared as a member of `OuterClass`, it is not listed by the `javap` tool, unlike other members such as fields or methods. The constructor method is the only member presented in the output of `javap OuterClass.class`:

```
class OuterClass {
    OuterClass();
}
```

To observe the resulting relationship between the two classes, include the `-verbose` (or simply `-v`) option, which will show additional output, such as:

```
NestMembers:
    OuterClass$NestedClass
InnerClasses:
    static #17= #14 of #7; // NestedClass=OuterClass$NestedClass of class OuterClass
```

Here we can observe that the actual name assigned by the compiler to the nested class is `OuterClass$NestedClass`. In fact, an additional class file generated by the compiler corresponds to `OuterClass$NestedClass.class`. This file contains the definition for `NestedClass` class, which we can also inspect using the `javap` tool. When using `javap -v 'OuterClass$NestedClass.class'`, the output displays its single constructor and provides information about the relationship with the outer class, as shown below:

```
...
SourceFile: "OuterClass.java"
NestHost: class OuterClass
```

Now, let's examine the difference with an inner class. Objects that are instances of an inner class exist within an instance of the outer class. This implies that an instance of an inner class can only exist within an instance of its outer class and has direct access to the methods and fields of its enclosing instance.

Figure 2.12. Example of an inner class in Java.

```

class OuterClass {
    private final String name;
    public OuterClass(String name) { this.name = name; }
    class InnerClass {
        public void print() { out.println("Outer name: " + name); }
    }
}

```

In the example of [Figure 2.12](#) the `InnerClass` has access to the instance members of the `OuterClass`, such as the field `name`, which is accessed within the `print()` method. Accessing `name` is equivalent to `OuterClass.this.name`, where the prefix `OuterClass` is used to distinguish between the reference to its own instance, denoted by `this`, and the reference to the enclosing instance. [Figure 2.13](#) presents its equivalent implementation in Kotlin. In this case, accessing the `name` property of `OuterClass` from within the inner class `InnerClass` is equivalent to using `this@OuterClass.name` in Kotlin, similar to `OuterClass.this.name` in Java. Note that while Java assumes `InnerClass` to be an inner class by default, its Kotlin counterpart requires the `inner` keyword to grant `InnerClass` access to the instance scope of `OuterClass`.

Figure 2.13. Example of an inner class in Kotlin.

```

class OuterClass(private val name: String) {
    inner class InnerClass {
        fun print() {
            println("Outer name: ${name}")
        }
    }
}

```

The connection between the inner and enclosing instances is facilitated by an additional field automatically generated by the compiler within the inner class. When inspecting the `OuterClass$InnerClass.class` file using the `javap -p` command, you will notice the presence of a field named `this$0` of type `OuterClass`. This field is initialized within the `InnerClass` constructor, with the value passed as a parameter representing the enclosing instance (`OuterClass` in this case).

```

class OuterClass$InnerClass {
    final OuterClass this$0;
    OuterClass$InnerClass(OuterClass);
}

```

Thus, to instantiate an inner class, you must first create an instance of the outer class. Then, you can create the inner object within the outer object using the following syntax in Java programming language:

```
OuterClass outerObject = new OuterClass("ISEL");
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

2.2: Kotlin to Java type system

Kotlin ensures *null safety* by establishing two distinct type universes—*nullable* and *non-nullable*. All types declarations, whether built-in or user-defined, inherently create *non-nullable* types. In other words, these types cannot store a `null` value at runtime. To specify a *nullable* version of a type `T`, one must use `T?` as the type. **Note that in Kotlin, a nullable type corresponds to a reference type in the JVM, whereas a non-nullable type may or may not be a primitive type in the JVM.**

While certain types have an optimized internal representation as primitive values at runtime (such as numbers, characters, booleans, and others), the corresponding *nullable* versions of these types cannot directly hold primitive values. Instead, they store instances of the corresponding wrapper class, particularly in the case of the JVM. Thus we may establish the following correspondence between Java wrapper classes and Kotlin built-in types:

Java Wrapper class	Kotlin Type
Boolean	Boolean?
Byte	Byte?
Character	Char?
Float	Float?
Integer	Int?
Long	Long?
Short	Short?
Double	Double?

2.2.1: Kotlin Class Members

As indicated in the Type Fundamentals section, there are **two fundamental kinds of members**: *fields* and *methods*. Therefore, any other type of member construction available in Kotlin will be translated to one of these fundamental entities—either a *field* or a *method*—when compiled to JVM bytecode.

Referring to the example of the Java class `Person` in Section 1.2, we can rewrite it in Kotlin using the following single-line definition:

```
class Person(val name: String)
```

Considering the fundamental JVM rule that acknowledges only two types of members: *fields* and *methods*, what members are generated from the previous Kotlin class `Person` definition? We can examine the compilation result using `javap -p Person.class`, which indicates the presence of a field `name`, a getter method `getName()`, and a constructor `Person(String)`, as illustrated in the following output:

```
public final class pt.isel.Person {
    private final java.lang.String name;
    public pt.isel.Person(java.lang.String);
    public final java.lang.String getName();
}
```

Thus, the property name in the definition of the Kotlin class `Person` corresponds to the constructor's parameter `name`, a field `name`, and the method `getName()`. If we make the property name mutable by using the `var` keyword instead of `val`, the compiler will also generate a `setName(String)` method with a default implementation that updates the field `name` with the provided value. Nevertheless, developers have the flexibility to override the default setter behavior with a custom implementation. In the following example, we have made the property name mutable, restricting it to accept values with a maximum length of 70 characters.

```
class Person(name: String) {
    var name: String = name
    set(value) {
        require(value.length <= 70)
        field = value
    }
}
```

However, a property in Kotlin does not always generate a backing field. A property with a custom getter will not produce the default backing field. In the case of the property `area` in the following class `Rectangle`, there is no corresponding backing field, and its value is calculated from the two other properties, `width` and `height`:

```
class Rectangle(val width: Int, val height: Int) {
    val area get() = width * height
}
```

In Kotlin, top-level declarations are mapped to members of a class corresponding to the name of the owner file with the suffix `.kt`. For instance, consider the following top-level declarations in the file `App.kt`:

```
const val DEFAULT_TIMEOUT = 5000
const val APPLICATION_NAME = "MyApp"

fun isEven(number: Int): Boolean {
    return number % 2 == 0
}
```

The previous top-level declarations generate two static fields `DEFAULT_TIMEOUT` and `APPLICATION_NAME`, as well as a static method `isEven()` in a class named `AppKt`. This is illustrated in the output of `javap -p AppKt.class`:

```
public final class pt.isel.AppKt {
    public static final int DEFAULT_TIMEOUT;
    public static final java.lang.String APPLICATION_NAME;
    public static final boolean isEven(int);
}
```

Alternatively, while keeping `isEven` as a top-level declaration, we can redefine it as an extension function of `Int` as follows:

```
fun Int.isEven(): Boolean {
    return this % 2 == 0
}
```

This new implementation enables us to directly call `isEven` on an `Int` expression as the receiver, for example, `nr.isEven()`. Yet, there is no difference in the generated bytecode implementations of `isEven`, which remains a static method of the `AppKt` class. The only distinction lies in the metadata of `isEven`, indicating that it is an extension method of `int`.

2.3: Conversions and Casting

Conversions of values or references on the JVM can be grouped into three main categories, depending on the types involved:

1. Primitive numeric conversions,
2. Reference type casting,
3. Boxing and unboxing.

Each category has distinct compile-time rules and runtime behavior.

2.3.1: Primitive Numeric Conversions

For numeric primitive types, the JVM supports **widening** and **narrowing** conversions.

- A **widening conversion** maps a smaller range type to a larger range type, e.g., `int` to `long`.
- A **narrowing conversion** may lose information, e.g., `long` to `int`, and therefore requires an explicit conversion.

In Java programming language, widening numeric conversions can be implicit whereas narrowing conversions require an explicit cast:

```
int i = 7;
long l = i;           // widening conversion
int j = (int) l;      // narrowing conversion
```

In Kotlin, numeric conversions are always explicit, whether widening or narrowing, to avoid accidental assignments. Despite the differences to the Java example, both produce similar bytecode including conversion opcodes for widening and narrowing as will be presented in Chapter 5. Next, we have the equivalent Kotlin code for primitive numeric conversions:

```
val i: Int = 7
val l: Long = i.toLong()
val j: Int = l.toInt()
```

2.3.2: Reference Type Casting

Reference-type conversions are performed by **casting**.

- **Upcast** (derived type to base type) is implicit and always safe.
- **Downcast** (base type to derived type) is explicit and verified at runtime.

In [Figure 2.14](#), we have an example of upcasting and downcasting in Java. The upcast from `String` to `Object` is implicit and always safe, while the downcast from `Object` to `String` is explicit and requires a runtime check to ensure that the object being cast is indeed an instance of `String`. If the downcast is successful, it will succeed at runtime; otherwise, it will throw a `ClassCastException`. The commented-out line demonstrates an invalid downcast that would fail at runtime.

Figure 2.14. Example of upcasting and downcasting in Java.

```
String s = "ISEL";
Object obj = s;           // implicit upcast
String s = (String) obj;   // downcast, checked at runtime successfully
// Integer n = (Integer) obj; // downcast, throws ClassCastException at runtime
```

Because an upcast is always safe it only involves copying a reference, and no runtime checks are needed. On the other hand, a downcast may fail at runtime if the actual type of the object being referenced is not compatible with the target type of the cast. Therefore, a downcast requires an explicit cast operator and a runtime check to ensure type safety. This runtime check is a result of the `checkcast` opcode in the generated bytecode, which verifies that the object being cast is an instance of the target type. If the check fails, a `ClassCastException` is thrown.

In Kotlin, as performs a checked cast equivalent to the casting operator in Java and produces similar bytecode with the `checkcast` instruction. Next in [Figure 2.15](#), we have an equivalent example in Kotlin for reference type casting.

Figure 2.15. Example of upcasting and downcasting in Kotlin.

```
fun main() {
    val s: String = "ISEL"
    foo(s) // implicit upcast to Any
}

fun foo(obj: Any) : Int{
    val s: String = obj as String // explicit downcast, checked at runtime
    return s.length
}
```

Kotlin also provides a safe cast operator `as?` that returns `null` if the cast is not successful, instead of throwing an exception. This allows for safer handling of casts that may fail at runtime, as presented in the example of [Figure 2.16](#).

Figure 2.16. Example of safe casting in Kotlin.

```
val n: Int? = obj as? Int. // explicit downcast, returns null on failure
```

2.3.2.1: Type Erasure, and `checkcast`

On the JVM, generic type arguments are **erased** at runtime. For example, `List<String>` and `List<Int>` are both represented simply as `List` at runtime. This means that the JVM does not retain information about the specific type arguments used in generic types. In this example, it looks like

`List` is a **raw type**, equivalent to `List<Object>`. Because of this, the JVM cannot fully validate generic arguments when a list is cast; validation is deferred to the moment the values are accessed or read.

In the example of [Figure 2.17](#), we have a `List<String>` that is assigned to a raw `List` reference. This raw reference allows adding an integer to the list, which is an unchecked operation. When we later attempt to access the first element of the list as a `String`, a runtime cast check occurs, and if the element is not a `String`, it will throw a `ClassCastException`. This illustrates how type erasure leads to unchecked casts when working with generics in Java.

Figure 2.17. Example of unchecked casting with generics in Java.

```
List<String> names = new ArrayList<>();
List raw = names;
raw.add(123);           // unchecked operation
String first = names.get(0); // runtime cast check occurs here
```

When compiling `names.get(0)`, the generated bytecode includes a `checkcast` opcode to convert the erased `Object` result to `String`.

Kotlin behaves similarly on the JVM and also generates a `checkcast` opcode for the statement `val first = names[0]` in the example of [Figure 2.18](#). This is because the type of `names` is `MutableList<String>`, and when we access an element from this list, the JVM needs to ensure that the retrieved object is indeed a `String`.

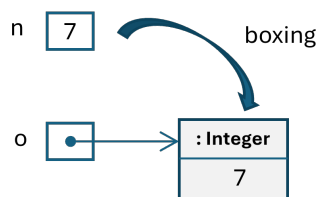
Figure 2.18. Example of unchecked casting with generics in Kotlin.

```
val names = mutableListOf<String>()
val raw = names as MutableList<Any> // warning unchecked cast
raw.add(123);           // unchecked operation
val first = names[0]; // runtime cast check occurs here
```

2.3.3: Boxing and Unboxing

Rather than holding a *reference*, a *primitive* type variable holds its **value** directly in-place. When we copy the value of a primitive type to a variable of a reference type, we need to place that value into an object and obtain a reference to that object. This process is called **boxing**.

The [Figure 2.19](#) illustrates an example of the JVM boxing process, where a primitive `int` variable `n` containing the value 7 is assigned to a variable `o` of type `Object`. The primitive value exists as a raw value in memory; during the boxing operation, the JVM wraps it into a heap-allocated `Integer` object. As shown, the variable `o` (of type `Object`) holds a reference to this newly created instance (denoted as `:Integer`) and the stored value 7.

Figure 2.19. Boxing an `int` value into an `Integer` instance.

To that end, for each Java primitive type there is a corresponding Wrapper class according to the [Table 2.1](#).

Table 2.1. Mapping of JVM primitive types to their corresponding wrapper classes.

Primitive type	Wrapper class
<code>boolean</code>	<code>Boolean</code>
<code>byte</code>	<code>Byte</code>
<code>char</code>	<code>Character</code>
<code>float</code>	<code>Float</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>short</code>	<code>Short</code>
<code>double</code>	<code>Double</code>

For example, passing an integer value to a function `foo` that receives an argument of type `Object` in Java (or `Any` in Kotlin) involves an **implicit** conversion of that value to an instance of the corresponding wrapper class `Integer`. This implicit conversion exists in both Java and Kotlin examples in the listings of [Figures 2.20](#) and [2.21](#).

Figure 2.20. Implicit boxing passing a primitive integer value as an `Object` argument in Java

```
foo(7)
static void foo(Object obj) { ... }
```

Figure 2.21. Implicit boxing passing a primitive integer value as an `Object` argument in Kotlin

```
foo(7)
fun foo (obj: Any) { ... }
```

Although you are passing a primitive type value in the `foo(7)` call, instead of an `Integer` object, the code compiles successfully. The compiler does not generate an error because it automatically creates an `Integer` object from the literal `7`. Thus, both the Java and Kotlin compilers convert the `foo(7)` code to bytecode equivalent to `foo(Integer.valueOf(7))`.

Converting a primitive value into an object of the corresponding wrapper class (e.g., `Integer`) is known as *autoboxing*. The Java compiler performs autoboxing in the following scenarios:

- When a primitive value is passed as a parameter to a method that expects an object or a corresponding wrapper class instance.
- When a primitive value is assigned to a variable of the type object or the corresponding wrapper class.

When performing arithmetic operations with instances of wrapper classes, it is necessary to convert them to the corresponding primitive type values. This process is known as **unboxing**. In the following `foo` method, the argument `n` must first be converted to an integer primitive type before performing an addition operation with 7.

```
static Integer foo(Integer n) {  
    return n + 7; // <=> Integer.valueOf(n.intValue() + 7)  
}
```

Additionally, since `foo` returns an `Integer`, there is also **boxing** of the result of the addition operation. The Java compiler translates the body of `foo` into bytecode equivalent to `return Integer.valueOf(n.intValue() + 7);`.

The same occurs in Kotlin, where the statement `return n + 7` encompasses an implicit unboxing and boxing process:

```
fun foo(n: Int?) : Int? {  
    require(n != null)  
    return n + 7  
}
```

2.4: Abstract and Base Types

Abstract classes cannot be directly instantiated. Interfaces, like abstract classes, represent abstract types that cannot be instantiated too. The main distinction between abstract classes and interfaces today is that the former can declare instance fields, while the latter cannot.

Let's compare the following Java definition of an abstract class with a single abstract method with the equivalent definition of that same type as an interface, both for Java in [Figure 2.22](#) and Kotlin in [Figure 2.23](#).

Figure 2.22. Definition of abstract class A and equivalent interface I in Java

```
abstract class A { public abstract void foo(); }
interface I { abstract void foo(); }
```

Figure 2.23. Definition of abstract class A and equivalent interface I in Kotlin

```
internal abstract class A { abstract fun foo() }
internal interface I { fun foo() }
```

The `foo()` method within the interface `I` is inherently abstract, so using the `abstract` keyword is optional in this case. In contrast, within the abstract class `A`, we must explicitly declare `foo()` as abstract using the `abstract` keyword. Otherwise, it should have a concrete implementation (a method body). To define a method with a body implementation in an interface, you should use the `default` keyword.

When analyzing the output from `javap` for resulting `.class` files, irrespective of whether their source code is defined in Java or Kotlin, we can verify that type `A` corresponds to an abstract class, and type `I` represents an interface. Both of these types feature an abstract method named `foo()`. However, it's important to note that while the abstract class `A` includes a default constructor, the interface `I` does not have any constructors.

```
abstract class A {
    A();
    public abstract void foo();
}
interface I {
    public abstract void foo();
}
```

To create an instance of an abstract Java type (be it an abstract class or an interface), we need a non-abstract class that inherits from that type. This non-abstract class must provide implementations for all the abstract methods declared directly in the type it's inheriting from, as well as any abstract methods inherited from all the base types.

In Java, there's a distinction in syntax for inheriting from a class or implementing an interface, which is indicated by using `extends` for classes and `implements` for interfaces, as demonstrated in the following example:

```
class CA extends A { public void foo() {} }
class CI implements I { public void foo() {} }
```

In Kotlin, a class that provides an implementation of inherited abstract methods **must** annotate those methods with the `override` keyword, as illustrated in the following example:

```
class CA : A() { override fun foo() {} }  
class CI : I { override fun foo() {} }
```

If a class inherits from an abstract type but doesn't provide implementations for all the mandatory abstract methods, it inherits those unimplemented methods and, as a result, the inheriting class itself must also be declared as abstract.

Note that interfaces are data types in Java, just like classes. When a class implements an interface, instances of that class can be assigned to variables of the interface type.

While a class can inherit from only a single class, it may implement multiple interfaces, allowing it to inherit and incorporate behavior and contracts from multiple sources. By implementing multiple interfaces, a class can take on various roles and fulfill diverse sets of functionality. This combination of single inheritance and multiple interface implementation is one of the foundations of object-oriented programming.

Classes in both Java and Kotlin that do not explicitly specify a base class inherit the default `java.lang.Object` as their base class. The choice of a base class allows the organization of common members shared among multiple types. When defining a class, you have the ability to determine whether other classes can inherit from it, or not. In Java, marking a class as `final` restricts it from being used as a base class. Conversely, in Kotlin, a class must be marked as open for other classes to inherit from it, as denoted in the example of [Figure 2.25](#) presented later.

The accessible members of a base type automatically become a part of the derived type. Methods within the derived type can interact with non-private members of the base type as if they were originally declared within the derived type itself. Occasionally, a situation may arise where a member in the derived type shares the same name as a non-private member in the base type. In such cases, both members exist in the derived type. If the member is a static one, you can use the type name to distinguish them. However, if the member is non-static, language-specific keywords like `this` or `super` are employed to specify whether to access the derived or base member, respectively.

Fields

Member access ambiguity is solved in a different way when looking from the outside. In the following example, both the base and the derived types have a `name` field. The `Student` constructor first calls the constructor of the parent class, `Person`, to initialize the `name` field in the `Person` class with `null`. Then, it initializes the `name` field of the `Student` class with the `name` parameter passed to the `Student` constructor.

```

class Person {
    final String name;
    public Person(String name) { this.name = name; }
}
class Student extends Person{
    final String name;
    public Student(String name) {
        super(null);
        this.name = name;
    }
}
public class Program {
    public static void main(String[] args) {
        Student s = new Student("Bart");
        Person p = s;
        System.out.println(s.name); // > Bart
        System.out.println(p.name); // > null
    }
}

```

In the `main()` function, both `s` and `p` reference the same object. However, due to the difference in variable types, these two variables perceive distinct **public contracts**. In the case of `s`, it adheres to the `Student`'s definition of the `name` field, which takes precedence over the definition in the base class. Consequently, `s` reads the `name` field defined in the `Student` class. On the other hand, when using the `p` variable, it is oblivious to any public contracts from derived types. It solely recognizes the base class, `Person`, and as a result, it accesses the `name` field belonging to the `Person` class.

The earlier explanation regarding name conflicts demonstrated the outcome when a field name is reused in a derived type. Dealing with conflicts in the context of method names, however, presents a different scenario, as method names can be reused already due to method overloading.

Methods

Formal parameters are defined in the method declaration, whereas **actual parameters** (also known as *arguments*) are the values passed to the method during its invocation. *Formal parameters* act as placeholders, while *actual parameters* provide the concrete data to be used by the method.

Two methods can share the same name if they differ in the number or types of their *formal parameters*. In such situations, these two methods are considered distinct members, and this mechanism is known as method **overloading**. Name resolution occurs at compile time and is based on the arguments (*actual parameters*) passed in the method call.

In the next example, the class `Truncate` contains two overloaded methods named `cut`. The first `cut` method without arguments returns the integer part of the stored double value (`nr`). It casts the double to an `int` effectively truncating the decimal part. The second `cut` method with an integer parameter `decimals` returns the double value with a specific number of decimal places.

Figure 2.24. class Truncate in Java

```

class Truncate {
    protected final double nr;

    public Truncate(double nr) { this.nr = nr; }

    public double cut() { return (int) nr; }
    public double cut(int decimals) {
        int d = 1;
        while(decimals-- > 0) { d *= 10; }
        return ((double) ((int) (nr * d))) / d;
    }
}

```

Figure 2.25. class Truncate in Kotlin

```

open class Truncate(protected val nr: Double) {
    open fun cut(): Double = nr.toInt().toDouble()

    open fun cut(decimals: Int): Double {
        var decimals = decimals
        var d = 1
        while (decimals-- > 0) { d *= 10 }
        return (nr * d).toInt().toDouble() / d
    }
}

```

When you call the `cut` method with an argument, the Java or Kotlin compiler determines which version of the method to invoke based on the type of the argument passed to it. If you pass no argument, the first method will be executed, and if you pass a double argument, the second method will be executed. This allows the same method name to be used for different data types.

On the other hand, a derived class may **override** the method of its base class by redefining the method in the subclass with the same **method signature** (including the **name**, the **number** and **types of its parameters**).

Method overriding is allowed only for non-final methods in Java (or open methods in Kotlin). In this case, when you have an instance of the derived type, the overridden method in the derived class will always be called, regardless of the actual type of the receiver.

Take, for instance, the following example of the `Round` class, which is derived from the `Truncate` class. While it inherits the `cut` methods from its parent class `Truncate`, it goes on to provide its own implementation, which returns a value being rounded.

```

class Round extends Truncate {

    public Round(double nr) { super(nr); }

    public double cut() { return (int) (nr + 0.5); }
    public double cut(int decimals) {
        int d = 1;
        while (decimals-- > 0) { d *= 10; }
        return ((int) ((nr * d) + 0.5)) / (double) d;
    }
}

class Round(nr: Double) : Truncate(nr) {
    override fun cut(): Double = ((nr + 0.5).toInt()).toDouble()

    override fun cut(decimals: Int): Double {
        var decimals = decimals
        var d = 1
        while (decimals-- > 0) { d *= 10 }
        return (nr * d + 0.5).toInt() / d.toDouble()
    }
}

```

In the next use case we have two distinct references, namely `tr` and `rd`, both pointing to the same instance of the `Round` class. When we invoke the method `cut`, it is always the implementation in the derived class that will be executed, irrespective of the type of reference being used.

```

final Round rd = new Round(73.5362);
final Truncate tr = rd;
out.println(tr.cut(2)); // 73.54
out.println(rd.cut(2)); // 73.54

```

2.5: Anonymous classes

As discussed in the previous section, abstract types, whether they are interfaces or abstract classes, cannot be directly instantiated. Instead, you require a concrete class that inherits from the abstract type and provides implementations for all its abstract methods.

Anonymous classes provide a way to write more concise code. You can declare and instantiate a class at the same time using an anonymous class, without the need to explicitly implement a named class and then instantiate it separately.

In previous section we have seen how to implement concrete named classes (e.g. CA and CI) that inherit from an abstract class or an interface, respectively. Once these classes are implemented we may then instantiate them through, e.g. CA `ca = new CA();` and CI `ci = new CI();`.

Named classes are defined using class declarations, while anonymous classes are expressed as class definitions within other expressions. Anonymous class expressions resemble constructor invocations but include a class definition enclosed within a block of code. Below, we provide examples of equivalent anonymous classes for CA and CI in [Figure 2.26](#) and also with Kotlin object expression in [Figure 2.27](#).

Figure 2.26. Anonymous classes in Java

```
public class App {  
    public static void main(String[] args) {  
        final A ca = new A() {  
            public void foo() {}  
        };  
        final I ci = new I() {  
            public void foo() {}  
        };  
    }  
}
```

Figure 2.27. Equivalent definition of former Java anonymous classes with Kotlin object expressions.

```
fun main() {  
    val ca: A = object : A() {  
        override fun foo() { }  
    }  
    val ci: I = object : I {  
        override fun foo() { }  
    }  
}
```

For each anonymous class, the compiler generates a class definition equivalent to what we've seen for CA and CI. When we inspect the generated .class files, we can find files named `App$1.class` and `App$2.class` for the Java anonymous classes definitions, and `AppKt$main$ca$1.class` and `AppKt$main$ci$1.class` for the Kotlin *object expressions*. The Java compiler names these classes using the name of the outer class followed by a suffix \$ and a cardinal number, starting from 1. In contrast, the Kotlin compiler adopts a similar approach but also includes the function and the variable name.

There is a slight difference in anonymous classes when they are defined within an instance method, rather than in a static method, like the `main` method seen in the previous class. In this case, the

anonymous class becomes an inner class and includes an additional field that holds a reference to the outer class instance.

Consider, for example, that we have now defined our anonymous classes as presented in [Figure 2.28](#) with the equivalent Kotlin implementation depicted in [Figure 2.29](#)

Figure 2.28. Anonymous class capturing outer state.

```
class Outer {
    private final int nr;
    public Outer(int nr) {this.nr = nr}
    public void bar() {
        A ca = new A() { public void foo() { out.println(nr); } };
        I ci = new I() { public void foo() { out.println(nr); } };
    }
}
```

Figure 2.29. Object expression capturing outer state.

```
class Outer(val nr: Int) {
    fun bar() {
        val ca: A = object : A() { override fun foo() { println(nr) } }
        val ci: I = object : I { override fun foo() { println(nr) } }
    }
}
```

If we inspect the generated classes, for instance, using `javap -private 'Outer$1.class'` or `javap -c -p 'Outer$bar$ca$1.class'` for the equivalent resulting class in Kotlin, we will find an additional field `this$0` that refers to the outer class `Outer`.

```
class Outer$1 extends A {
    final Outer this$0;
    Outer$1(Outer);
    public void foo();
}
```

In addition to being able to access the members of its containing class, an anonymous class is granted access to local variables within the surrounding scope. However, it's important to note that an anonymous class in Java can only access local variables in its enclosing scope if those variables are declared as either `final` or effectively final. The term “effectively final” is used in Java to describe a local variable that, while not explicitly labeled as `final`, behaves as if it were. In other words, it's a variable whose value remains immutable once it's initially assigned, even without the explicit `final` modifier.

In Kotlin, there is no such restriction, and an anonymous class can access any variable of the enclosing scope, regardless of whether it is immutable or not.

2.6: Kotlin Object declarations

Kotlin offers an efficient way to declare a singleton, a pattern that restricts the instantiation of a class and ensures that only one instance of the class exists [Gamma, 1993]. This is achieved through an *object declaration*, which always has a name following the object keyword. The rules for an object declaration are similar to a class definition, except that the keyword `object` replaces the keyword `class`. Let's examine the result of compiling the object definition of [Figure 2.30](#).

Figure 2.30. Object definition of an `ElapsedTimeCalculator`.

```
object ElapsedTimeCalculator {  
    private val startTime: Long = System.currentTimeMillis()  
  
    fun elapsedTime(): Long {  
        val currentTime = System.currentTimeMillis()  
        return currentTime - startTime  
    }  
}
```

Figure 2.31. Output of `javap` for `ElapsedTimeCalculator.class`

```
public final class pt.isel.ElapsedTimeCalculator {  
    public static final pt.isel.ElapsedTimeCalculator INSTANCE;  
    private static final long startTime;  
    private pt.isel.ElapsedTimeCalculator();  
    public final long elapsedTime();  
    static {};  
}
```

We can find, in the output of `javap -p ElapsedTimeCalculator.class` of [Figure 2.31](#), the declaration of the class `ElapsedTimeCalculator` featuring a parameterless constructor and two members corresponding to the `startTime` field and the method `elapsedTime`. Additionally, the output reveals a static field named `INSTANCE`, which holds the unique existing instance of `ElapsedTimeCalculator`. It is noteworthy that the parameterless constructor is marked as `private`, thereby prohibiting instantiation of this class outside of its own class. Lastly, the `static {}` block denotes the type initializer responsible for initializing static fields; in this case, it initializes the `INSTANCE` field.

The Java singleton implementation equivalent to the `ElapsedTimeCalculator` Kotlin definition in [Figure 2.30](#) will match the structure of the `javap` output shown in [Figure 2.31](#). In this case, the equivalent Java implementation is shown in [Figure 2.32](#).

Figure 2.32. Singleton definition of an ElapsedTimeCalculator in Java.

```

public class ElapsedTimeCalculator {
    // The singleton instance
    public static final ElapsedTimeCalculator INSTANCE;
    private static final long startTime = System.currentTimeMillis();

    static {
        INSTANCE = new ElapsedTimeCalculator();
    }

    // Avoid the instantiation of this class
    // out of the scope of this class.
    private ElapsedTimeCalculator() {
    }

    public long elapsedTime() {
        long currentTime = System.currentTimeMillis();
        return currentTime - startTime;
    }
}

```

A *companion object* is a specific type of *object declaration* that is associated with its owner class. A *companion object* can be anonymous, and its members act similarly to static members in Java. For instance, the Java class `Account` mentioned in [Figure 2.8](#) could be translated into Kotlin using the definition of [Figure 2.33](#).

Figure 2.33. Account class in Kotlin.

```

class Account {
    private val created: Long

    init {
        numberOfAccounts++ // <=> Account.nrOfAccounts++;
        created = currentTimeMillis()
    }

    companion object {
        var numberOfAccounts = 0
        private set
    }
}

```

In the output of `javap -p Account.class` presented in [Figure 2.34](#), we can observe the declaration of the static field `numberOfAccounts`, corresponding to the property of the companion object of the `Account` class. Additionally, another static field named `Companion` is present, which holds the singleton instance of the class `Account$Companion`.

Figure 2.34. Output of `javap` for `Account` class implementation in Kotlin.

```
public final class pt.isel.Account {
    public static final pt.isel.Account$Companion Companion;
    private final long created;
    private static int numberOfAccounts;
    public pt.isel.Account();
    public static final int access$getNumberOfAccounts$cp();
    static {};
}
```

The class `Account$Companion` encompasses the definition of the public API of the companion object of the `Account` class, specifically the corresponding getter `getNumberOfAccounts()` as denoted in the output of `javap` presented in [Figure 2.35](#)

Figure 2.35. Output of `javap` for the companion object of `Account` class.

```
public final class Account$Companion {
    private Account$Companion();
    public final int getNumberOfAccounts();
}
```

2.7: Kotlin Function types

Function types in Kotlin are denoted by a special notation that reflects the signatures of the functions, encompassing their parameters and return value. All function types consist of a parenthesized list specifying parameter types and a return type. For instance, `(A, B) -> C` signifies a type representing functions that accept two arguments of types `A` and `B` and yield a result of type `C`. The list of parameter types may be empty, denoted for example by `() -> A`. It's important to note that the `Unit` return type cannot be omitted.

Considering for example the `isEven` function defined in [Figure 2.36](#), we can express it as a **lambda** expression, which is one of the ways to create an instance of a function type, as shown in the following example. In this case, the lambda expression `{ n -> n % 2 == 0 }` is assigned to a variable `isEven` of type `(Int) -> Boolean`, which represents a function type that takes an `Int` as an argument and returns a `Boolean`.

Figure 2.36. Example of a `isEven` function defined as a lambda expression in Kotlin.

```
// App.kt file
fun main() {
    val isEven: (Int) -> Boolean = { n -> n % 2 == 0 }
}
```

There is a set of predefined generic **interfaces** in the Kotlin standard library, each corresponding to a specific kind of possible function type. These interfaces reside in the package `kotlin.jvm.functions` and are named `Function#`, where `#` indicates the number of arguments the function type takes. For instance, the interface `Function3` aligns with a function type that has 3 arguments. In [Table 2.2](#), we can find the list of interfaces corresponding to function types in Kotlin, along with the signature of their `invoke` method for the first 5 function types. The package `kotlin.jvm.functions` provides interfaces for function types with up to 22 parameters.

Table 2.2. Interfaces corresponding to function types in Kotlin.

Interface	invoke signature
<code>Function1<P1, R></code>	<code>invoke(P1): R</code>
<code>Function2<P1, P2, R></code>	<code>invoke(P1, P2): R</code>
<code>Function3<P1, P2, P3, R></code>	<code>invoke(P1, P2, P3): R</code>
<code>Function4<P1, P2, P3, P4, R></code>	<code>invoke(P1, P2, P3, P4): R</code>
<code>Function5<P1, P2, P3, P4, P5, R></code>	<code>invoke(P1, P2, P3, P4, P5): R</code>

For the example of `isEven` defined as a lambda expression in [Figure 2.36](#), the corresponding function type is `(Int) -> Boolean`, which corresponds to the `Function1<Int, Boolean>` interface. Therefore, the lambda expression `{ n -> n % 2 == 0 }` is translated into an instance of the `Function1` interface, which implements the `invoke` method with the logic defined in the lambda expression.

There are **two possible strategies** for generating an instance of a function type in the JVM.

Since **function types** are **interfaces** in the JVM and cannot be directly instantiated, the **1st approach** (used by the **Kotlin compiler during version 1.x**) involves translating lambda expressions into **anonymous classes** that implement the corresponding function type interface. This method is similar to how *object expressions* are handled in Kotlin (discussed in [Section 2.6](#)).

Since **Kotlin version 2**, the compiler adopts a different strategy to avoid generating an anonymous class at build time, instead deferring its creation to runtime using the `invokedynamic` opcode. As a result, the compiler generates only a method corresponding to the lambda expression.

For the example in [Figure 2.36](#), and analyzing the metadata of the resulting `AppKt.class` with `javap -p AppKt.class`, we may observe the output presented in [Figure 2.37](#), which includes a method named `main$lambda$0` that corresponds to the lambda expression `{ n -> n % 2 == 0 }`.

Figure 2.37. Output of javap for the isEven function defined as a lambda expression.

```
public final class AppKt {
    public static final void main();
    public static void main(java.lang.String[]);
    private static final boolean main$lambda$0(int);
}
```

The bytecode of the main function in the AppKt class then leverages `invokedynamic` in combination with `LambdaMetafactory` to obtain an instance of the `Function1` interface, which wraps the logic of `main$lambda$0`.

The details of this process are explained in Brian Goetz’s April 2012 report, “*Translation of Lambda Expressions*,” [Goetz, 2012].

2.7.1: Lambdas Translation in former Kotlin 1.x

Considering the example of [Figure 2.38](#) where the function `evenNumbers` is defined in an `App.kt` file, the Kotlin compiler will generate an additional class named `AppKt$evenNumbers$1` with the output from `javap` presented in [Figure 2.39](#).

Figure 2.38. Filtering even numbers from a sequence in Kotlin.

```
fun evenNumbers(numbers: Sequence<Int>): Sequence<Int> {
    return numbers
        .filter { it % 2 == 0 }
}
```

Figure 2.39. Anonymous class generated from the lambda expression in the evenNumbers function.

```
final class pt.isel.AppKt$evenNumbers$1
    extends kotlin.jvm.internal.Lambda
    implements kotlin.jvm.functions.Function1<java.lang.Integer, java.lang.Boolean>
{
    public static final pt.isel.AppKt$evenNumbers$1 INSTANCE;
    pt.isel.AppKt$evenNumbers$1();
    public final java.lang.Boolean invoke(int);
    public java.lang.Object invoke(java.lang.Object);
    static {};
}
```

Note that `AppKt$evenNumbers$1` class implements the `Function1` interface, which is designed for a single argument. It is parameterized with both the parameter type (i.e., `int`) and the return type (i.e. `Integer`). The class overrides the generic method with erased types: `Object invoke(Object)`, and it also provides an additional method specific to the argument type `int` and return type `boolean`. These methods implement the logic passed in the lambda to the `filter`, i.e., `it % 2 == 0`. Additionally, it's worth mentioning that this class represents a singleton instance stored in the `INSTANCE` field. This singleton can be reused across multiple usages, avoiding unnecessary instantiations.

Summary

This chapter provides a comprehensive understanding of type basics in the JVM, focusing on how types are handled in both Kotlin and Java. It begins by explaining the distinction between primitive and reference types. Primitive types, such as `int`, `boolean`, and `char`, store actual values directly in memory, which results in efficient performance. In contrast, reference types, such as classes, interfaces, and arrays, store references to objects, which incurs overhead due to object allocation and management.

The chapter then discusses **members**, which include **fields** and **methods** within classes and interfaces. **Fields are used to store data**, while methods define the behaviors that can be invoked on objects. The concept of **boxing** and **unboxing** is also covered, referring to the automatic conversion between primitive types and their corresponding wrapper classes. This process, known as autoboxing and autounboxing, is supported in both Java and Kotlin, allowing for seamless handling of primitive values and objects.

Nested types are explained in terms of how they are represented in compiled code. Nested types, such as inner classes or static classes, result in separate `.class` files during compilation. For example, an inner class within a class `Outer` is compiled into a file named `Outer$Inner.class`, following a naming convention where the `$` separates the outer class name from the nested class name. Each nested or inner class creates a new class file, which is important to understand when dealing with bytecode generated by Java and Kotlin.

The chapter then differentiates between **static** and **dynamic dispatch** when resolving members. **Static dispatch** occurs when the method to be invoked is determined at compile time. This is typical of methods in classes that do **not involve inheritance or virtual methods**, where the method call is resolved directly based on the reference type. On the other hand, **dynamic dispatch** occurs when the method to be invoked is determined at runtime, usually in the case where a **virtual method in a subclass overrides a method in a superclass**. Java and Kotlin both use dynamic dispatch for virtual methods, allowing the runtime to decide which method implementation to invoke based on the actual object type.

The chapter also covers anonymous classes, which are defined at the point of instantiation. These classes are often used for implementing interfaces or extending classes in a concise manner. In Java, anonymous classes are commonly used in event handling, and Kotlin provides similar functionality using lambda expressions and anonymous object declarations, allowing for more streamlined code.

Finally, the chapter discusses how Kotlin's type system integrates with Java's, ensuring full interoperability while addressing certain limitations of Java's type system. Kotlin introduces a more concise syntax for class members, such as properties and functions, compared to Java's getter and setter methods. Kotlin's object declarations simplify the creation of singletons using the `object` keyword, ensuring thread safety and proper initialization. Additionally, Kotlin supports function types, which allows functions to be treated as *first-class citizens*, a feature not natively supported in Java programming language, where functions are tied to objects and methods.

In summary, this chapter explores the key concepts related to types in Kotlin and Java, including primitive and reference types, boxing and unboxing, and how nested types are represented in bytecode. It emphasizes the distinction between static and dynamic dispatch in member resolution and provides insights into how Kotlin improves upon and interoperates with Java's type system.

2.8: Exercises

Exercise 1

Consider the Kotlin source code shown below, located in the file `Light.kt`. Indicate the names of the `.class` files where the definitions of `Adjustable`, `Light`, `DIMMED`, and `createLight` are stored after compilation.

```
interface Adjustable { fun adjust(diff : Int) }

class Light(private var intensity : Int) : Adjustable {
    override fun adjust(diff : Int) { intensity += diff; update() }
    private fun update() { }
}

object Candle { val light = createLight()}

val DIMMED = 15

fun createLight(intensity: Int = DIMMED) = Light(intensity)
```

Exercise 2

Sort the classes A, B, and C by the amount of space their instances occupy in the heap, providing justification. Consider that the space occupied by `Int` or `Float` is the same.

```
class A (val x: Int, val y: Int) {  
  fun modulus() : Float = sqrt(x.toFloat()*x + y*y)  
}  
  
class B (val x: Int, val y: Int, var modulus: Float = 0.0)  
  
class C (x: Int, y: Int, val modulus: Float = sqrt(x.toFloat()*x + y*y))
```

Exercise 3

Sort the classes A, B, and C by the amount of space their instances occupy in the heap, providing justification.

```
class A {  
  val x: Int = 999  
  val y: Int get() = 999  
  var z: Int = -1  
}  
class B {  
  val x: Int = 0  
  val y: Int = 999  
  val z: Int = -1  
}  
class C (  
  x: Int = 999,  
  y: Int = 999,  
  z: Int = 999  
)
```

Exercise 4

Consider the following definitions of types A, B, C, and I. Given their usage in the main function, answer the following questions:

1. Predict the output of the execution of the main function.
2. What would be the difference in the resulting output if we remove the `final` keyword from `virtualFoo` in class B?
3. Keeping the definitions of types A, B, C, and I unchanged, what would be the result of adding the following implementation in class C?

- `public void virtualFoo() { out.println("C"); }`

4. Provide an equivalent implementation in Kotlin for types A, B, C, and I, with only the method `virtualFoo` to achieve the same result as in question 1. Do not include the static method `foo`.

```
interface I { void virtualFoo(); }
class A {
    public static void foo() { out.println("A"); }
    public void virtualFoo(){ out.println("B"); }
}
class B extends A implements I {
    public static void foo() { out.println("B"); }
    public final void virtualFoo(){ out.println("B"); }
}
class C extends B {
    public static void foo(){ out.println("C"); }
}
public class App {
    public static void main(String[] args) {
        final C c = new C();
        final A a = c;
        final B b = c;
        final I i = c;
        a.foo();
        a.virtualFoo();
        b.foo();
        b.virtualFoo();
        c.foo();
        c.virtualFoo();
        i.virtualFoo();
    }
}
```

Chapter 3: Reflection

Reflection, also known as “Reflect,” is a feature present in all modern managed runtimes, such as Java, .NET, JavaScript, Python, and others. It empowers developers to craft customizable applications, offering the flexibility to adapt dynamically at runtime.

Many utility libraries, essential in software development, leverage the capabilities of reflection to implement various auxiliary features. Some prominent examples of these libraries that play a fundamental role in the daily routines of developers include loggers (such as Log4J and SLF4J), unit testing frameworks (such as Junit and TestNG), JSON parsers (such as Gson and Jackson), dependency injection frameworks (like Guice and Spring), among many other examples.

However, keep in mind:

reflection may have some performance overhead and is generally best used sparingly when necessary.

In this chapter, we will develop straightforward examples of such libraries to showcase the capabilities of Reflection in the JVM, employing both the Kotlin and Java programming languages. Additionally, we will explore various strategies to mitigate the performance overhead associated with reflection.

Reflection operates as an API over metadata. While Kotlin is interoperable with Java, it also introduces its own language constructs, such as nullable types, extension functions, and other elements, which may not map directly onto Java’s reflection API. While it is possible to use Java Reflection API in Kotlin, it would likely lead to high complexities. By providing its own Reflection API, Kotlin can better cater to its language features.

Reflection is a standard feature in Java and is accessible by the Java compiler. In contrast, when it comes to the Kotlin compiler, reflection is treated as a separate artifact, which is included in the `kotlin-reflect.jar`. This separation is designed to minimize the size of the runtime library for applications that do not make use of reflection features. If you’re working on a Gradle project and need to use reflection, you can add the `kotlin-reflect` dependency by including the Kotlin statement shown in [Figure 3.1](#) to your dependencies. Alternatively, you can use the appropriate snippet for another build system, such as those available from the [Maven Central Repository for kotlin-reflect](#)¹.

Figure 3.1. Gradle dependency on `kotlin-reflect`.

```
implementation(kotlin("reflect"))
```

¹<https://central.sonatype.com/artifact/org.jetbrains.kotlin/kotlin-reflect>

3.1: Kotlin Reflection

Reflection serves as an interface to metadata. Consequently, because metadata adheres to a type system, Reflection must also adhere to the same principles of the targeted type system.

As a result of the JVM type system, the central reflection unit in Kotlin is the `KClass` type. It often serves as the primary entry point for examining the contents of a JVM class file.

A `KClass` instance is a representation of a type, encompassing both primitive types and custom types defined by classes, interfaces, or other constructs.

To obtain a reference to a statically known Kotlin class, you can use the class literal syntax: `val c = MyClass::class`. You can also get the reference to the class of a specific object using the `::class` syntax, by using the object as a receiver. This allows you to obtain the reference to the exact class of an object.

From a `KClass`, we can examine its members using the `members` property, which provides a collection of `KCallable` objects. `KCallable<R>` serves as the common supertype encompassing all members and callable references, with `R` representing the return value type. In the example of [Figure 3.2](#), we demonstrate how to list all public members of a class in Kotlin.

Figure 3.2. Listing members of a class in Kotlin.

```
import kotlin.reflect.full.*

fun main() {
    Person::class
        .members
        .forEach { println(it) }
}
```

In Kotlin, there are two primary subtypes of `KCallable` that represent different kinds of members: `KFunction` and `KProperty`. **It's worth noting that a constructor is a specific instance of a function and is also represented by a `KFunction`.**

From a `KClass`, we can directly access specific kinds of members using its specific properties: `memberFunctions` or `memberProperties`. To obtain its constructors, we may use the property `constructors`, or if we specifically need the primary constructor, we can use `primaryConstructor`. Regardless of the kind of member, for each `getter` members, `memberProperties`, and `memberFunctions`, there is a corresponding property with the prefix `declared` (e.g., `declaredMemberProperties`), which includes only the members declared within the class itself, excluding inherited ones.

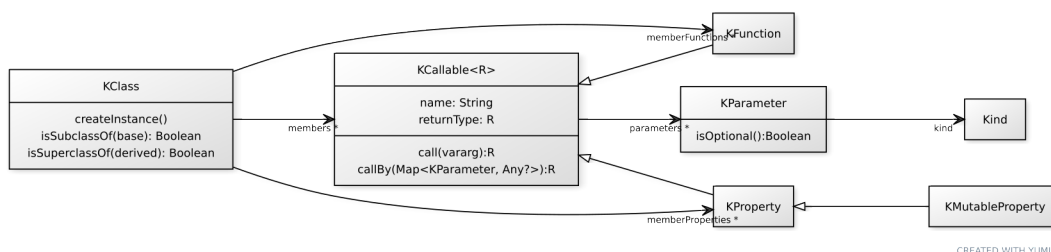


Figure 3.3. Kotlin Reflection API

Regardless of the kind of member, you may invoke it through the `call` method of its `KCallable` representation. The `call()` method invokes a member with the specified list of arguments and returns the result. Note that the arguments must be passed in the same order as the formal parameters of the function represented by the `KCallable`. Additionally:

If this callable requires a `this` instance or an extension receiver parameter, it should come first in the list in that order.

Consider for example the definition of the class `Utils` in Figure 3.4. Assuming you get a reference to the `KCallable` instance representing the `foo` method (e.g. `funcFoo`), take into account the several examples of invoking `foo` via reflection in the Listing of Figure 3.5. Note that the first parameter is an instance of `Utils`, corresponding to the receiver of the `foo` method. The remaining arguments must be supplied in the exact order and with the expected types defined in the method signature. Hence, the invocation `funcFoo.call(Utils(), 7, "hi", "world")` succeeds, as it matches the parameters (`Int`, `String?`, `String`). On the other hand, swapping argument types results in an argument type mismatch, and omitting arguments leads to an arity mismatch, since `call` requires all parameters to be explicitly provided, including those with default values.

Figure 3.4. Demo `Utils` class with a `foo` method

```

class Utils {
    fun foo(nr: Int, label: String? = "hello", msg: String) {
        println("$nr $label $msg")
    }
}

```

Figure 3.5. Retrieving the foo function via reflection and invoking it with valid and invalid argument combinations

```

val funcFoo: KFunction<*> = Utils::class
    .memberFunctions
    .first { it.name == "foo" }

funcFoo.call(Utils(), 7, "hi", "world")
// funcFoo.call(Utils(), "hi", 7, "world") // Argument type mismatch
// funcFoo.call(Utils(), 7, "world") // Callable expects 4 arguments

```

One question may arise from the previous example: how can an optional argument such as `label` be omitted when invoking `foo` via reflection? The answer lies in the use of the alternative `callBy` method. This approach not only resolves this issue but also allows arguments to be provided in any order, since each value is mapped to the corresponding `KParameter`.

The `callBy()` of `KCallable` takes a `Map<KParameter, Any>` where each entry associates an argument's value with its corresponding parameter. Using this information, the Kotlin reflection infrastructure correctly arranges the arguments before invoking the target function.

The example of [Figure 3.6](#) retrieves specific `KParameter` instances from the reflective representation of `funcFoo`, namely `nr` and `msg`, by filtering the function's parameter list by name. It also obtains the implicit receiver parameter via `instanceParameter`, which represents the `Utils` instance required to invoke the method. After that, the method is invoked using `callBy`, where arguments are provided as a map from `KParameter` to value. In this case, the arguments are intentionally supplied in a non-declared order—`msg`, `this` (the receiver), and `nr`—to illustrate that `callBy` resolves parameters based on their metadata rather than positional order, correctly binding each value to its corresponding parameter.

Figure 3.6. Invoking foo using callBy with explicit KParameter mapping and unordered arguments.

```

val pNr = funcFoo.parameters.first { it.name == "nr" }
val pMsg = funcFoo.parameters.first { it.name == "msg" }
val paramThis = funcFoo.instanceParameter
checkNotNull(paramThis)
funcFoo.callBy(mapOf(
    pMsg to "world",
    paramThis to Utils(),
    pNr to 7,
))

```

Note that you should use a `KCallable` representing a member only on the corresponding owner type. For example, this means that a `KCallable` representing a `country` property of a `Person` class may not be used to get the value of a property `country` from an object of the `Artist` class.

Consider, for example, the following function, `checkCountry`, that prints the value of the `country` property for objects of the `Person` class. Since `checkCountry` receives an argument of type `Any`, you can pass it any kind of parameter:

```
val countryProp: KCallable<*> = Person::class.members.first { it.name == "country" }

fun checkCountry(obj: Any) {
    println(countryProp.call(obj))
}
```

Although the statement `checkCountry(Artist("band", "Queen", country = "UK"))` compiles successfully, it throws an exception at runtime. Despite the `Artist` class having a property named `country`, it is not the same property represented by `countryProp`. The reflection API throws an `IllegalArgumentException` with the message “*object is not an instance of declaring class*” in this case.

Finally, we can **dynamically instantiate an object** of the type represented by a `KClass` using its `createInstance` method, which utilizes the parameterless constructor. Alternatively, we can use a `KFunction` representing one of its constructors to instantiate it with specific arguments.

3.2: A Simple Logger Using Kotlin Reflection

To demonstrate the utilization of Kotlin reflection, we will create an extension function called `Appendable.log(obj: Any)`. This function is designed to append information about the object’s type, property names, and their respective values to the specified `Appendable`. The `Appendable` can be any object capable of appending text, including a `StringBuilder`, a `PrintStream`, or similar text-handling objects.

In Listing of [Figure 3.7](#) we present an implementation of the `log` function. Initially, the Kotlin `KClass` representing the runtime type of the object is obtained using reflection (i.e. `val objClass = obj::class`). This allows us to determine the object’s class and, subsequently, retrieve its properties. Next, by invoking `objClass.memberProperties`, we collect all properties of the object’s class. Inside the loop, we continue to utilize reflection (`property.name` and `property.call(obj)`) to obtain the name and value of each property of the given `obj` object. To access private members, we also need to include `property.isAccessible = true` (line 8) to bypass JVM access checks.

Figure 3.7. log() function for properties in Kotlin

```
1 fun Appendable.log(obj: Any) {  
2     val objClass = obj::class  
3     appendLine("Type: ${objClass.simpleName}")  
4  
5     val properties = objClass.memberProperties  
6  
7     for (property in properties) {  
8         property.isAccessible = true  
9         val propertyName = property.name  
10        val propertyValue = property.call(obj)  
11        appendLine("- $propertyName: $propertyValue")  
12    }  
13 }
```

This log function can inspect objects of any type. For instance, consider a class like `Person(val name: String, val age: Int)`. If we call `System.out.log(Person("Alice", 30))`, we can expect to receive the following output::

```
Type: Person  
- age: 30  
- name: Alice
```

For a more complex domain model example, let's consider a class `Rectangle` as shown in [Figure 3.8](#). This class includes both properties in the primary constructor (i.e., `width` and `height`) and a property with a custom accessor (i.e., `area`). If we use `System.out.log(Rectangle(7, 9))`, we can anticipate the following output:

```
Type: Rectangle  
- area: 63  
- height: 9  
- width: 7
```

Figure 3.8. class Rectangle in Kotlin

```
class Rectangle(val width: Int, val height: Int) {  
    val area get() = width * height  
}
```

Considering the interoperability between Kotlin and any JVM-compliant module, we can employ our utility `log()` function with a domain model defined in the Java programming language. Therefore, let's consider the equivalent Java definition of a `Rectangle` class as illustrated in [Figure 3.9](#).

Figure 3.9. class Rectangle in Java

```
public class Rectangle {  
    private final int width, height;  
  
    public Rectangle(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    public int getWidth() { return width; }  
    public int getHeight() { return height; }  
    public int getArea() { return width * height; }  
}
```

Utilizing an instance of the `Rectangle` class from Figure 3.9 with the `log()` function can lead to a couple of issues. When running this example with `System.out.log(Rectangle(7, 9))`, the output will not include the area property.

Type: Rectangle

- width: 7
- height: 9

The absence of the area property in the output occurs because the Java compiler does not include any metadata indicating that member is a property. Consequently, the Kotlin Reflection API does not recognize the getter as a property.

In contrast, the Kotlin Compiler automatically adds the `@kotlin.Metadata` annotation to classes and members during compilation. Inside this annotation, there are fields that furnish essential information about the member, including its name, type, and kind. Specifically, for properties, the `kind` field specifies that it is indeed a property. This enables the Kotlin Reflection API to correctly identify and recognize the member as a property.

So, how does the Kotlin Reflection API regard Java definitions of `width` and `height` as properties? Kotlin Reflect relies on a blend of standards and conventions to recognize properties, even when dealing with classes that lack the `@kotlin.Metadata` annotation. In such cases, it can revert to utilizing standard Java reflection capabilities to examine the class's fields, which will be interpreted as Kotlin properties. That's why it misses the area as a property, as there is no corresponding field associated with it.

In the upcoming [Section 3.3](#), we will develop a new version of the `log()` function using the Java Reflection API to address the aforementioned limitations.

3.3: Java Reflection

In the Java Reflection API, we encounter a design concept similar to that presented for the Kotlin Reflection API, following the conventions of the Java Type System. In Java's reflection, there is a representation for types defined by the `Class` class, and another for members defined by the `Member` interface.

In the Java programming language, you can use the `.class` syntax to statically obtain the `Class` object for a specific class. On the other hand, to get the class of a specific object at runtime, you can invoke the `getClass()` method, which is inherited from the `Object` class and is available on all objects.

In the Java Reflection API, there are distinct implementations of the `Member` interface for each type of member, including `Field`, `Method`, and `Constructor`. It's worth noting that, unlike Kotlin `KFunction`, Java methods and constructors have separate representations, as they don't share the same type. Additionally, Java lacks a specific representation for properties because they don't exist at the metadata level or within the Java programming language itself. Instead, properties are a convention often implemented using getter and setter methods.

Finally, in Kotlin, we may use the `.kotlin` and `.java` getters to obtain references to the Kotlin and Java classes, respectively, that represent a specific type or class. The `.kotlin` getter provides a reference to the corresponding Kotlin `KClass` object, while the `.java` getter provides a reference to the corresponding Java `Class` object.

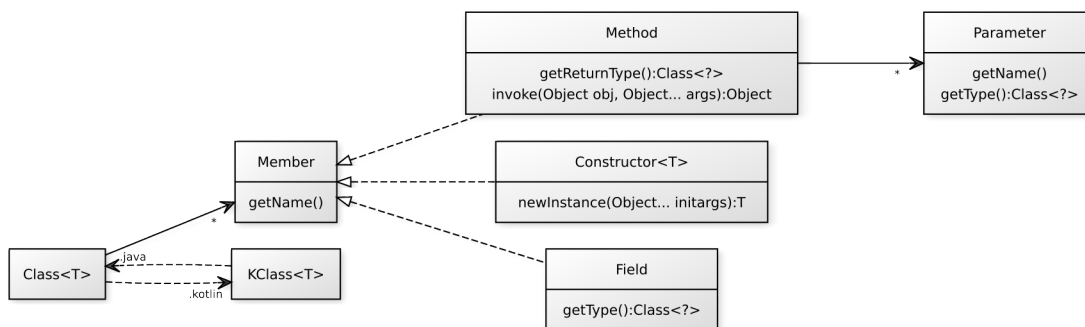


Figure 3.10. Java Reflection API

The invocation of a `Method` in Java's Reflection API is equivalent to the call of a `KCallable` in Kotlin's Reflection API. Both of these operations allow you to invoke a method or function dynamically at runtime. In Java, you use the `invoke()` method on the `Method` object, passing in the target object (or `null` for static methods) and the arguments. In Kotlin, you use the `call()` method on a `KCallable` object, providing the target object as the receiver and the arguments as parameters.

An alternative implementation of the `log()` function in Java, utilizing its Reflection API, follows a comparable approach: it initially obtains the `Class` representing the object's runtime type and

subsequently gathers all properties of the class. However, due to the absence of a direct mechanism in the Java Reflection API for retrieving properties, we need to adhere to a specific convention. To achieve this, we consider parameterless methods with non-void return types as properties. Additionally, we can simplify the property names by removing the “get” prefix typically associated with getter methods.

```
public class Logger {
    public static void log(Appendable out, Object obj) throws Exception {
        Class<?> clazz = obj.getClass();
        out.append("Type: " + clazz.getSimpleName() + lineSeparator());

        for (Method mth : clazz.getDeclaredMethods()) {
            if (mth.getParameterCount() == 0 && !mth.getReturnType().equals(void.class)) {
                String propName = mth.getName().replaceFirst("^get", "");
                Object propValue = mth.invoke(obj);
                out.append("- " + propName + ": " + propValue + lineSeparator());
            }
        }
    }
}
```

This Java version of the `log()` function is compatible with both Java and Kotlin definitions of `Rectangle`, accommodating the `area` property, and it does not require making the fields of the `Rectangle` class visible.

In the [Section 3.5](#) we will create a more complex utility function and explore additional features provided by the Java reflection API.

3.4: Checking Type Compatibility

Type compatibility between an object and a type in Java or Kotlin can be checked using the `instanceof` operator and the `is` operator, respectively. These operators allow verifying whether an object is an instance of a given type or implements a particular interface. Note that being compatible does not necessarily mean being an exact instance of a specific class, but rather that the object conforms to the required type through inheritance or interface implementation, making it safely usable in contexts expecting that type.

Sometimes it is necessary to perform such a verification without having an instance available, relying instead solely on reflective metadata. In these cases, type compatibility can be assessed using runtime type information (e.g., `Class` in Java or `KClass` in Kotlin) to determine whether a type is assignable to another.

Both languages provide methods for this kind of verification, namely `isAssignableFrom` in Java and `isSubclassOf` in Kotlin. These methods operate on type metadata rather than instances, allowing

developers to reason about type compatibility without object creation. Importantly, they are defined in opposite directions: in Java, `A.class.isAssignableFrom(B.class)` checks whether values of type `B` can be assigned to a reference of type `A`, while in Kotlin, `B::class.isSubclassOf(A::class)` checks whether `B` is a subtype of `A`. Despite this inversion in method direction, both ultimately express the same underlying relationship of type substitutability in the type hierarchy.

Considering, for example, the hierarchy defined between `I`, `A`, and `B` in Figure 3.11, the type compatibility checks performed in Java in Figure 3.12 are equivalent to those in Kotlin in Listing Figure 3.13, where all of them evaluate to `true`.

Figure 3.11. Example of a type hierarchy.

```
interface I
open class A : I
class B : A()
```

Figure 3.12. Java reflect type compatibility checks using `Class.isAssignableFrom`.

```
out.println(B.class.isAssignableFrom(B.class));
out.println(A.class.isAssignableFrom(B.class));
out.println(I.class.isAssignableFrom(B.class));
out.println(Object.class.isAssignableFrom(I.class));
```

Figure 3.13. Kotlin reflect type compatibility checks using `KClass.isSubclassOf`.

```
println(B::class.isSubclassOf(B::class))
println(B::class.isSubclassOf(A::class))
println(B::class.isSubclassOf(I::class))
println(I::class.isSubclassOf(Any::class))
```

In the Kotlin Reflection API, there is a distinct representation of a **fully resolved type** through `KType`. This corresponds, for example, to a type in which all **type arguments** (i.e., generics) are explicitly known.

Note that both `Class` (in Java) and `KClass` (in Kotlin) always refer to the **raw type** representation, without preserving type arguments. For instance, when obtaining the `KClass` representation of the generic Java type `java.util.List`, one can simply write `List::class` in Kotlin. Any attempt to use type arguments, such as `List<String>::class`, results in a **compilation error**.

On the other hand, a `KType` holds information about **nullability and type arguments**, commonly referred to as “*generics*”. The next figure in the book “*Kotlin Essentials*” [Moskala, 2022] illustrates the relationship between Types and Classes.

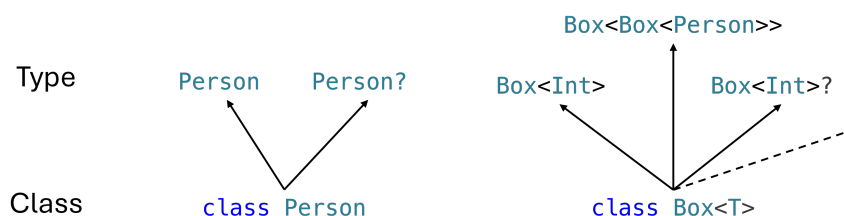


Figure 3.14. Relations between classes and types

The auxiliary function `typeOf` allows obtaining a `KType` instance that fully represents a type, including its nullability and type arguments, at compile time. It is typically used with reified type parameters, enabling expressions such as `typeOf<List<String?>>()` to capture complete type information that would otherwise be erased.

`KType` includes an auxiliary `isSubtypeOf` function, equivalent to the aforementioned `isSubclassOf` in `KClass`, but operating on fully resolved types. The `typeOf` function can also be used with non-generic types, and the previous example in Figure 3.13 can be expressed using the `KType` API, as shown in Section 3.4.

```
println(typeOf<B>().isSubtypeOf(typeOf<B>()))
println(typeOf<B>().isSubtypeOf(typeOf<A>()))
println(typeOf<B>().isSubtypeOf(typeOf<I>()))
println(typeOf<I>().isSubtypeOf(typeOf<Any>()))
```

The property `arguments` provides the type arguments of a `KType` instance. For example, for the `List<Int>` type, the `KType` instance will have a single argument of type `Int`, while for a `Map<Long, Char>`, it will have two type arguments: `Long` and `Char`.

Finally, there's the `classifier` property, serving as a means for a type to reference its associated class. Its return type is `KClassifier?`, which acts as a supertype of both `KClass` and `KTypeParameter`.

`KClass` represents a class or an interface, while `KTypeParameter` represents a generic type parameter.

The type of a classifier can be `KTypeParameter` when referencing generic class members. For instance, in a class `A<T>` with a member of type `T`, e.g., `val foo: T`, the corresponding classifier of the `KType` of the property `foo` would be a `KTypeParameter`.

When inspecting members of a type (i.e., properties or functions) using the Kotlin Reflection API, their types can be accessed through the `returnType` property, which is itself represented as a `KType`. This allows checking compatibility against other types. For example, if `prop` is a `KCallable` representing a property, its type compatibility with `OtherType` can be verified as follows:

```
prop.returnType.isSubtypeOf(typeOf<OtherType>())
```

The same verification can be performed using its `KClass`. Assuming the types are non-generic, the equivalent check can be written as follows:

```
(prop.returnType.classifier as KClass<*>).isSubclassOf(OtherType::class)
```

3.5: NaiveMapper: automatic object to object mapper

3.5.1: NaiveMapper Motivation

Our illustration, called `NaiveMapper`, takes inspiration from libraries like [AutoMapper](https://github.com/AutoMapper/AutoMapper)² or [MapStruct](https://github.com/mapstruct/mapstruct)³. These libraries simplify the process of mapping data between objects of different types by automatically copying values from properties or fields of one object to corresponding properties or fields of another object. This type of mapping proves particularly beneficial when dealing with *data transfer objects* (DTO), domain models, or view models [Fowler, 2012].

To illustrate the practical use of the `NaiveMapper` library, let's consider a hypothetical scenario: You are developing an application for managing music artists, and you need to gather information from various data providers, such as MusicBrainz, Spotify, Apple Music, and others. Despite the diversity of data sources, you will be working with the same domain type, namely, `Artist`, which includes properties such as `name`, `country`, and others. However, each data source may employ distinct terminology to represent these properties. For instance, the MusicBrainz artist entity might use the term `from` instead of `country`, while in Spotify, it might be referred to as `origin`. In light of these differences, you will manage separate DTOs for each data source, like `ArtistMb` and `ArtistSpotify`, and subsequently, you'll need to map these DTOs to your `Artist` domain entity.

²<https://github.com/AutoMapper/AutoMapper>

³<https://github.com/mapstruct/mapstruct>

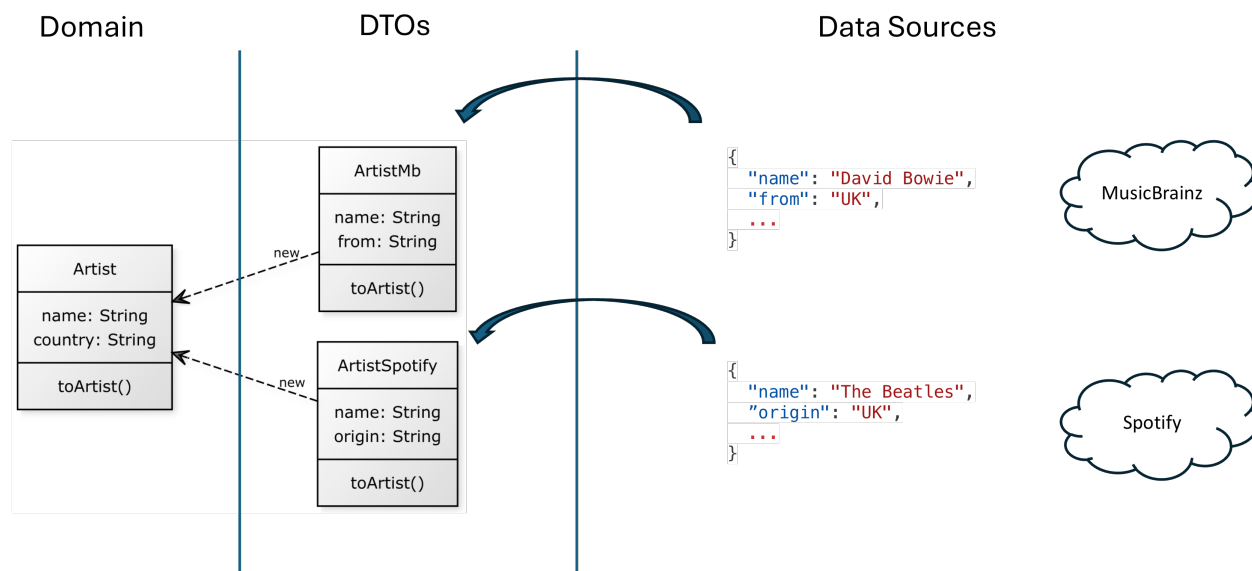


Figure 3.15. Auxiliary `toArtist()` functions to map DTOs to the `Artist` domain entity

Without the assistance of an automated mapping tool, you would find yourself duplicating code, as demonstrated in Figure 3.16. Furthermore, as the number of properties grows, the redundancy of writing similar code increases. This manual approach is also error-prone, and there is a potential for mismatches if, for instance, a developer alters the order of parameters in the `Artist` constructor.

Figure 3.16. Manual implementation of `toArtist()` extension function

```
fun ArtistMb.toArtist() = Artist(name, from, /* other properties */)
...
fun ArtistSpotify.toArtist() = Artist(name, origin, /* other properties */)
```

In the next section, we will implement the `NaiveMapper` utility to address the aforementioned issues. This utility is designed to be independent of any specific domain and can be reused for a wide range of scenarios. To achieve this, we will harness the power of Reflection mechanisms in our implementation.

In its most basic form, the `NaiveMapper` offers an extension function `Any.mapTo(dest: KClass<*>)`: Any that produces a new instance of the type represented by the `dest` class, populated with properties from the source object.

This function can be employed without the necessity of any boilerplate code, such that of Figure 3.16, and can be used for any DTO or domain model that is not known beforehand. For example: `val artist = artistMb.mapTo(Artist::class)` or `val artist = artistSpotify.mapTo(Artist::class)`, or any other DTO built in the future.

3.5.2: NaiveMapper Implementation

Before diving into the development of a fully-featured version, let's start with the most basic iteration of the `NaiveMapper`. This initial version adheres to the following constraints:

1. The destination type must have a parameterless constructor, indicating that all constructor parameters should be optional.
2. The destination properties are mutable.
3. The source and destination properties share the same name and type.

Our initial implementation of `mapTo()` shares a common approach with the `log()` method, commencing by identifying the runtime type of the object and then proceeding to analyze its properties. In this case, we need to collect properties from both the source object (i.e. `this`) and the destination type (i.e. `dest`). However, for the destination type, we are specifically interested in mutable properties. As a result, we selectively choose properties that are compatible with `KMutableProperty`. Subsequently, for each property, we assign the corresponding value to the newly created instance of the destination type (i.e., `val target = dest.createInstance()`).

Figure 3.17. Implementation of `mapTo()` extension function using Kotlin Reflection API

```
fun Any.mapTo(dest: KClass<*>): Any {
    val srcProps = this::class.memberProperties
    val destProps = dest
        .memberProperties
        .filter{ it is KMutableProperty<*> }
        .map { it as KMutableProperty<*> }
    val target = dest.createInstance()
    for (from in srcProps) {
        destProps
            .firstOrNull { it.name == from.name && it.returnType == from.returnType }
            ?.setter
            ?.call(target, from.call(this))
    }
    return target
}
```

Statements in lines 5 and 6 of Figure 3.17 can be combined into a single statement using `.filterIsInstance(KMutableProperty::class.java)`

As previously mentioned, **working with Reflection can introduce inherent overhead**. Upon revisiting the `mapTo()` implementation, we can identify redundant work when mapping objects of the same type consecutively. Considering, for example, that we are mapping a list of objects of the same type, each object will possess the same properties. Therefore, there is no necessity to repeatedly check for matching properties every time.

An alternative approach involves creating a dedicated mapper object for each use case. These mapper objects store the source and destination properties, which are later utilized in the `mapFrom()` method. For instance, you could create a mapper like: `val mapper = NaiveMapper(ArtistMb::class, Artist::class)`. In the implementation shown in [Figure 3.18](#), we integrated the initial `mapTo()` algorithm within the initializer block of the `NaiveMapper` class and its `mapFrom()` method. During initialization, it identifies matching properties between the source and destination types and stores the resulting pairs of `<KProperty<*>, Setter<*>>` in an auxiliary `Map` within the `props` field. Subsequently, the `mapFrom()` method merely needs to iterate over this map and retrieve the value of each property from the source object, assigning it to the corresponding property in the new object. This approach effectively mitigates the overhead of searching for matching properties within the `mapTo()` method.

Figure 3.18. Enhanced `mapFrom()` keeping auxiliary `Setter` instances.

```
class NaiveMapper<T : Any, R : Any>(src: KClass<T>, private val dest: KClass<R>) {
    private val props: Map<KProperty<*>, Setter<*>?>
    init {
        val destProps = dest
            .memberProperties
            .filter{ it is KMutableProperty<*> }
            .map { it as KMutableProperty<*> }
        props = src
            .memberProperties
            .associateWith { prop -> destProps
                .firstOrNull { it.name == prop.name && it.returnType == prop.returnType }
                ?.setter
            }
    }
    fun mapFrom(src: T) = dest.createInstance().also { target ->
        props.forEach { (from, to) ->
            to?.call(target, from.call(src))
        }
    }
}
```

Now, let's improve our `NaiveMapper` to handle **immutable domain entities** instead of mandating a parameterless constructor and mutable properties. With this improvement, we will **eliminate the previous constraints 1 and 2**. To achieve this, we will instantiate the destination type using a constructor whose parameters match the properties of the source in both name and type.

Therefore, the `props` field will contain a `Map<KProperty<*>, KParameter>` mapping the properties in the source type to the parameters of a constructor in the destination type (line 12). Additionally, we'll introduce another field `destInit` (line 2) to store the constructor that will instantiate the destination type (`dest`). We choose the constructor that contains all the required parameters, meaning those

that are not optional (i.e. `!param.isOptional` line 6), and are matched with the properties of the source class (i.e. `src`).

Finally, the `mapFrom` method will call the `destInit` constructor using `callBy()` (line 24), passing a `Map` with arguments for each parameter represented by a `KParameter` key.

Figure 3.19. Enhanced `mapFrom()` supporting immutable domain entities

```

1  class NaiveMapper<T : Any, R : Any> (src: KClass<T>, dest: KClass<R>) {
2      private val destInit = dest
3          .constructors
4          .first { it
5              .parameters
6              .filter { param -> ! param.isOptional }
7              .all { param -> src
8                  .memberProperties
9                  .any { param.name == it.name && param.type == it.returnType }
10             }
11         }
12     private val props: Map<KProperty<*>, KParameter> = destInit
13         .parameters
14         .associateBy { param -> src
15             .memberProperties
16             .firstOrNull { param.name == it.name && param.type == it.returnType }
17         }
18         .filterKeys { it != null } as Map<KProperty<*>, KParameter>
19
20     fun mapFrom(src: T) : R {
21         val args: Map<KParameter, Any?> = props
22             .entries
23             .associate { (srcProp, destParam) -> destParam to srcProp.call(src) }
24         return destInit.callBy(args)
25     }
26 }

```

In the next section, we will further enhance `NaiveMapper` by gathering additional information from annotations, facilitating correspondence between properties with different names and suppress the aforementioned constraint 3.

3.6: Annotations

Annotations in the context of the Java Virtual Machine (JVM) are a form of metadata that can be added to Java classes, methods, fields, and other program elements to provide additional information

about them. These annotations are used to convey metadata that can be processed by various tools and frameworks during compilation, build processes, or runtime. Annotations are a powerful way to extend the metadata of your code and can be used for a variety of purposes, including documentation, code analysis, and runtime behavior configuration.

Annotations themselves are strongly typed. Each annotation is an instance of a type that indirectly inherits from `java.lang.annotation.Annotation`. As an illustration, let's take the JUnit annotation `@Test`, whose `.class` file corresponds to the following type:

```
public interface org.junit.Test extends java.lang.annotation.Annotation { ... }
```

In this example, `org.junit.Test` is a type that extends the `java.lang.annotation.Annotation` interface, making it a strongly typed annotation.

The Reflection APIs in both Kotlin and Java provide various methods for locating annotations on annotated elements (such as classes, functions, fields, properties, and more). One of these methods (`findAnnotations(KClass)` in Kotlin and `getAnnotationsByType(Class)` in Java) accepts the annotation type as an argument and retrieves all annotations of the specified type associated with the given element. The annotations returned are represented as runtime-generated proxy objects that implement the interface defined by the annotation. For instance, you might encounter a generated proxy class like `class Proxy13 implements org.junit.Test ...`, which serves as a runtime representation of the `@Test` annotation.

In the context of `NaiveMapper`, we will utilize an annotation called `@Match` to enable developers to specify the corresponding property name in the destination type. For instance, in the `ArtistMusicBrainz` DTO, we should annotate the `from` property with `@Match(name = "country")`. This annotation informs `NaiveMapper` that the `from` property in `ArtistMusicBrainz` corresponds to the `country` property in the destination type:

```
class ArtistMusicBrainz(  
    @property:Match(name = "country") val from: String,  
    val kind: String, // "Band", "Person"  
    val name: String,  
)
```

The `:property` prefix preceding the annotation represents the use-site target. When a Kotlin member generates multiple Java members, there are multiple potential locations for the annotation within the generated Java metadata. If there are multiple applicable targets and you do not explicitly specify a use-site target, the target chosen will be the first applicable one from the following list: `param`, `property`, and `field`. To resolve this ambiguity, you can apply the use-site target to explicitly specify the destination location within the metadata.

Annotations in the JVM can be defined in either Java or Kotlin and are fully interoperable between these two programming languages. Next, we present two equivalent definitions of the `Match` annotation in both Kotlin and Java:

```
@Retention(AnnotationRetention.RUNTIME) // implicit
annotation class Match(val name: String)
```

```
@Retention(RetentionPolicy.RUNTIME) // required
public @interface Match { String name(); }
```

Note the presence of `@Retention` in both versions of the `Match` annotation. This annotation indicates that the `Match` annotation should be available at runtime, allowing it to be utilized by the Reflection API. In Kotlin, this behavior is implicit, and annotations are automatically retained with `RUNTIME`-like behavior. Therefore, the target annotation is visible through reflection at runtime without the need for explicit retention policy specification. However, in Java, without specifying `@Retention(RetentionPolicy.RUNTIME)`, the `Match` annotation would not be visible to the Reflection API at runtime.

The other `Retention` options available in Java and Kotlin are:

- `SOURCE`: Annotations are only available in the source code and are not included in the compiled bytecode or available at runtime. These annotations are typically used for code generation or static analysis tools.
- `CLASS` in Java or `BINARY` in Kotlin: Annotations are included in the compiled bytecode but are not available for reflection at runtime. They are primarily used for processing by bytecode manipulation tools or frameworks.

Additionally, you can specify the permissible types of program elements that can be annotated with an annotation (such as classes, functions, properties, and expressions) using the `@Target` annotation. For instance, if you want to restrict an annotation to be used exclusively on constructors, you should annotate it with `@Target(CONSTRUCTOR)`. It's worth noting that the set of available target elements specified with `@Target` may vary depending on whether you are working in Java or Kotlin.

Regardless of whether the `Match` annotation is defined in Java or Kotlin, in the `NaiveMapper`, we need to modify the predicate `{ param.name == it.name && param.type == it.returnType }` used in `any` and `firstOrNull` (lines 9 and 16 of [Figure 3.19](#)), to match either the name of the property or the name specified in the `Match` annotation. To that end, the predicate responsible for matching properties with constructor parameters will be defined as shown, for example, in the initialization statement of the `props` field presented in [Figure 3.20](#).

Figure 3.20. Initialize props field by matching constructor parameters with either the name specified in the property's annotation or the property's own name.

```

1 private val props: Map<KProperty<*>, KParameter> = destInit
2   .parameters
3   .associateBy { param -> src
4     .memberProperties
5     .firstOrNull { prop ->
6       val propName = prop.findAnnotation<Match>()?.name ?: prop.name
7       param.name == propName && param.type == prop.returnType
8     }
9   }
10 .filterKeys { it != null } as Map<KProperty<*>, KParameter>

```

In line 6 of the code snippet of Figure 3.20, `findAnnotation<Match>()` is used to find an annotation of the `Match` type associated with a property (i.e. `prop`). Next, it determines the name to be used for the property in the destination class. If a `Match` annotation is found (`match` is not `null`), it retrieves the name value from the annotation. If there is no `Match` annotation, it uses the name of the source property (`prop.name`) as the default name.

3.7: Reflection of generic types

In this section, we will extend the functionality of the `NaiveMapper` to handle properties of non-primitive, such as other domain types defined by classes. Following this, we will incorporate the ability to map properties corresponding to sequences of elements. To simplify matters, we will focus solely on properties of type `List`.

Addressing the first requirement, whenever we encounter a property with a type different from that of the destination constructor parameter, we will delegate the mapping process to an auxiliary instance of `NaiveMapper`. This auxiliary instance will handle the mapping of the source property type to the destination parameter type. It's important to note that this approach is **only applicable to non-primitive types**, as the transformation of primitive types is beyond the scope of this feature.

Consider, for example, the case where the origin property in `ArtistSpotify` is of type `State`, whereas the country property in `Artist` is of type `Country`.

```

class ArtistSpotify(
    @property:Match(name = "country") val origin: State,
    val kind: String,
    val name: String
)
class Artist(
    val kind: String,
    val name: String,
    val country: Country
)

```

So, the predicate used in lines 6 and 7 of the code snippet from [Figure 3.20](#), which is responsible for matching properties, becomes more complex and is therefore encapsulated in an auxiliary function named `matchPropertyToParam()`, as shown in [Figure 3.21](#). This function will initially retrieve the name of the source property through the `Match` annotation if present, or else it will use the property name itself. If this name does not match the destination parameter name, the matching process will terminate with a false result. Subsequently, if the property is of type `String` or a primitive in the JVM, it must have the same type as the destination parameter. Finally, for other reference types, we will assume that they are domain entities defined by classes, which can be mapped using an appropriate instance of the `NaiveMapper`. Thus, the matching process concludes with a true result.

Figure 3.21. `matchPropertyToParam` function that checks whether a property matches the given constructor parameter by name and type.

```

private fun matchPropertyToParam(prop: KProperty<*>, param: KParameter): Boolean {
    val propName = prop.findAnnotation<Match>()?.name ?: prop.name
    if(propName != param.name) {
        return false
    }
    val classifier = prop.returnType.classifier
    check(classifier is KClass<*>)
    if(classifier.java.isPrimitive || classifier == String::class) {
        return prop.returnType == param.type
    }
    return true
}

```

Note that the **Kotlin Reflect API** lacks information about whether a type is primitive, as this depends on the target platform and runtime environment. A primitive type is a fundamental data type that is natively supported by the runtime environment, without the need of a class file definition. Thus, the knowledge of whether a type is primitive or not depends on the runtime. For example, the `String` type may be considered primitive in a JavaScript runtime, while in the JVM, it is non-primitive.

In Kotlin's Reflection API, we must obtain the corresponding representation of the type in the Java Reflect API (i.e., `classifier.java`) to determine if it is primitive or not (i.e., using `isPrimitive`).

Next, we need to refine the approach used in the `mapFrom()` method of [Figure 3.19](#) for collecting the arguments that will be passed to the constructor in the following statement:

```
val args: Map<KParameter, Any?> = props
    .entries
    .associate { (srcProp, destParam) -> destParam to srcProp.call(src) }
```

We should parse the value retrieved from the property using `srcProp.call(src)` and transform it using an auxiliary `NaiveMapper` instance for non-String and non-primitive types. To accomplish this, we will delegate that task to an **auxiliary function named `parseProp()`**, which accepts the types of both the property and destination parameter, along with the property value. Thus, the **new version of `mapFrom()`** will follow the implementation shown in the listing of [Figure 3.22](#).

Figure 3.22. `mapFrom()` handling properties with types different from the destination constructor parameters.

```
fun mapFrom(src: T): R {
    val args: Map<KParameter, Any?> = props
        .entries
        .associate { (srcProp, destParam) ->
            val propValue = srcProp.call(src)
            destParam to parseProp(srcProp.returnType, destParam.type, propValue)
        }
    return destInit.callBy(args)
}
```

The `parseProp()` function initially checks if the property's value is `null`. In that case, it simply returns `null` since there is no mapping task to perform. Next, it checks if the property and the parameter share the same type. If they do, the function returns the original source value. However, if they differ, it retrieves the `KClass` of both the property and the parameter in order to instantiate the appropriate `NaiveMapper`, which is then used to transform the property value via a call to `mapFrom(propValue)`.

```
private fun parseProp(propType: KType, paramType: KType, propValue: Any?): Any? {
    if(propValue == null) {
        return null
    }
    if(propType == paramType) {
        return propValue
    }
    val propKClass = propType.classifier as KClass<Any>
    val paramKClass = paramType.classifier as KClass<Any>
    return NaiveMapper(propKClass, paramKClass).mapFrom(propValue)
}
```

In the context of `NaiveMapper`, we will assume that the classifier of the type of members will always be a `KClass`. To obtain the `KClass` of a property given its type `propType`, we can use the following code snippet:

```
propType.classifier as KClass<Any>
```

Now, we will tackle the second issue by extending the functionality of `NaiveMapper` to handle properties of type `List`. Let's consider a scenario where we have an `ArtistSpotify` class containing a list of songs that should be mapped to the `tracks` property of the `Artist` class:

Figure 3.23. `ArtistSpotify` and `Artist` depending on other domain entities.

```
class ArtistSpotify(
    @property:Match(name = "country") val origin: State,
    val kind: String,
    val name: String,
    @property:Match(name = "tracks") val songs: List<Song>
)
class Artist(
    val kind: String,
    val name: String,
    val country: Country,
    val tracks: List<Track>
)

```

In this scenario, we will once again require an auxiliary `NaiveMapper` instance. However, this time, it will not be for the type of the property (`List`) but for the type of the elements contained within the list (i.e., `Song` and `Track`). Therefore, instead of accessing the `classifier`, we need to obtain the arguments of the `KType`. For the sake of simplicity, we will only consider unidimensional lists. Once we retrieve the arguments, we can assume that we have a valid classifier for the class of the elements. To obtain the class of the elements of the list, given the property type (`propType`), we can use the following code:

```
val propElemClass = propType.arguments[0].type!!.classifier as KClass<Any>
```

Therefore, the **final version of the parseProp** function will include an additional branch to handle lists. In this branch, each element of the source list will be mapped using an auxiliary NaiveMapper according to the definition shown in [Figure 3.24](#).

Figure 3.24. parseProp() handling properties with types different from the destination constructor parameters.

```
private fun parseProp(propType: KType, paramType: KType, propValue: Any?): Any? {
    if (propValue == null) { return null }
    if (propType == paramType) { return propValue }
    val propClass = propType.classifier as KClass<Any>
    val paramClass = paramType.classifier as KClass<Any>
    if (propClass != List::class && paramClass != List::class) {
        val mapper: NaiveMapper<Any, Any> = NaiveMapper(propClass, paramClass)
        return mapper.mapFrom(propValue)
    }
    val propElemClass = propType.arguments[0].type!!.classifier as KClass<Any>
    val paramElemClass = paramType.arguments[0].type!!.classifier as KClass<Any>
    val mapper: NaiveMapper<Any, Any> = NaiveMapper(propElemClass, paramElemClass)
    return (propValue as List<*>).map {
        it?.let { mapper.mapFrom(it) }
    }
}
```

To conclude the implementation of the NaiveMapper, we will introduce an optimization to avoid the unnecessary instantiation of a new NaiveMapper inside the parseProp function (lines 7 and 12). It's worth noting that these auxiliary objects are collected by the garbage collector upon completion of parseProp(). To avoid it, we will make the constructor of NaiveMapper private and maintain instances of NaiveMapper in an internal map stored in the companion object. Access to this map will be **controlled by the invoke operator**, which is called as if we were invoking the constructor. This operator implementation will instantiate a new NaiveMapper **only when** there isn't already one for a pair of KClass objects corresponding to the source and destination types. The updated version of NaiveMapper conforms to the following definition:

Figure 3.25. Cache NaiveMapper instances in companion object

```

class NaiveMapper<T : Any, R : Any> private constructor(
    src: KClass<T>,
    dest: KClass<R>,
) : Mapper<T, R> {
    companion object {
        val mappers = mutableMapOf<Pair<KClass<*>, KClass<*>>, NaiveMapper<*,*>>()

        operator fun <T : Any, R : Any> invoke(
            srcType: KClass<T>,
            destType: KClass<R>,
        ): NaiveMapper<T, R> = mappers.getOrPut(srcType to destType) {
            NaiveMapper(srcType, destType)
        } as NaiveMapper<T, R>
    }
    ...
}

```

The rest of the `NaiveMapper` implementation remains unchanged. However, now when calling `NaiveMapper(...)`, the `invoke` operator of the companion object (as shown in [Figure 3.25](#)) will be triggered instead of directly instantiating a new object.

3.8: Reflection Overhead and the Enhancement of NaiveMapper

In the next Chapter 4, we will explore how to perform accurate performance measurements in detail. For now, however, we will use a state-of-the-art Java benchmarking tool—JMH—to analyze the performance impact of using reflection in the `NaiveMapper` implementation, without delving into the internals of JMH itself.

3.8.1: Reflection Overhead

As previously mentioned, reflection offers powerful capabilities for building applications and frameworks. However, it also introduces unavoidable performance overhead [Evans, 2023], which can affect the efficiency of your application. To measure and analyze the impact of reflection overhead, we need to understand two key aspects:

1. **How much overhead** is actually introduced by using reflection?
2. What is the **acceptable overhead** in the context of our application's performance requirements?

To answer these questions, we need to identify the **optimal implementation** that can serve as a **baseline** for comparison with a reflection-based alternative. In the case of object-to-object mapping, the ad-hoc implementation shown in [Figure 3.16](#) represents the most efficient solution, as it avoids any form of indirection or automation.

Second, when comparing the `NaiveMapper` with an ad-hoc alternative, we must also consider that `NaiveMapper` may introduce varying overhead depending on whether it is mapping objects with only simple properties—such as primitives or `String`—or objects that include associations with other objects. Thus, in our analysis, we will consider use cases involving three different domain entities, each with a distinct level of complexity:

1. The `Country` class contains two properties of type `String`.
2. The `Channel` class defines seven properties, all of which are either primitive types or `String`.
3. The `Artist` class includes four properties, one of which references an associated `Country` object, and another that holds a list of `Track` instances.

The definition of `Artist` follows the listing in [Figure 3.23](#), while the implementations of `Country` and `Channel` are presented in the listing of [Figure 3.26](#).

Figure 3.26. class `Country` and class `Channel`.

```
class Country(val name: String, val idiom: String)

class Channel(
    val name: String,
    val created: Long,
    val archived: Boolean,
    val maxChars: Int,
    val maxMembers: Int,
    val readOnly: Boolean,
    val lastMessageTimestamp: Long,
)
```

The ad-hoc implementation of object mappers for the entities shown in [Figure 3.26](#) follows the same approach as the manual `toArtist` implementation presented in [Figure 3.16](#). These implementations simply instantiate the destination class by passing the corresponding property values from the source object as constructor arguments. In the [Table 3.1](#), we present the **performance measurements in nanoseconds** for mapping the `Country`, `Channel`, and `Artist` entities using both the baseline and reflection-based approaches. Higher values indicate lower performance. The **overhead** is measured by dividing the performance using the `reflect` approach by the performance of the baseline approach.

Table 3.1. Performance in nanoseconds of NaiveMapper (lower is better).

Entity	Baseline (ns)	Reflect (ns)	Overhead (×)
Country	2.305	1001.868	434
Channel	2.744	3704.252	1380
Artist	15.958	5056.808	315

All results shown in the [Table 3.1](#) for the NaiveMapper using a reflective approach demonstrate performance levels that are **unsuitable for production-ready** projects. This approach should be reserved for testing, prototyping, or other experimental purposes only.

That said, the results still offer valuable insights worth considering. As the complexity of the entity increases, the performance of the NaiveMapper decreases. However, this decline is not proportional to the baseline approach. For example, mapping a Channel object with the baseline method results in only about a 15% overhead compared to mapping a Country object, while the NaiveMapper takes nearly 3.7 times longer to map a Channel than a Country. This overhead is nearly proportional to the increase in the number of primitive-type properties (recall the definitions of Country and Channel in [Figure 3.26](#)).

On the other hand, mapping objects with associations—such as an Artist—incurs a higher overhead due to the need to handle both nested objects and lists. In the baseline approach, for instance, mapping an Artist takes nearly six times longer than mapping a simpler entity like Channel, reflecting the added complexity of processing associated data. As a result, the relative overhead of the NaiveMapper compared to the baseline becomes slightly less pronounced, at around 315 times.

3.8.2: Enhancing the NaiveMapper

Our approach to addressing the reflection overhead is straightforward: **avoid using reflection—or at least minimize it as much as possible**. While reflection is still necessary to fulfill the goals of the NaiveMapper, we can reduce its usage, especially in methods that are likely to be invoked repeatedly throughout the application’s lifecycle.

Examining the structure of NaiveMapper, we see that its constructor is called only once per instance, while the same instance may be reused to map many objects. Therefore, we should concentrate the use of reflection within the constructor and minimize it in the mapFrom function.

Looking at the definition of mapFrom in [Figure 3.22](#), we conclude that there are two inevitable reflective operations:

1. Retrieving the value of a property from the source object, i.e., `srcProp.call(src)`
2. Instantiating the destination class with `destInit.callBy(args)`

However, there is a hidden type check performed in the auxiliary function `parseProp`, shown in [Figure 3.24](#), which is called from within `mapFrom`. This function repeatedly checks the correspondence between each source property and the destination constructor parameter, introducing unnecessary overhead. These checks could be executed once during the initialization of the `NaiveMapper`, rather than on every mapping call. Specifically, the `parseProp` function performs the following type verifications:

1. Whether the property type is equal to the constructor parameter type
2. Whether both the property and the parameter are not lists
3. If they are lists, it extracts the type parameters of those lists

To enhance the `mapFrom` function and eliminate the overhead of the aforementioned type checks, imagine associating each pair of `srcProp` and `destParam` with a function responsible for parsing the corresponding property value. In that case, the new version of `mapFrom` would follow the definition shown in [Figure 3.27](#). Note that when iterating over `props` (line 4), each entry includes three fields: `srcProp`, `destParam`, and `parseProp`.

Figure 3.27. New version of `mapFrom` function invoking a different `parseProp` for each pair of `srcProp` and `destParam`.

```

1 fun mapFrom(src: T): R {
2     val args: Map<KParameter, Any?> =
3         props
4         props.associate { (srcProp, destParam, parseProp) ->
5             val propValue = srcProp.call(src)
6             destParam to parseProp(propValue)
7         }
8     return destInit.callBy(args)
9 }

```

Now, there is a distinct `parseProp` function associated with each pair of `srcProp` and `destParam`. This function exhibits different behaviors depending on the types of the source property and the destination parameter. For instance, when both types match, the function simply returns the original value without any transformation.

These functions are constructed based on the same type checks previously performed in the original `parseProp` function. To achieve this, we have refactored the original `parseProp` into a new function called `buildParseProp`, which, **instead of directly parsing the property value, generates and returns a function capable of performing that parsing**.

Compared to the original `parseProp` of [Figure 3.24](#), the new `buildParseProp` function no longer takes `propValue` as a parameter as illustrated in [Figure 3.28](#). Instead, `propValue` becomes the argument of the lambda expressions constructed by `buildParseProp`, as illustrated in [Figure 3.29](#) (lines 3, 9 and 14).

```
private fun parseProp(propType: KType, paramType: KType, propValue: (Any?): Any?) {
```

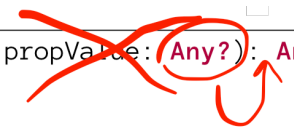


Figure 3.28. Refactoring parseProp so that propValue is passed as an argument to the returned lambda.

Figure 3.29. buildParseProp function constructs a new parsing function based on the types of the source property and the destination constructor parameter.

```
1 private fun buildParseProp(propType: KType, paramType: KType): (Any?) -> Any? {
2     if (propType == paramType) {
3         return { propValue -> propValue }
4     }
5     val propClass = propType.classifier as KClass<Any>
6     val paramClass = paramType.classifier as KClass<Any>
7     if (propClass != List::class && paramClass != List::class) {
8         val mapper = NaiveMapperEnhanced(propClass, paramClass)
9         return { propValue -> propValue?.let { mapper.mapFrom(propValue) } ?: null }
10    }
11    val propElemClass = propType.arguments[0].type!!.classifier as KClass<Any>
12    val paramElemClass = paramType.arguments[0].type!!.classifier as KClass<Any>
13    val mapper = NaiveMapperEnhanced(propElemClass, paramElemClass)
14    return { propValue -> (propValue as List<*>).map {
15        it?.let { mapper.mapFrom(it) }
16    } }
17 }
```

The buildParseProp() function is used during the initialization of NaiveMapper to construct a List<PropInfo>, which replaces the previous Map<KProperty<*>, KParameter>. Each PropInfo is a triple that associates a source property with a destination parameter and a parser function (i.e., (Any?) -> Any?), as defined in Figure 3.30.

Figure 3.30. PropInfo class associating a source property with a destination parameter and a parser function.

```
data class PropInfo(
    val srcProp: KProperty<*>,
    val destParam: KParameter,
    val mapPropValue: (Any?) -> Any?,
)
```

Finally, the props field in NaiveMapper is now initialized following the logic shown in the listing of Figure 3.31. It begins by iterating through each parameter of the destination constructor (i.e. destInit) and, for each one, searches for a matching property in the source class (i.e. src) using

the `matchPropertyToParam` function. If a match is found, a pair of the property and the parameter is created; otherwise, the result is discarded (i.e. `filterNotNull`). Only the valid pairs are kept, and each is then transformed into a `PropInfo` instance.

Figure 3.31. Initializing the `props` field in `NaiveMapper` with a list of `PropInfo` objects.

```
val props: List<PropInfo> = destInit
    .parameters
    .map { param -> src
        .memberProperties
        .firstOrNull { prop -> matchPropertyToParam(prop, param) }
        ?.let { it to param }
    }.filterNotNull()
    .map { (srcProp, destParam) ->
        val parseProp = buildParseProp(srcProp.returnType, destParam.type)
        PropInfo(srcProp, destParam, parseProp)
    }
}
```

Assessing the performance of the new version of `NaiveMapper`, we observe an improvement of approximately 7 to 10 times compared to the previous implementation, as shown in [Table 3.2](#). However, despite these gains, the performance is still significantly behind the baseline and remains unsuitable for use in real-world projects.

Table 3.2. Performance in nanoseconds of enhanced `NaiveMapper` (lower is better).

Entity	Baseline (ns)	Reflect (ns)	Overhead (×)
Country	2.305	131.879	57
Channel	2.744	368.374	134
Artist	15.958	612.658	38

Only in [Chapter 6 of this book](#) will we introduce a different approach that **completely eliminates the overhead of reflection** in the `mapFrom` method and achieves performance equivalent to the baseline.

Summary

This chapter provides an in-depth exploration of reflection in both Kotlin and Java, focusing on how it enables the inspection and manipulation of types and objects at runtime. Reflection is a powerful feature that allows developers to analyze and interact with code dynamically. The chapter begins by discussing Kotlin reflection, which is compatible with the JVM and allows for runtime inspection of

Kotlin classes, functions, properties, and types. Kotlin's reflection is based on the `kotlin.reflect` package, which enables accessing metadata about types and their members, including class names, annotations, properties, and functions.

After that, it presents the alternative Java reflection defined in the `java.lang.reflect` package, which allows developers to inspect metadata such as class, method, field, and annotation definitions.

The chapter then introduces a reflection-based example of an automatic object-to-object mapper called **NaiveMapper**. This tool is motivated by the need to map objects from one type to another without manually defining the mapping logic. **NaiveMapper** leverages reflection to inspect the source and target objects, matching properties by name and type, and automatically populating the target object's properties with values from the source object.

The implementation of **NaiveMapper** is discussed in detail, showing how reflection is used to dynamically access the members of source and target objects. The mapper performs type checks and utilizes the property names to ensure that data is correctly transferred between compatible objects. Different mapping strategies and optimization techniques are also explored. However, since this approach uses reflection extensively, it may have performance drawbacks, especially when used in performance-critical sections of an application.

The chapter also covers annotations, which are metadata that can be attached to code elements such as classes, methods, or fields. Annotations provide a way to store additional information about the code without affecting its execution. Reflection is often used to read these annotations at runtime, allowing frameworks and libraries to process them dynamically. The chapter highlights common use cases for annotations.

Finally, the chapter addresses the reflection of generic types. In both Java and Kotlin, **generics are typically erased at runtime**, meaning that the type information is not available through traditional means. However, both languages provide ways to reflect on generic types by using specialized methods or leveraging the type parameters passed at runtime. This is particularly useful in situations where the type of a generic class or function needs to be determined dynamically, such as when working with collections.

In summary, the chapter explains the fundamentals of reflection in Kotlin and Java, its practical applications, and how it can be used to build dynamic features such as object mappers and annotation processors. By examining both languages' approaches to reflection and generic type handling, it provides valuable insights into how reflection can be applied effectively in modern software development.

3.9: Exercises

Exercise 1

The function `matchProps` is a generic function that compares and matches the properties of two objects, which may be of different types. The function returns a list of pairs containing the values of the properties that share the same name in both objects.

Implement the function:

```
fun <T : Any, V : Any> matchProps(a: T, b: V): List<Pair<Any?, Any?>>
```

Exercise 2

The `@Copy` annotation allows specifying which properties of a class can be copied using the `copyTo` function. In the following example, the annotation indicates that only the properties `birth` and `name` can be copied:

```
@Copy(properties = ["birth", "name"])
class Person(val id: Int, val birth: LocalDate, val name: String, val address: String)
```

The function `copyTo(from: Any, to: Any)` copies the values of the properties identified by `@Copy` from the `from` object to the `to` object. These objects may be instances of different classes.

1. Implement the `@Copy` annotation.
 2. Implement the `copyTo` function in Kotlin. Assume that the class of the target object has properties with the same name and type, and with a public setter.
-

Exercise 3

You need to develop a class `ComparerOrder<T : Any>(klass: KClass<T>)` that implements `Comparator`⁴ and is capable of comparing instances of the type represented by `klass`, based on the properties that are **both** `Comparable` and annotated with `@Comparison`. Note that the comparison must respect the order specified in the annotation.

Example:

```
class Student (
    @Comparison(2) val nr: Int,
    val name: String,
    @Comparison(1) val nationality: String,
)
```

⁴<https://docs.oracle.com/javase/8/docs/api/java/util/Comparator.html>

```

val s1 = Student(12000, "Ana", "pt")
val s2 = Student(14000, "Ana", "pt")
val s3 = Student(11000, "Ana", "en")
val cmp = ComparerOrder(Student::class)
assertTrue { cmp.compare(s1, s2) < 0 } // same nationality and 12000 is < 14000
assertTrue { cmp.compare(s2, s3) > 0 } // "pt" is > "en"

```

Exercise 4

You need to develop a `Comparer<T : Any>(klass: KClass<T>)` that implements `Comparator`⁵ and is capable of comparing instances of the type represented by `klass`, based on properties that are either `Comparable` or annotated with a `Comparison` specifying a `Comparator` for that property, as shown in the following example:

```

class Person(
    val id: Int,
    val name: String,
    @Comparison(cmp = AddressByRoad::class) val address: Address,
    @Comparison(cmp = AccountByBalance::class) val account: Account) {
}

class AccountByBalance : Comparator<Account>{
    override fun compare(o1: Account, o2: Account): Int {
        return o1.balance.compareTo(o2.balance);
    }
}

class AddressByRoad : Comparator<Address> {
    override fun compare(o1: Address, o2: Address): Int {
        return o1.road.compareTo(o2.road)
    }
}

```

⁵<https://docs.oracle.com/javase/8/docs/api/java/util/Comparator.html>

```
val p1 = Person(11000, "Ana", Address("Rua Amarela", 24), Account("FD3R", 9900))
val p2 = Person(11000, "Ana", Address("Rua Rosa", 24), Account("8YH5", 9900))
val p3 = Person(11000, "Ana", Address("Rua Rosa", 24), Account("JK2E", 100))
val p4 = Person(11000, "Ana", Address("Rua Rosa", 97), Account("BFR5", 100))
val p5 = Person(17000, "Ana", Address("Rua Rosa", 97), Account("BFR5", 100))
val cmp = Comparer<Person>(Person::class)

assertTrue { cmp.compare(p1, p2) < 0 } // Rua Amarela is < Rua Rosa
assertTrue { cmp.compare(p2, p3) > 0 } // 9900 is > 100
assertEquals(0, cmp.compare(p3, p4)) // All properties are equal
assertTrue { cmp.compare(p4, p5) < 0 } // 11000 is < 17000
```

Chapter 4: Performance testing and Microbenchmarking

Microbenchmarking focuses on measuring the performance of small, isolated pieces of code or functionality, such as the `NaiveMapper` developed in Chapter 3 of this book. It is worth noting that `NaiveMapper` is domain-independent and can serve as one of the micro-components used to build broader systems, such as web applications or other software solutions. In contrast, **Benchmarking** evaluates the overall performance of an entire system, providing a comprehensive perspective on its efficiency and effectiveness.

An important aspect of microbenchmarking and striving for the best possible performance is having a **reference for the optimal performance** we aim to achieve. Note, for example, when we presented the motivation for the need for object-to-object mapping, the ad-hoc implementation of [Figure 3.16](#) is the most efficient solution, as it avoids any indirection or automation. Despite being redundant and leading to code repetition for each DTO (Data Transfer Object), this remains the optimal implementation in terms of performance. Thus, this serves as our reference point and will be used as the **baseline** when optimizing the `NaiveMapper`.

Keep in mind that when conducting microbenchmarking, an absolute performance metric is meaningless without a reference point, as **microbenchmarking assesses relative performance**.

4.1: Performance Measurement Pitfalls

Microbenchmarking is a delicate task that demands careful attention to **avoid biased analyses**, which can result in misguided optimizations. It is crucial to have a clear understanding of the target component and avoid attempting to optimize something without a thorough and precise knowledge of its internal workings; otherwise, incorrect conclusions may be drawn.

Avoid relying on guesswork. Instead, isolate the specific part you want to assess and strive to measure its performance as accurately as possible.

Remember the famous quote from Donald Knuth, in 1974 [Knuth, 1974]:

“Premature optimization is the root of all evil.”

Avoid focusing on optimization before you are confident in the reliability of your software component. Your primary goal should be to deliver a reliable, tested, and fully functional component. Only after achieving this foundation should you consider optimizing it.

Additionally, unit tests help establish a solid foundation. They provide confidence and ensure that if something breaks, it is likely due to your optimizations. While unit tests are not 100% reliable — as passing tests do not guarantee everything is functioning correctly — they assist in identifying the source of any issues when they fail, making it easier to pinpoint where the problem lies.

Next, we will highlight a couple of common errors when measuring performance. To demonstrate this, we will measure the performance of the `log()` utility function from [Figure 3.7](#).

To that end, let's start with a naive performance test that takes advantage of the built-in `measureTimeMillis()` function from the Kotlin standard library. This function accepts another function as an argument and returns the elapsed time in milliseconds it took to execute that function. It calculates the elapsed time by subtracting the value of `System.currentTimeMillis()` before executing the block function from its value after execution, as shown in [Figure 4.1](#).

Figure 4.1. `measureTimeMillis` implementation from Kotlin standard library.

```
public inline fun measureTimeMillis(block: () -> Unit): Long {  
    val start = System.currentTimeMillis()  
    block()  
    return System.currentTimeMillis() - start  
}
```

The first version of the performance test is shown in [Figure 4.2](#).

Figure 4.2. Naive implementation of a performance test.

```
measureTimeMillis {  
    System.out.log(Rectangle(7,9))  
}.also {  
    println("log(Rectangle) took $it ms")  
}
```

Next, we will analyze each of the potential flaws in this test, namely:

1. Microbenchmarking is not unit testing
 2. Avoid unrelated operations in the measurement
 3. First execution includes JIT compiler overhead and misses optimization
 4. Avoid IO
 5. Minimize the system call overhead
-

1. Microbenchmarking is not unit testing

We should avoid conducting performance tests within a unit testing environment. As discussed in this section, performance measurements need to be carefully designed and free of side effects. Unit testing frameworks like JUnit often rely on meta-programming techniques, such as the Reflection API, to inspect tests at runtime and gather status information, all of which introduce unavoidable overhead.

Measuring performance within a unit test environment can lead to undesirable costs, which may far exceed the performance of the component or function being tested, resulting in misleading conclusions.

Microbenchmarking should be conducted in an isolated environment, without the use of development tools like an IDE. It's recommended to run microbenchmarks from the command line and ensure that no other programs are running simultaneously. Before executing the benchmark, make every effort to ensure the CPU is not under unnecessary processing tasks and is as free as possible.

2. Avoid unrelated operations in the measurement

Isolating the test from [Figure 4.2](#) inside a `main` function and running it on my personal laptop, I observe an execution time of approximately 170 ms across multiple runs. **However, can we accurately conclude that the average execution time of our `log()` function implementation is 170 milliseconds?** No, we cannot.

We are not measuring only the execution of the `log` function but also the instantiation of the `Rectangle`. To obtain a more accurate measurement, we should instantiate the `Rectangle` outside the test scope and measure only the execution of the `log` function using a reference to the pre-instantiated `Rectangle`.

Although the instantiation cost of `Rectangle` may be negligible compared to the overhead of the `log` function, we must ensure that our test remains free of any external factors unrelated to the specific aspect we intend to measure.

3. First execution includes JIT compiler overhead and misses optimization

We should not rely on a single execution for performance measurement. Initially, during the first method execution, there is a performance penalty due to the JIT compiler translating the method's bytecode into machine code. This translation process can take longer than the operation we are trying to measure, potentially skewing the results.

Running the previous test inside a **loop of 10 iterations**, we can easily observe that the **time elapsed is 0 milliseconds** from the second iteration onward.

Thus, we do not have enough precision in milliseconds to accurately calculate the elapsed time. Therefore, we can modify our test to use `measureNanoTime` instead. With this change, we can observe the elapsed time decreasing across the iterations. The initial decrease is substantially higher than the subsequent ones. Nevertheless, we can still see the value decrease due to several potential optimizations that the JVM may apply during the execution process.

Running the new version of this test, shown in [Figure 4.3](#), on my personal laptop with 10 iterations, I observe that the **fastest execution takes around 80 microseconds**.

Figure 4.3. Performance test with repetitions for a specified number of iterations.

```
fun benchLogRectangle(iters: Int = 10) {  
    val rect = Rectangle(7,9)  
    repeat(iters) {  
        measureNanoTime {  
            System.out.log(rect)  
        }.also {  
            println("log(Rectangle) took ${it/1000} micros")  
        }  
    }  
}
```

4. Avoid IO

Again, our performance test of the `log()` function is still biased by other effects. In this case, we are sending the output of `log()` to `System.out`, which incurs in additional overhead due to the time it takes to write data to the console. The time spent on I/O operations can easily overshadow the actual execution time of the `log()` function itself, leading to inaccurate performance measurements. To avoid this bias, it's important to either redirect the output or suppress it during performance testing, ensuring that only the function's execution time is measured.

In our example, we can replace `System.out` with an instance of `StringBuilder`, which uses an in-memory data structure to store the appended text. With the new implementation shown in [Figure 4.4](#), I observe the **fastest execution time on my personal laptop to be around 20 microseconds**. Note how significantly we have reduced the elapsed time and how biased the results were in previous approaches.

Figure 4.4. Replace System.out by a StringBuilder instance.

```
fun benchLogRectangle(iters: Int = 10) {  
    val rect = Rectangle(7,9)  
    repeat(iters) {  
        measureNanoTime {  
            StringBuilder().log(rect)  
        }.also {  
            println("log(Rectangle) took ${it/1000} micros")  
        }  
    }  
}
```

Although this last enhancement solves the I/O issue, it is not the best solution to the problem. Specifically, we are initializing a new `StringBuilder` on each iteration and still managing the resulting text within an internal data structure. Nevertheless, it remains fairly accurate and produces valid results. Moreover, the results are even more reliable if we maintain consistency and use the same output approach when comparing against the baseline strategy.

A potential issue with this test is the JVM's optimizations, such as *dead-code elimination*, which can skip the execution of operations that have no effect on the overall state of the application. Since we are not using the result of the `StringBuilder` anywhere, the JVM may detect this as redundant and remove part of the code being tested. This could significantly inflate performance results, leading to a biased analysis.

Later, when we introduce a proper microbenchmarking framework in our performance tests, we will use the recommended approach through a **Black Hole**. As stated in the JMH documentation:

*A **Black Hole** “consumes” values without revealing any information to the JIT about whether the value is actually used afterward. This prevents dead-code elimination of computations that produce those values.*

For now, we can simply return the resulting `String` from the `log` function invocation and clear the `StringBuilder` before logging again to prevent it from growing indefinitely in memory. Of course, calling `clear()` introduces a slight overhead, but in this case, it is highly efficient since it only resets the internal character count to zero. This version of the `log` function aligns with the definition presented in [Figure 4.5](#).

Figure 4.5. Reuse the same `StringBuilder` instance.

```
fun benchLogRectangle(iters: Int = 10): String {
    val rect = Rectangle(7,9)
    val out = StringBuilder()
    repeat(iters) {
        measureNanoTime {
            out.clear()
            out.log(rect)
        }.also {
            println("log(Rectangle) took ${it/1000} micros")
        }
    }
    return out.toString()
}
```

The performance results observed when running the implementation of Figure 4.5 do not differ significantly from our initial observations in the experiments of Figure 4.4. They are consistent with the sample output of running the benchmark with 5 iterations presented in Figure 4.6.

Figure 4.6. Showing execution time per iteration in microseconds for `benchLogRectangle`.

```
log(Rectangle) took 76 micros
log(Rectangle) took 34 micros
log(Rectangle) took 22 micros
log(Rectangle) took 20 micros
log(Rectangle) took 20 micros
```

5. Minimize the system call overhead

Using `System.nanoTime()` to measure elapsed time may involve a system call, which has its own execution cost. Moreover, as stated in the JDK documentation:

This method provides nanosecond precision, but not necessarily nanosecond resolution (that is, how frequently the value changes). No guarantees are made except that the resolution is at least as good as that of `currentTimeMillis()`.

Therefore, it is not surprising that although measurements stabilize in the later iterations, they still exhibit variations of around 5 microseconds, fluctuating up and down. In fact, the last iteration may not always be the fastest one.

To mitigate this effect, we should execute the tested operation a large number of times and then divide the total elapsed time by the number of executions to determine the average time per operation. The implementation in [Figure 4.7](#) follows this approach, executing the log operation for 1 million iterations.

Figure 4.7. Execute the test for 1 million iterations.

```
fun benchLogRectangle(iters: Int = 10) : String {
    val rect = Rectangle(7,9)
    val opCount = 1_000_000
    val out = StringBuilder()
    repeat(iters) {
        measureNanoTime {
            repeat(opCount) {
                out.clear()
                out.log(rect)
            }
        }.also {
            val perOp = it/opCount
            println("log(Rectangle) took $perOp nanos")
        }
    }
    return out.toString()
}
```

Next, we present a sample output of running the benchmark with 5 iterations. Note that the execution time is now reported in nanoseconds, instead of microseconds as in the previous implementation. The best (lowest) observed execution time is 220 nanos.

Figure 4.8. Showing execution time per iteration in nanoseconds for benchLogRectangle.

```
log(Rectangle) took 502 nanos
log(Rectangle) took 224 nanos
log(Rectangle) took 234 nanos
log(Rectangle) took 239 nanos
log(Rectangle) took 220 nanos
```

Summary

Do not draw conclusions based on a single execution of your tests. Performance measurements must be approached systematically to avoid misleading results. Throughout this section, we analyzed several common pitfalls in microbenchmarking and how they can distort performance assessments.

First, we emphasized that **microbenchmarking is not unit testing**—performance tests should be executed in a controlled environment, free from external factors like runtime introspection and framework overhead.

Second, we highlighted the importance of **avoiding unrelated operations** within the measurement scope. Unnecessary computations, such as object instantiations, can interfere with the results and should be moved outside the benchmarking loop whenever possible.

Third, we examined how the **first execution is impacted by JIT compilation**. The Just-In-Time (JIT) compiler introduces an initial performance penalty, and optimizations may be applied dynamically across executions. Thus, benchmarking must account for warm-up phases to ensure accurate readings.

Additionally, we addressed the overhead introduced by **I/O operations**. Writing to the console or a file can significantly slow down execution, skewing results. Using an in-memory alternative helps mitigate this effect.

Finally, we discussed **minimizing system call overhead**. Measuring time with functions like `System.nanoTime()` involves system calls that introduce variability. Running multiple iterations and averaging the results provides a more reliable measurement.

By keeping these considerations in mind, we can refine our benchmarking methodology and obtain more accurate insights into system performance.

Even with a systematic approach, accurately measuring performance is challenging. To achieve reliable and reproducible results, it is highly recommended to use state-of-the-art benchmarking frameworks such as JMH, which are specifically designed to handle JVM optimizations and benchmarking pitfalls.

4.2: JMH

JMH (Java Microbenchmark Harness) is a state-of-the-art benchmarking tool developed by Aleksey Shipilev, one of the core contributors to the Java Virtual Machine at Oracle. JMH was developed within OpenJDK and is the official microbenchmarking harness for the Java platform.

JMH incorporates all the best practices of performance measurement described in the previous section, among others, namely:

- warm-up iterations
- prevention of dead-code elimination
- accurate timing and statistical analysis
- isolation of benchmark state and execution
- control of measurement iterations and forks

Like other test automation frameworks, JMH provides the `@Benchmark` annotation to allow programmers to identify which methods should be treated as performance tests. These methods are bytecode-processed by JMH to introduce interception points where time measurement can be inserted in

the most effective way, minimizing additional overhead. Note that, unlike common unit testing frameworks that rely on reflection to control test execution, JMH avoids using reflection in order to prevent introducing additional overhead that could interfere with the measured code.

Maven archetypes are the primary mechanism used to bootstrap JMH projects with a properly configured build setup. As an alternative, we will use the [JMH Gradle Plugin](#)¹ to integrate the JMH microbenchmarking framework with Gradle, configured in `build.gradle` according to the example in [Figure 4.9](#).

Figure 4.9. Gradle configuration applying the JMH plugin for microbenchmarking.

```
plugins {
    // Plugin to add support for Kotlin.
    alias(libs.plugins.kotlin.jvm)

    // Apply the JMH plugin for microbenchmarking.
    id("me.champeau.jmh") version "0.7.3"
}
```

Regardless of the build tool in use, the benchmarking development approach is the same and consists of building a self-running JAR that contains all the necessary class files from both JMH and the user's program to execute the benchmark. This allows the resulting JAR to be executed directly from the terminal on the JVM without the need for additional tools, such as an IDE, which may introduce intrinsic overhead.

In [Figure 4.10](#), we present the JMH equivalent of the microbenchmark implemented in Listing [Figure 4.7](#). We can see that all the control execution logic and the direct system call to `nanoTime` disappear, as they are delegated to the JMH framework. As a result, the `benchLogRectangle` method focuses solely on the target operation being measured, without any benchmarking infrastructure code.

Figure 4.10. JMH benchmark class measuring the average execution time of a logging operation on a `Rectangle` instance.

```
@BenchmarkMode(mode = Mode.AverageTime)
@State(scope = Scope.Benchmark)
open class LogReflectionBench {
    private val rect = Rectangle(7, 9)
    private val out = StringBuilder()

    @Benchmark
    fun benchLogRectangle(): String {
        out.clear()
        out.log(rect)
        return out.toString()
    }
}
```

¹<https://github.com/melix/jmh-gradle-plugin>

```
    }
}
```

Beyond the `@Benchmark` annotation, which identifies the method as a JMH benchmark target, the `LogReflectionBench` class is configured with `@BenchmarkMode(Mode.AverageTime)` to instruct JMH to report results as the average execution time per operation. It is also annotated with `@State(Scope.Benchmark)`, meaning that a single instance of the benchmark state is shared across all iterations. In this case, the state consists of a pre-initialized `Rectangle` instance and a reusable `StringBuilder`.

The benchmark operates on a fixed `Rectangle(7, 9)` instance to avoid including object construction overhead in the measurements, ensuring that only the cost of the logging operation is evaluated. Each invocation of `benchLogRectangle` first clears the `StringBuilder`, then applies the `log` function to the rectangle, which formats and appends its contents to the buffer. Finally, the resulting string is returned, allowing JMH to observe and stabilize the full execution path of the operation under test.

According to the JMH Gradle plugin convention, the class shown in [Figure 4.10](#) is located in the dedicated JMH source set, typically under the path `src/jmh/kotlin`, where benchmark classes are kept separate from the main application code to ensure proper isolation and correct execution during benchmarking.

We may invoke the Gradle task `jmhJar` using `gradle jmhJar` to produce a self-contained executable JAR that includes the benchmark code as well as the necessary JMH infrastructure. This JAR is typically generated in the `build/libs/` directory and can be executed directly using `java -jar <path-to-jar>`.

By default, JMH performs 5 warmup iterations followed by 5 measurement iterations of 10 seconds each. The warmup iterations are discarded, and the reported results correspond to the average of the measured execution times across the 5 measurement iterations. Additionally, JMH repeats this entire process 5 times, in separate processes called *forks*, to reduce the impact of JVM warmup effects, JIT compilation variability, and other runtime noise, thereby improving the reliability and stability of the benchmark results.

Given the simplicity of our logging utility function, we may reduce these parameters to 4 warmup iterations and 4 measurement iterations of 2 seconds each, with a single fork. This corresponds to the following JMH command-line parameters: `-wi 4 -i 4 -w 2 -r 2 -f 1`. We also include `-tuns` to report the results in nanoseconds.

In this case, the results reported by this JMH benchmark on the same local machine under the same conditions correspond to the output presented in [Figure 4.11](#). These results are close to those obtained with our ad-hoc measurement in [Figure 4.8](#), which suggests that the main sources of bias in the previous approach have been reduced, leading to reliable measurements. Nevertheless, JMH is the state-of-the-art tool for this purpose and should be the preferred choice in the JVM ecosystem. Also note in the results of [Figure 4.11](#) the inclusion of the error value, which reflects the variability across iterations and provides additional insight into the stability of the measurements.

Figure 4.11. JMH results for `benchLogRectangle` showing average execution time.

Benchmark	Mode	Cnt	Score	Error	Units
<code>LogReflectionBench.benchLogRectangle</code>	avgt	4	239.036 ±	33.549	ns/op

An important aspect of the benchmark logic is that it returns a meaningful value, ensuring that the results of the measured operation are actually used. Otherwise, the code could be eliminated as **dead code** by JVM at runtime as an optimization. Taking this into account, we return the resulting `String` collected in the buffer in our benchmark shown in [Figure 4.10](#).

As an alternative, we can take advantage of the JMH `Blackhole` feature, which prevents dead-code elimination by consuming computed values in a way that the JVM cannot optimize away. In this case, the previous `benchLogRectangle` benchmark can be rewritten as shown in [Listing Figure 4.12](#). Nevertheless, in this case there are no differences in the results reported by JMH between either of the approaches.

Figure 4.12. JMH benchmark using `Blackhole` to consume the computed result and prevent dead-code elimination during execution.

```
@Benchmark
fun benchLogRectangle(bh: Blackhole) {
    out.clear()
    out.log(rect)
    bh.consume(out.toString())
}
```

The `consumeFull(Object obj)` of [Figure 4.13](#) method is part of JMH's internal `Blackhole` implementation and is used to ensure that values passed to the benchmark are treated as actually used, preventing the JIT compiler from eliminating them as dead code. It does this by mixing a pseudo-random value (`t1r`) updated with a linear congruential generator and applying a bitmask (`t1rMask`) to it. In most cases, the method does nothing beyond updating this internal state, keeping the overhead minimal during measurement. However, in very rare cases the condition `(t1r & t1rMask) == 0` becomes true, triggering a fallback path where the object is stored inside a `WeakReference`. This introduces an observable side effect, ensuring the value cannot be safely optimized away by the compiler. Overall, this design provides a lightweight way to “consume” values while preserving benchmark correctness without significantly affecting performance results.

Figure 4.13. Excerpt from JMH internal implementation showing the `consumeFull` method used by Blackhole.

```
private void consumeFull(Object obj) {  
    int tlrMask = this.tlrMask; // volatile read  
    int tlr = (this.tlr = (this.tlr * 1664525 + 1013904223));  
    if ((tlr & tlrMask) == 0) {  
        // SHOULD ALMOST NEVER HAPPEN IN MEASUREMENT  
        this.obj1 = new WeakReference<>(obj);  
        this.tlrMask = (tlrMask << 1) + 1;  
    }  
}
```

Summary

This chapter distinguishes between **microbenchmarking** and **benchmarking**, clarifying that they operate at different levels of analysis. Microbenchmarking targets small, isolated pieces of code to understand their performance characteristics in detail, while benchmarking evaluates the performance of an entire system. Because microbenchmarking deals with fine-grained measurements, its results are only meaningful when interpreted **relatively**, not absolutely.

A central concept in this context is the **baseline**, which represents the most efficient known implementation of a task. For example, a manual, ad-hoc mapping implementation—despite being repetitive—is often the fastest because it avoids abstraction overhead. This optimal version serves as a benchmark against which more generic solutions, such as automated mappers, can be evaluated. Without such a baseline, performance measurements lack context and cannot effectively guide optimization efforts.

The chapter stresses that **performance measurement is prone to pitfalls** and must be approached carefully. Developers should avoid guesswork and instead isolate the exact component they want to measure. It also reinforces that optimization should only be attempted after achieving correctness and reliability, echoing the principle that premature optimization can lead to wasted effort and increased complexity. Unit tests are valuable for ensuring correctness but are not suitable environments for measuring performance.

Proper microbenchmarking should be conducted in a controlled and isolated environment, ideally from the command line and without interference from other processes, to ensure more accurate and consistent measurements.

The chapter also highlights the impact of JIT compilation and runtime optimizations. Additionally, I/O operations can heavily distort performance results. Writing to the console or external systems is significantly slower than in-memory operations and can dominate the measured time. Replacing I/O with in-memory structures like `StringBuilder` improves accuracy, but introduces other concerns such as object allocation and the risk of **dead-code elimination**, where the JVM removes

computations that appear unused. Ensuring that results are actually consumed helps prevent such optimizations from invalidating the benchmark.

JMH (Java Microbenchmark Harness) is the recommended tool for reliable microbenchmarking on the JVM. JMH is designed to address all the challenges discussed earlier by automating best practices such as warm-up iterations, multiple measurement phases, and statistical analysis. It isolates benchmark execution from external interference and avoids reflection-based overhead, ensuring that the measured code remains as close as possible to real execution conditions. Additionally, JMH prevents **dead-code elimination** through mechanisms like returning computed values or using the **Blackhole** utility, which safely consumes results without introducing significant overhead.

Overall, the chapter emphasizes that accurate microbenchmarking is difficult to achieve manually due to numerous subtle sources of bias. While ad-hoc approaches can provide rough estimates, they are prone to errors and misleading conclusions. By using a specialized framework like JMH, developers can obtain reliable, reproducible, and statistically meaningful performance measurements, enabling informed optimization decisions.

4.3: Exercises

Exercise 1

The benchmarking test `benchZaka()` evaluates the performance of `String zaka(List<String> lines)`, a static method of class `Foo`. Indicate, with justification, what problem(s) exist in the following implementation:

```
@State(Scope.Benchmark)
open class ZakaBenchmark {

    private val theMetamorphosis: Path = Paths.get("TheMetamorphosis.txt")

    @Benchmark
    fun benchZaka(): String {
        return Foo.zaka(Files.readAllLines(theMetamorphosis))
    }
}
```

Exercise 2

The benchmarking test `benchWacky()` was developed to evaluate the performance of the instance method `int wacky(String in)` from class `Foo`. Indicate, with justification, what problems exist in the following implementation:

```
import org.openjdk.jmh.annotations.Benchmark

open class WackyBenchmark {

    @Benchmark
    fun benchWacky() {
        val foo = Foo()
        val res = foo.wacky("babylon")
        println("wacky => $res")
    }
}
```

Exercise 3

The goal is to measure the performance of the `exec` function of the object `Crash` { `fun exec() : String { ... }` }.

Indicate which of the following options correctly defines a benchmarking class in JMH and justify your answer:

```
open class Bench {
    @Benchmark
    fun benchCrash() { println(Crash.exec()) }
}
```

```
open class Bench {
    @Benchmark
    fun benchCrash(): String { return Crash.exec() }
}
```

```
open class Bench {
    @Benchmark
    fun benchCrash() { Crash.exec() }
}
```

```
open class Bench {  
    lateinit var res: String  
  
    @Benchmark  
    fun benchCrash() { res = Crash.exec() }  
}
```

Chapter 5: Bytecode and the Just-in-time Compiler

Bytecode is an *intermediate representation* (IR) used internally by virtual machines, such as the JVM, to represent compiled source code. It is not as low-level as assembly languages like x86 or PowerPC, nor is it as human-readable as high-level programming languages such as Kotlin or Java. That's why it is considered *intermediate*—it sits between low-level machine code and high-level source code in terms of abstraction and readability.

Unlike assembly, bytecode does not manipulate CPU registers or have direct access to memory addresses. Instead, it operates at a higher level of abstraction and **includes built-in support for object-oriented** concepts such as object instantiation and virtual method dispatch. However, it does not support first-class functions natively—such functions must always be encapsulated within classes as methods.

Java was created at a time when object-oriented design was widely regarded as the modern and scalable approach to building software. However, this object-oriented foundation does not prevent the existence of pure functional programming languages, such as Clojure, which also compile to JVM bytecode. All functional abstractions—such as higher-order functions, closures, etc, can be translated by the compiler into object-oriented constructs to simulate functional behavior within the constraints of the JVM. This allowed languages like Clojure, Scala, Kotlin, and others to compile to bytecode without requiring changes to the JVM's core instruction set.

This chapter explores the structure and behavior of JVM bytecode, focusing on how core object-oriented constructs are translated into bytecode instructions.

5.1: Evaluation Stack

Understanding the operation of the stack machine that the JVM uses is crucial to comprehending bytecode. One key distinction from hardware CPUs like x64 or ARM chips is that:

The JVM lacks processor registers.

Instead, it employs a stack for all calculations and operations, known as the **evaluation stack** or **operand stack**. This stack is local to each method, and a new one is created each time a method is called. Additionally, the **JVM maintains a call stack for each Java thread**, which forms the basis of stack traces in Java. It's essential to distinguish between the per-thread call stack and the per-method evaluation stack.

JVM bytecode comprises a sequence of operation codes (**opcodes**), often accompanied by **arguments** following each instruction. Opcodes are designed to encounter the stack in a specific state and modify it accordingly, thereby **removing arguments** and **replacing them with results**.

The fundamental operation of the evaluation stack involves **pushing and popping values** to and from the stack. The family of load and store opcodes is responsible for **loading values** onto the stack or **retrieving them**. The `load` opcode pushes a value onto the evaluation stack, while the `store` opcode pops a value off the stack. The `load` operation **reads a variable** and **pushes** its value onto the stack, whereas the `store` operation **pops** a value from the stack and **writes it to a destination variable**.

Method **variables** are managed using an **array of local variables**, which the JVM initializes alongside the evaluation stack. These **local variables are accessed by their index**. The `load` and `store` opcodes require an argument identifying a **local variable** by an **index starting at 0**.

In the example shown in [Figure 5.1](#), the evaluation stack is initially empty, and there is an array containing 3 local variables (initialized in a way that is not relevant to this explanation). At each step, we present the updated state of the stack after the execution of the opcode, which is highlighted in red. We first push the values of the variables at indexes 0 and 2 onto the stack. Then, we pop the values from the stack, storing the first one into the variable at index 1, and the second one into the variable at index 2. In this example, the opcodes start with `i` because we are working with `int` values.

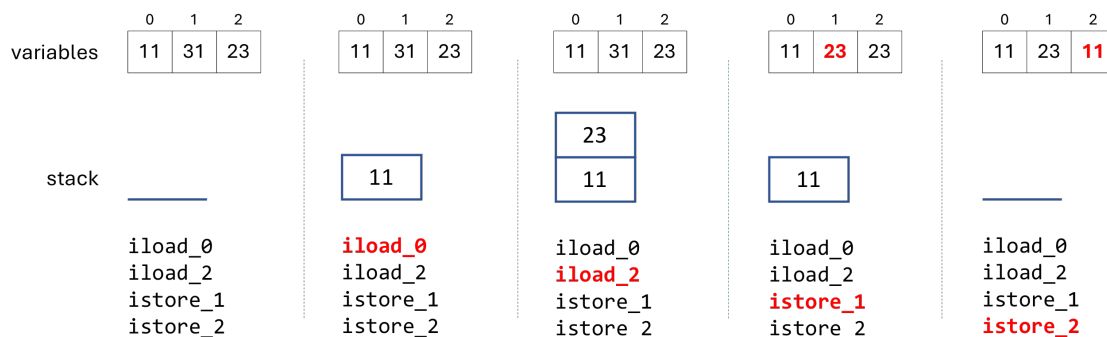


Figure 5.1. Simple example of pushing and popping values on the evaluation stack.

Arithmetic opcodes rely on the evaluation stack to manage both the operands and the result of the operation. To better understand how the stack is used in these cases, consider the example of a magnitude function shown in [Figure 5.2](#).

Figure 5.2. A `magnitude()` function in Kotlin.

```
import kotlin.math.sqrt

fun magnitude(x: Float, y: Float) : Float {
    val res = sqrt(x*x + y*y)
    return res
}
```

By utilizing `javap` with the `-c` switch, you can disassemble classes and inspect the bytecode of the `magnitude` function, which will appear according to the Figure 5.3. We also include the `-v` switch (*verbose*) to provide additional information, such as the maximum stack size and the count of **local variables** and **arguments**.

Figure 5.3. Bytecode listing of the `magnitude()` function.

```
public static final float magnitude(float, float);
Code:
    stack=3, locals=3, args_size=2
     0: fload_0
     1: fload_0
     2: fmul
     3: fload_1
     4: fload_1
     5: fmul
     6: fadd
     7: f2d
     8: invokestatic #12 // Method java/lang/Math.sqrt:(D)D
    11: d2f
    12: fstore_2
    13: fload_2
    14: freturn
```

Opcodes in the listing of Figure 5.3 are preceded by numbers indicating their offset within the bytecode stream, relative to the start of the method. In this example, most opcodes appear at consecutive offsets, each increasing by 1. However, note the jump from offset 8 to offset 11. This three-byte jump occurs because the opcode at offset 8 (`invokestatic`) is followed by a 16-bit operand—an index into the *constant pool*—making the total instruction size three bytes.

The opcodes in Figure 5.3 use a **shortcut form** to save bytes. This is visible in the offset of each bytecode, where most opcodes appear at consecutive positions. Opcodes such as `fload_0` and `fload_1` encode the local variable index directly in their own instruction, avoiding the need for an additional operand. The JVM defines these specialized opcodes for the first few local variable

indices (typically 0 to 3). Beyond that range, a more general opcode such as `fload` is used, which takes an explicit operand specifying the local variable index.

At the beginning of the code block in [Figure 5.3](#), you can see the line `stack=3, locals=3, args_size=2`, which indicates the following:

- the maximum expected stack size is 3 (`stack=3`);
- the number of local variables is 3 (`locals=3`);
- the method receives 2 arguments (`args_size=2`).

For each method execution, the JVM initializes a new evaluation stack alongside a new **array for local variables**, with the method **arguments occupying the first positions** in that array. Thus, in the example of [Figure 5.3](#), indexes 0 and 1 refer to the arguments (i.e., `x` and `y`), while index 3 refers to an auxiliary local variable (i.e., `res`).

The JVM also recognizes primitive types. This is evident in certain opcode families. Various basic opcodes (such as `load`, `mul`, `add`, `return`, etc) must have multiple variations that differ based on the primitive type they operate on.

In [Figure 5.3](#) the opcodes beginning with `f` (e.g. `fload_0`, `fmul`, `fadd`) indicate operations involving float values. However, if our function operates on integer values, the opcodes will begin with `i` (e.g. `iload_0`, `imul`, `iadd`), for long values, they will start with `l` (e.g. `lload_0`, `lmul`, `ladd`), and for double values, they will start with `d`. In contrast, all reference types use opcodes beginning with `a`, which stands for *address* and indicates a reference value.

Arithmetic opcodes retrieve their operands from the evaluation stack. Operations like `add` or `mul` consume two values from the stack and, in turn, push the result of the computation back onto it. The example in [Figure 5.4](#) shows the state of the evaluation stack after each opcode is executed, assuming the `magnitude` function is called with arguments 3 and 4. Notice that after executing `fmul` or `fadd`, the two topmost values are removed from the stack, and the resulting value is pushed back onto it.

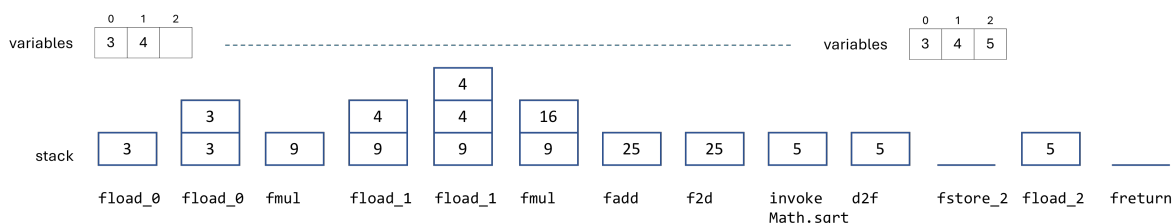


Figure 5.4. Example of the stack considering the `magnitude` function is called with arguments 3 and 4.

The opcodes `f2d` and `d2f` perform **primitive type conversions**. As previously mentioned, the JVM handles primitive types explicitly, and **opcodes are type-safe**. This type safety also applies to function calls—**arguments must match the expected parameter types**.

In the `magnitude` function example, the code invokes the `sqrt` function from the `Math` class, which expects a parameter of type `double`. Since the `magnitude` function operates on `float` values, it must first convert the `float` to a `double` using the `f2d` opcode (float to double) before making the function call. Likewise, the result returned by `sqrt` is a `double`, which must be converted back to `float` using the `d2f` opcode (double to float) before being stored in the auxiliary float variable at index 2.

5.2: Constant Pool and Descriptors

The **constant pool** is a collection of constants and symbolic references used by a class, serving as a symbol table that the JVM consults for field names, method names, string literals, and other metadata.

In the `magnitude` function example, the `invokestatic` opcode is used to call the static `sqrt` method of the `Math` class. In the `javap` output shown in [Figure 5.3](#), the `sqrt` method is referenced by the **constant pool** entry #12. The #12 represents a 16-bit index into the constant pool.

To inspect the contents of the constant pool, you can use the `javap -v` command. This provides a detailed view of the class file, including much more information than just the constant pool. [Figure 5.5](#) displays a portion of the constant pool output produced by `javap -v` for the class file containing the previously defined `magnitude` function.

Figure 5.5. Portion of the of the constant pool output for the class file with `magnitude` function.

```
Constant pool:
  #1 = Utf8               MagnitudeKt
  #2 = Class                #1                // MagnitudeKt
  #3 = Utf8               java/lang/Object
  #4 = Class                #3                // java/lang/Object
  #5 = Utf8               magnitude
  #6 = Utf8               (FF)F
  #7 = Utf8               java/lang/Math
  #8 = Class                #7                // java/lang/Math
  #9 = Utf8               sqrt
  #10 = Utf8              (D)D
  #11 = NameAndType        #9:#10            // sqrt:(D)D
  #12 = Methodref          #8.#11            // java/lang/Math.sqrt:(D)D
  ...
```

Constant pool entries are typed and can reference each other. For example, a `Class` entry usually points to a `Utf8` entry, which represents a string. In this case, the `Utf8` entry contains the name of the class. In the sample from [Figure 5.5](#), we can observe that #2 corresponds to a `Class` entry, whose name is defined by the `Utf8` entry at index #1, which contains the string `MagnitudeKt`. In this case, the top-level `magnitude` function was defined in a source file named `Magnitude.kt`, which causes

the compiler to generate a class file named `MagnitudeKt.class`. This class serves as the container for the top-level function.

Each class referenced by the `MagnitudeKt` class—such as `java.lang.Object` and `java.lang.Math`—has a corresponding `Class` entry in the constant pool, specifically entries #4 and #8. These `Class` entries point to `Utf8` entries that contain the class names in the form `java/lang/Object` and `java/lang/Math`, respectively.

Class and interface names in class file structures are always represented in a **fully qualified form** known as the ***internal form***. In this internal JVM format, periods (.) are replaced by forward slashes (/) to separate package components.

A `Methodref` entry points to a combination of a `Class` entry and a `NameAndType` entry. The `Class` entry refers to the class that owns the method, while the `NameAndType` entry specifies the **method name and its function type**. Both the name and the type are represented by `Utf8` entries. In the case of the referenced square root function, the method name is `sqrt`, and its function type is represented by the descriptor string `(D)D`.

In the constant pool, a string defining a function type is represented by a ***function type descriptor***, also known simply as a ***method descriptor***. A *method descriptor* lists the parameter *type descriptors* enclosed in parentheses, followed by the return *type descriptor*—for example, `(D)D`. A ***type descriptor*** is a compact representation of a type; for instance, `int` is represented by `I`. A complete list of type descriptors is provided in [Table 5.1](#).

Table 5.1. Type descriptors for core JVM types.

Java Type	Descriptor
byte	B
char	C
double	D
float	F
int	I
long	J
short	S
boolean	Z
void	V
Class or interface e.g. <code>String</code>	<code>L<classname>;</code> e.g. <code>Ljava/lang/String;</code>
Array of T e.g. <code>String[]</code> e.g. <code>int[][]</code>	<code>[<descriptor of T></code> e.g. <code>[Ljava/lang/String;</code> e.g. <code>[[I</code>

For reference types—such as those defined by classes or interfaces—the *type descriptor* can be longer than the type name that appears in source code. In this case, the descriptor begins with `L`, followed by the JVM internal name of the class, and ends with a semicolon (;). Because *method descriptors* do not use separators like commas between parameters, a distinct format is required to clearly delimit

reference type descriptors. The `L...; structure` serves this purpose.

For example, the method `int add(int a, int b)` has the descriptor `(II)I`, where the two parameter types `int` are represented by `I` and appear consecutively without any separator. This lack of commas makes the use of clear boundaries essential for distinguishing between multiple reference types.

5.3: Bytecode for Comparison and Control Flow

The JVM provides a set of bytecode instructions dedicated to value comparison and control flow manipulation, which form the basis for implementing high-level constructs such as conditionals and loops. Unlike method invocation or field access instructions, these opcodes operate primarily on the operand stack and directly influence the execution path of a method.

For primitive comparisons, the JVM uses specialized comparison instructions depending on the type. For example, long values are compared using `lcmp`, which consumes the two topmost values from the stack and pushes an integer result: `-1` if the first value is less than the second, `0` if they are equal, and `1` if it is greater. Similar instructions exist for floating-point types, such as `fcmp1` and `fcmpg`.

Once a comparison result is produced, conditional branching instructions determine the subsequent execution flow. Instructions such as `ifgt`, `iflt`, `ifeq`, and their variants inspect the top of the operand stack and redirect execution based on the evaluated condition. If the condition is satisfied, control jumps to a specified bytecode offset; otherwise, execution continues sequentially.

Unconditional control transfer is handled by the `goto` instruction, which performs an immediate jump to a target instruction without evaluating any condition. This instruction is commonly used to implement loops, skip blocks of code, or represent the else branch of conditional structures.

To better illustrate the use of these opcodes, let us consider the recursive implementation of the Euclidean algorithm for computing the greatest common divisor (GCD), shown in Listing of [Figure 5.6](#). This implementation expresses the algorithm in terms of a base case and a recursive step. It checks whether `b` is zero; if so, it returns `a` as the greatest common divisor. Otherwise, it recursively calls `gcd` with the arguments `b` and `a % b`.

Figure 5.6. Implementation of the Euclidean algorithm for computing the greatest common divisor.

```
fun gcd(a: Int, b: Int): Int {  
    return if (b == 0) a else gcd(b, a % b)  
}
```

The code of [Figure 5.7](#) presents the bytecode resulting from the compilation of the `gcd` method. This sequence shows how comparison (`ifne`), arithmetic (`irem`), control flow (`goto`), and recursion (`invokestatic`) are combined to implement the algorithm at the bytecode level.

Figure 5.7. Bytecode of the gcd method.

```

0: iload_1
1: ifne      8
4: iload_0
5: goto      15
8: iload_1
9: iload_0
10: iload_1
11: irem
12: invokestatic #8    // Method gcd:(II)I
15: ireturn

```

Execution of bytecode of [Figure 5.7](#) starts by loading the second parameter `b` (`iload_1`) onto the operand stack. The instruction `ifne` checks whether this value is not equal to zero; if so, control jumps to offset 8, corresponding to the recursive case. Otherwise, execution continues at offset 4, where `iload_0` loads `a` onto the stack, and `goto` transfers control to the return instruction, implementing the base case `return a`.

In the recursive branch, `iload_1` pushes `b`, followed by `iload_0` and `iload_1` to prepare the operands for the modulo operation. The `irem` instruction computes `a % b`, leaving the result on the stack. At this point, the stack contains the arguments `b` and `a % b`, which are then consumed by `invokestatic` to recursively invoke the `gcd` method. Finally, `ireturn` returns the computed result.

5.4: Managing Instances and Members

Bytecode includes object-oriented constructs in its instruction set, such as method invocation, field access, and object creation, which directly support the execution model of languages like Java and Kotlin on the JVM.

5.4.1: Creating Objects

Creating an instance in Java or Kotlin results in bytecode that includes the `new` opcode followed by an `invokespecial` instruction, regardless of other aspects that will be discussed later. Thus, for example, creating an instance of class `A` in Java using `new A()` or in Kotlin using `A()` produces the bytecode shown in the listing of [Figure 5.8](#).

Figure 5.8. Bytecode sequence for object instantiation.

```

0: new      #20    // class A
3: dup
4: invokespecial #23 // Method A."<init>":()V

```

The `new` opcode is responsible for allocating memory for a **new object on the heap** and pushing a reference to that object onto the operand stack. **Note** that this opcode **does not invoke a constructor**; it simply allocates memory on the heap. The allocated memory is zero-initialized, meaning all fields are set to their default values (e.g., `0`, `false`, or `null`). This behavior is analogous to low-level allocation routines such as `calloc` (or `malloc` followed by explicit initialization), with object initialization performed later by separate bytecode instructions.

The `invokespecial` opcode is then responsible for invoking the constructor. Note that any constructor receives at least the `this` reference as its implicit parameter. For this reason, `invokespecial` must find a reference to the object being initialized on the operand stack. Consequently, we often see a `dup` instruction before `invokespecial`, which duplicates the object reference on the stack so that one copy can be consumed by the constructor call while the other remains available as the result of the expression.

Considering, for example, the following instantiation: `val p = Person(name = "John Doe", age = 25)`, which produces the bytecode shown in the listing of [Figure 5.9](#). The corresponding bytecode begins by creating a new instance of the `Person` class using the `new` instruction, followed by `dup` to preserve a copy of the reference on the operand stack. The constructor arguments are then prepared, where the string constant `"John Doe"` is loaded onto the stack using `ldc`, and the integer value `25` is pushed using `bipush` (for a signed byte value). These values, together with the object reference, are then consumed by the `invokespecial` instruction, which calls the `Person` constructor with the signature `(String, int)`. Finally, the resulting initialized object reference is stored in local variable slot `0` using `astore_0`, completing the assignment of the newly created instance to the variable `p`.

Figure 5.9. Bytecode sequence for creating an instance of `Person`.

```

0: new          #8      // class pt/isel/Person
3: dup
4: ldc          #10     // String John Doe
6: bipush       25
8: invokespecial #14     // Method pt/isel/Person."<init>":(Ljava/lang/String;I)V
11: astore_0

```

5.4.2: Accessing Fields

The JVM provides different bytecode instructions to access static and instance (non-static) fields. Static fields are accessed using `getstatic` and `putstatic` to read and write values, respectively, whereas instance fields are accessed through `getfield` and `putfield`. The key difference is that **instance field access requires a reference to the target object**, which must be present on the operand stack and is consumed by the instruction. In contrast, **static field access operates at the class level and does not require any object reference**, as the field is associated with the class rather than a specific instance.

To exemplify both usages, consider the following Java example of `ElapsedTimeCalc` in [Section 5.4.2](#), which measures elapsed time since object creation using an instance field and also maintains a global

counter of created instances using a static field. The instance field `startTime` is initialized in each object and used by the `elapsedTime()` method to compute the difference between the current time and the creation time, while the static field `instances` is shared across all instances of the class and incremented in the constructor each time a new object is created.

```
import static java.lang.System.currentTimeMillis;

public class ElapsedTimeCalc {
    private final long startTime = currentTimeMillis();

    private static int instances = 0;

    public ElapsedTimeCalc() { instances++; }

    public long elapsedTime() { return currentTimeMillis() - startTime; }

    public static int getInstances() { return instances; }
}
```

Starting with the instance constructor, let us analyze its bytecode presented in Listing of [Figure 5.10](#). The execution begins with `aload_0`, which pushes the current object reference (`this`) onto the operand stack, followed by `invokespecial` to invoke the superclass constructor (`Object.<init>`). After the object is properly constructed at the superclass level, `aload_0` is used again to place the current instance on the stack, and `invokestatic` calls `System.currentTimeMillis()` to obtain the current time. The returned value is then assigned to the instance field `startTime` via `putfield`, which consumes both the object reference and the computed value.

Next, the static field `instances` is accessed using `getstatic`, retrieving its current value. The instruction `iconst_1` pushes the constant 1 onto the stack, and `iadd` then consumes the two values at the top of the stack—the current number of instances and the constant 1—adds them together, and pushes the resulting sum back onto the stack. This updated value is subsequently written to the static field using `putstatic`. Finally, the constructor completes with the `return` instruction. This sequence clearly illustrates the use of `putfield` for instance state initialization and `getstatic/putstatic` for managing shared class-level state.

Figure 5.10. Constructor bytecode showing instance field initialization and static field update.

```
public pt.isel.ElapsedTimeCalc();
```

```
Code:
```

```

0: aload_0
1: invokespecial #1    // Method java/lang/Object."<init>":()V
4: aload_0
5: invokestatic  #7     // Method java/lang/System.currentTimeMillis():J
8: putfield     #13    // Field startTime:J
11: getstatic    #19    // Field instances:I
14: iconst_1
15: iadd
16: putstatic    #19    // Field instances:I
19: return

```

On the other hand, the `elapsedTime` method presented in Figure 5.11 has a simpler bytecode structure. It begins with `invokestatic` to call `System.currentTimeMillis()`, pushing the current time onto the operand stack. Then, `aload_0` loads the current instance (`this`), and `getfield` retrieves the value of the instance field `startTime`. The instruction `lsub` then consumes the two long values on the stack—the current time and the stored `startTime`—computes their difference, and pushes the result back onto the stack. Finally, `lreturn` returns this value to the caller.

Figure 5.11. `elapsedTime` bytecode showing instance field access.

```
public long elapsedTime();
```

```
Code:
```

```

0: invokestatic  #7     // Method java/lang/System.currentTimeMillis():J
3: aload_0
4: getfield     #13    // Field startTime:J
7: lsub
8: lreturn

```

5.4.3: Invoking Methods

Method invocation in the JVM can be distinguished based on whether an implicit `this` argument is involved and whether the dispatch is static or dynamic. **Static dispatch** means that the target method is known at compile time; once resolved (and possibly inlined) by the JIT compiler, it remains fixed and corresponds to a well-defined address. In contrast, **dynamic dispatch**—often referred to in object-oriented terminology as *polymorphic* invocation—resolves the target method at runtime, at the moment of invocation. This means that the actual method executed may vary depending, for example, on the runtime type of the *receiver* instance.

A receiver object is the instance on which an instance method is invoked.

The four types of method invocation in the JVM can be organized according to this classification, as shown in the [Table 5.2](#). We exclude `invokedynamic` from this discussion, as it is primarily used to provide dynamic linkage, such as implementing interfaces from method references or lambda expressions.

Table 5.2. JVM method invocation instructions classified by dispatch type and use of the implicit `this` reference.

	Without <code>this</code> (no instance)	With <code>this</code> (instance)
Static dispatch	<code>invokestatic</code>	<code>invokespecial</code>
Dynamic dispatch	—	<code>invokevirtual</code> , <code>invokeinterface</code>

The `invokevirtual` and `invokeinterface` instructions have similar behavior, but the latter is specifically used for method calls on interfaces, whereas the former is used for all other cases of virtual method invocation on class types.

Now, we can also understand why the JVM employs a distinct `invokespecial` bytecode when invoking the base class constructor. Note that when calling a superclass constructor with the same signature as the constructor being defined, the call must be performed in a special, non-virtual manner. Otherwise, using virtual dispatch would lead to an infinite recursion, since the call would be dynamically resolved back to the same constructor implementation in the current class, rather than explicitly targeting the superclass constructor.

5.5: Method Dispatch Internals

To enable **dynamic dispatch**, the JVM maintains a per-class structure that associates method calls with their concrete implementations at runtime, commonly referred to as a *vtable* (a term borrowed from C++’s virtual function table). In the JVM, the *vtable* resides within an internal per-class metadata structure commonly referred to as the *Klass*. We intentionally use the term *Klass* to distinguish it from its reflective counterpart, `java.lang.Class`.

Unlike `java.lang.Class`, which is a programmatic API accessible to developers, the *Klass* is an **internal JVM structure** and is intentionally hidden from user code. This encapsulation ensures that critical runtime metadata—such as method dispatch tables, field layouts, and type information—cannot be modified directly, which would otherwise risk corrupting the execution model and compromising type safety and JVM integrity.

As with `java.lang.Class`, where the JVM maintains a **single instance per loaded type**, there is also a single internal *Klass* structure for each type. This structure is **shared by all instances of the same type**, and every object carries a reference to it. To achieve this, the `class` portion of the object header, shown in the layout of [Figure 2.5](#), stores a pointer to the corresponding *Klass* structure, which is used by the JVM for type metadata and runtime dispatch.

Simply put, the *Klass* structure can be viewed as being composed of **two main parts: metadata and vtable**. The metadata section captures all descriptive information about a type, including its flags, implemented interfaces, field layout, and other runtime type descriptors required by the JVM. This information is used for type checking, reflection, and object layout computation.

The *vtable*, on the other hand, is a dispatch table that contains an entry for each method defined or overridden by the type. **Each entry points to the concrete method implementation that should be executed at runtime.**

To better illustrate this, consider the type hierarchy defined between *Shape*, *Rectangle*, and *Triangle* in [Figure 5.12](#), with the implementation presented in the listing of [Figure 5.13](#).

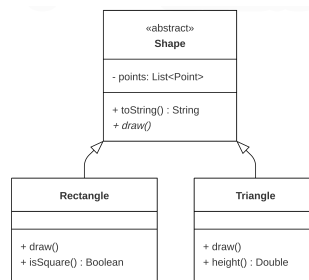


Figure 5.12. Class diagram illustrating a shape hierarchy.

Figure 5.13. Kotlin implementations of a shape hierarchy.

```

data class Point(val x: Double, val y: Double)

abstract class Shape(protected val points: List<Point>) {
    override fun toString() : String { ... }

    // Abstract □ must be implemented by subclasses (virtual dispatch)
    abstract fun draw()
}

class Triangle(
    p1: Point, p2: Point, p3: Point
) : Shape(listOf(p1, p2, p3)) {
    override fun draw() { ... }

    // Triangle-specific non-virtual method
    fun height(): Double { ... }
}

class Rectangle(

```

```

    p1: Point, p2: Point, p3: Point, p4: Point
) : Shape(listOf(p1, p2, p3, p4)) {

    override fun draw() { ... }

    // Rectangle-specific non-virtual method
    fun isSquare(): Boolean { return false }
}

```

The hierarchy illustrated in [Figure 5.12](#), with the implementation of [Figure 5.13](#) is rooted in the abstract class `Shape`, which defines the common structure and behavior shared by all shapes. It stores a list of `Point` objects and provides an implementation of `toString()`, overriding the method inherited from `java.lang.Object`. Since `toString()` is not declared as open in `Shape`, it is **non-virtual in Kotlin terms (final by default)** and therefore inherited as-is by subclasses without the possibility of further overriding. In contrast, the method `draw()` is declared as abstract, which makes it inherently **virtual** and requires every concrete subclass to provide its own implementation.

Both `Triangle` and `Rectangle` extend `Shape` and therefore inherit its state (points) and its concrete `toString()` implementation. Each class provides its own implementation of `draw()`, overriding the abstract declaration in `Shape`. These overridden `draw()` methods remain **virtual**, participating in dynamic dispatch, so the actual method executed depends on the runtime type of the receiver.

In addition to inherited and overridden behavior, each subclass introduces its own specific methods. `Triangle` defines the method `height()`, and `Rectangle` defines `isSquare()`. These methods are **new** (i.e., not present in the superclass) and, since they are not marked as open, they are **non-virtual (final)**. As such, they cannot be overridden further and are typically invoked without dynamic dispatch.

Next, in [Figure 5.14](#) we present an example of the object layout for two instances of `Triangle` (`tri1` and `tri2`) and one instance of `Rectangle` (`rect1`). For these instances, we depict the object header (the `mark` and `class` slots) and the `points` field. For simplicity, we do not include the objects referenced by the `points` field. Note that each variable `tri1`, `tri2`, and `rect1` stores a reference to the corresponding object header (`mark` word) of the referenced object. We also include in the diagram the *Klass* structure for each type, which is pointed to by the `class` header of each object. Both `Triangle` instances share the same *Klass* structure, as they belong to the same runtime type. Each *Klass* structure contains, in its metadata, a reference to its superclass, as shown in [Figure 5.14](#): `Shape` points to `Object`, and both `Triangle` and `Rectangle` reference their common base class `Shape`.

In the *vtable* section of the *Klass* structure, we denote with lowercase letters (a, b, c, etc.) the **offsets** of each entry. These offsets represent fixed positions within the *vtable* array where method references are stored. At runtime, when a virtual method is invoked (e.g., via `invokevirtual`), the JVM uses the method's offset to index into the *vtable* of the receiver's *Klass*, retrieve the corresponding implementation, and execute it.

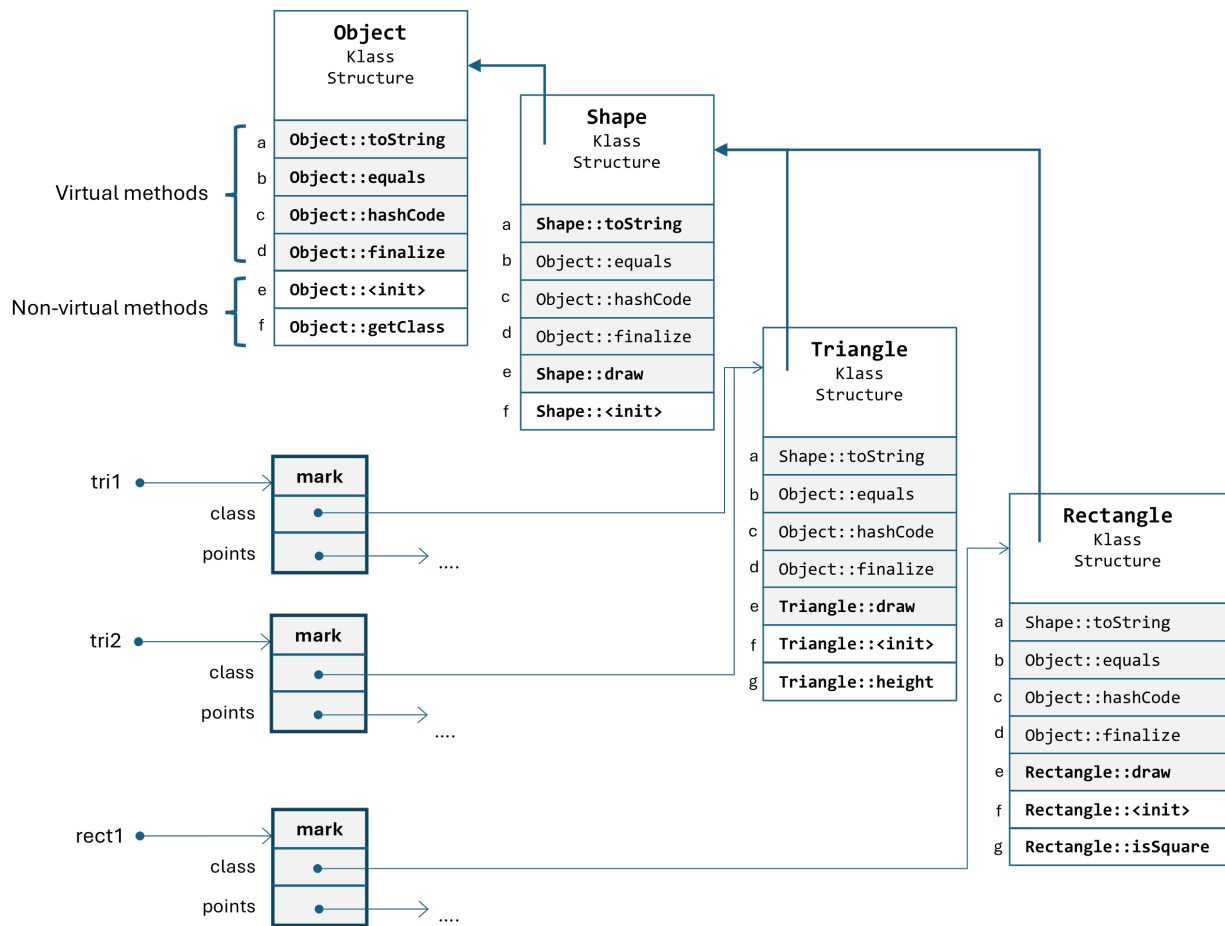


Figure 5.14. Example of Class structure and vtable layout.

To make it efficient, vtables are laid out in a specific way. Each *Klass* organizes its vtable so that it **begins with the inherited virtual methods**, followed by **newly declared virtual methods**, and then the **non-virtual methods**. Note that the constructor corresponding to `<init>` is treated as a non-virtual method. When a subclass overrides a method, the corresponding vtable entry remains at the **same offset as in the superclass**. This ensures that method lookup for overridden methods is straightforward, since their position in the vtable is preserved across the hierarchy.

For example, in the *Klass* of **Shape** in Figure 5.14, the first entries in the vtable correspond to the virtual methods defined in **Object**, followed by the new virtual method `draw` introduced by **Shape**. After that come the non-virtual methods, including `<init>`. In this vtable, the entry corresponding to `toString` is replaced with the overridden implementation provided in **Shape**, while `equals`, `hashCode`, and `finalize` continue to point to the inherited implementations from **Object**. Note that the non-virtual `getClass` method of **Object** will not appear in the vtable of derived classes, since it is always resolved via static dispatch to a well-known method entry.

This process continues with **Triangle** and **Rectangle**, each including in the virtual-methods section of the vtable the same five virtual methods in the same order as defined in the base class **Shape**.

However, each class replaces the entry for the `draw` method with its own specific implementation. In addition, both classes define their own non-virtual methods—`height()` in `Triangle` and `isSquare()` in `Rectangle`—which appear in the non-virtual section of the `vtable`.

Summary

We began by examining how the JVM represents and executes object-oriented constructs at the bytecode level, focusing on how high-level concepts such as object creation, method invocation, field access, and control flow are translated into low-level stack-based instructions. A central theme was how the JVM operand stack is used to pass arguments, hold intermediate results, and coordinate execution across instructions.

We connected these concepts to control flow and comparison bytecode, showing how instructions such as `if`, `goto`, and arithmetic comparisons like `lcmp` encode high-level constructs such as conditionals and recursion. Across all these topics, the JVM execution model was consistently shown to rely on a stack-based architecture, compact bytecode representations, and carefully structured metadata to efficiently support object-oriented programming semantics.

After that we have analyzed object instantiation, showing how the `new` opcode allocates memory on the heap and pushes a reference to the operand stack without invoking any constructor. We clarified that the allocated memory is zero-initialized and that actual initialization is performed in subsequent steps. The `dup` instruction was introduced as a necessary mechanism to preserve the object reference when both initialization and assignment are required.

This led to the study of constructors in bytecode, where we observed how `invokespecial` is used to explicitly invoke constructors, including superclass initialization via `Object.<init>`. We also examined how instance fields are initialized using `putfield`, while static fields are manipulated using `getstatic` and `putstatic`. The distinction between instance and class-level state was emphasized, particularly how instance field access requires a receiver object reference on the operand stack.

We then explored method invocation semantics in the JVM, distinguishing between static and dynamic dispatch. Static dispatch, implemented via `invokestatic` and `invokespecial`, resolves method targets at compile time or through direct linking, while dynamic dispatch, implemented via `invokevirtual` and `invokeinterface`, resolves method calls at runtime based on the receiver type. We also noted that `invokedynamic` was excluded from this discussion due to its role in supporting higher-level language features such as lambdas and method references.

To illustrate dynamic dispatch, we introduced a Kotlin hierarchy example to motivate the internal structure of the JVM's method dispatch mechanism and the `vtable` stored inside the *Klass* structure. We explained that each class has a corresponding *Klass* metadata structure containing type information and a `vtable`. The `vtable` entries are arranged so that inherited methods appear first, followed by newly declared methods, and finally non-virtual methods. Method overriding works

by preserving the same vtable offset as the parent method, ensuring constant-time lookup during dispatch.

We also clarified how object headers store a reference to the *Klass* structure via the `class` slot, enabling runtime type identification and method resolution. Multiple instances of the same type share a single *Klass* structure, reinforcing the idea that method dispatch metadata is class-level rather than instance-level.

Chapter 6: Metaprogramming

Metaprogramming is a programming technique where a computer program interacts with other programs, either by *reading* existing ones or *generating* new ones. While Reflection enables us to read existing programs at runtime, it lacks the ability to generate new ones.

Moreover, utilizing Reflection, for example, to invoke methods, may incur a performance overhead. This overhead stems from the dynamic nature of Reflection, which involves runtime lookups and method invocations, potentially slowing down the program execution.

Now, our goal is to build software that explores the same capabilities provided by reflection without incurring performance overhead.

To achieve this goal, for highly executed tasks such as the `mapFrom` method of `NaiveMapper`, we will replace the use of Reflection with a dynamically generated implementation of `mapFrom`. This implementation will utilize simple bytecode instructions for the JVM, eliminating the need for Reflection.

Note that we will **continue to utilize Reflection**, albeit solely for the purpose of dynamically generating an implementation of the `mapFrom` method.

6.1: Introduction

There are several **forms of metaprogramming**, including: *Compile-time* and *Run-time metaprogramming*

Compile-time metaprogramming involves crafting code that generates or alters code during the compilation phase. Examples of this technique include **macros** in languages such as C/C++ and Lisp. Macros define rules or patterns dictating how specific input should transform into output. They're frequently employed to **condense a series of computing instructions into a single program statement**.

Run-time metaprogramming entails developing code that dynamically generates or manipulates code at runtime. **Reflection** is an example of a run-time metaprogramming technique, allowing access to metadata and execution of existing routines, **but it cannot generate new ones**.

Lisp is renowned for its robust support for run-time metaprogramming, enabling the creation of new functions with dynamically constructed statements. Similarly, **JavaScript** and many other programming languages offer capabilities for dynamic code generation. Next is an example of a JavaScript snippet that constructs a function using source code read at runtime:

Figure 6.1. A JavaScript program (e.g. `index.js`) to build a function dynamically.

```
// Extract the statement from the command line arguments
const statement = process.argv[2]

// Build a new function using the provided statement
const dynamicFun = Function(statement)

// Now you can call the dynamically created function
dynamicFun()
```

Running the JavaScript program shown in [Figure 6.1](#) in Node.js with the command line `node index "console.log('Hello, world!');"` may produce the following output:



Figure 6.2. Running the JavaScript sample to build a function dynamically.

We can also generate more structured code by declaring a new function at runtime (e.g., `circleArea()`) to be used in other statements. For example, you could invoke `index.js` in the following way:

```
node index.js "function circleArea(r) { return Math.PI * r * r; }; console.log('Circle area with radius 7 = ' + circleArea(7));"
```

Until JDK 22, neither Java nor Kotlin offered a built-in method for dynamically generating code in the JVM ecosystem. However, numerous alternative libraries have served this purpose over the years, including:

- [ASM](#)¹
- [Javassist](#)²
- [Byte Buddy](#)³
- [Cojen Maker](#)⁴
- [JavaPoet](#)⁵

¹<https://asm.ow2.io/>

²<https://www.javassist.org/>

³<https://bytebuddy.net/#/>

⁴<https://github.com/cojen/Maker>

⁵<https://github.com/square/javapoet>

- [KotlinPoet](#)⁶

They differ in various aspects, including their ability to be used at compile-time, run-time, or both. Additionally, the last two options, JavaPoet and KotlinPoet, generate source code rather than bytecode.

Among these various alternatives, **ASM** [Bruneton, 2002] stands out as one of the most commonly used technologies, and it is notably utilized in the Kotlin compiler.

A new **Class-File API** was released in JDK 24 in 2025 to provide a standardized and up-to-date solution for class-file processing. The Class-File API was originally proposed as a preview feature by JEP 457 in JDK 22 and refined through JEP 466 in JDK 23.

Any of the available alternatives is suitable for constructing a dynamic version of the `NaiveMapper` discussed in Chapter 3. In this chapter, we provide two implementations: one using the new **Class-File API**, and another using **Cojen Maker**, due to its simplicity and ease of use, while still offering insights into JVM low-level details.

6.2: Class-File API

The Java **Class-File API** provides a standard mechanism for parsing, generating, and transforming Java class files. Parsing and transformation are outside the scope of this book, so we will focus solely on the code generation API.

One characteristic of the Class-File API, shared with other alternatives for code generation on the JVM, is that its API is structured similarly to the Java Reflection API, with similar names and organization. This resemblance is not surprising, as generating code for the JVM must **adhere to the JVM type system**, similar to how the Reflection API operates. Hence, for each `Class`, `Field`, and `Method` in the Java Reflection API, we can find corresponding classes named `ClassBuilder`, `FieldBuilder`, and `MethodBuilder` in the Class-File API, as denoted in [Figure 6.3](#)



Figure 6.3. Java reflect API side by side with Class-File API.

⁶<https://square.github.io/kotlinpoet/>

The builders shown in [Figure 6.3](#) are not directly instantiated by client code but are instead created by passing a **lambda that receives a builder as an argument**. In the listing of [Figure 6.4](#), we show the layout of using the Class-File API, where builders are passed as lambda arguments on lines 1, 4, 5, 11, and 12. For now, ignore the other parameters denoted by “...”. Note how method calls to the builders, such as `withFlags`, `withMethod`, and `withCode`, as well as opcode generators like `aload` and `ldc`, are fluently chained together.

Figure 6.4. Structure of code using the Class-File API Builder pattern

```

ClassFile.of().build(...) { clb: ClassBuilder ->
  clb
    .withFlags(...)
    .withMethod(...) { mb: MethodBuilder ->
      mb.withCode { cob: CodeBuilder ->
        cob
          .aload(0)
          ....
          .return_()
        }
      }.withMethod(...) { mb: MethodBuilder ->
        mb.withCode { cob: CodeBuilder ->
          cob
            .ldc(...)
            ....
            .ireturn()
          }
        }
      }
    }
  }
}

```

The `CodeBuilder` provides several convenience methods such as `aload(n)`, `ldc()`, and others, mimicking the bytecode opcodes. **By capturing the sequence of operations in a lambda, the Class-File library can replay the building process**, allowing it to detect misleading states or illegal accesses.

In the Class-File API, the information regarding the **type of a field** or the **function type of a method** is passed through a **type descriptor**. In the builder methods shown in [Figure 6.5](#), you will find parameters of type `ClassDesc` to specify a **reference type** (e.g., class or interface) and of type `MethodTypeDesc` to identify a **function type**.

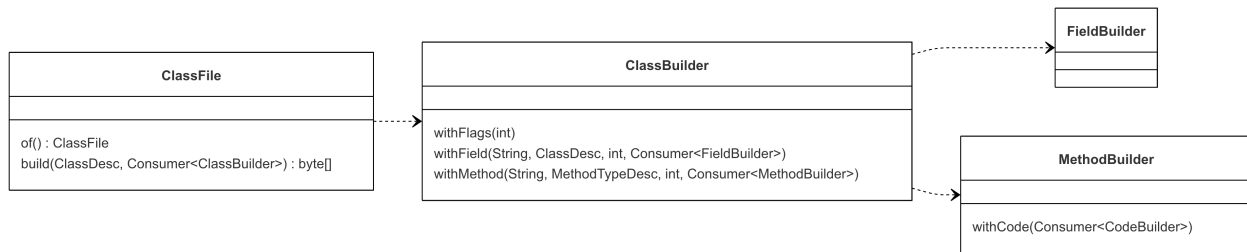


Figure 6.5. Class-File API Builders.

The remaining parameters in Figure 6.5 include a `String` for the name and an `int` for bitwise access flags. The last parameter, a `Consumer`, will receive the function or lambda to which a builder is passed.

Now, let's look at a complete example. Suppose we want to dynamically build a class named `pt.isel.Bar`, which includes a parameterless method `foo` that returns the integer 67895, similar to the following Java definition of `Bar`:

```
package pt.isel;

public class Bar {
    public int foo() { return 67895; }
}
```

The snippet in Figure 6.6 demonstrates how to generate such a class. Constants beginning with `CD` or `MTD` are **type descriptors** or **function type descriptors** for the most commonly used types. For example, `CD_Object` is the type descriptor for `java.lang.Object`, `CD_int` is the descriptor for the Java `int` primitive type, and `MTD_void` is the function type descriptor for a parameterless `void` method. Constants beginning with `ACC` represent accessibility flags, such as `ACC_Public`. Finally, `INIT_NAME` refers to the default name for the constructor, which is `<init>`.

Whenever a required type descriptor is not available among the existing constants, it can be specified using the `ClassDesc.of()` or `MethodTypeDesc.of()` utility methods, which generate type descriptors for reference types and function types, respectively. For example, `ClassDesc.of("pt.isel.Bar")` creates a `ClassDesc` instance representing the type descriptor `Lpt/isel/Bar;`, while `MethodTypeDesc.of(CD_int)` produces a method type descriptor for a parameterless method that returns an `int` value. The `MethodTypeDesc.of()` method has the signature `of(ClassDesc, varargs ClassDesc)`, utilizing `varargs` to support zero or multiple parameter type descriptors.

Figure 6.6. A Kotlin program to generate a Bar class dynamically with Class-File API.

```

import java.io.*
import java.lang.classfile.*
import java.lang.classfile.ClassFile.*
import java.lang.constant.*
import java.lang.constant.ConstantDescs.*

val bytes =
    ClassFile.of().build(ClassDesc.of("pt.isel.Bar")) { clb ->
        clb
            .withFlags(ACC_PUBLIC)
            .withMethod(INIT_NAME, MTD_void, ACC_PUBLIC) { mb ->
                mb.withCode { cob: CodeBuilder ->
                    cob
                        .aload(0)
                        .invokespecial(CD_Object, INIT_NAME, MTD_void)
                        .return_()
                    }
                }
            .withMethod("foo", MethodTypeDesc.of(CD_int), ACC_PUBLIC) { mb ->
                mb.withCode { cob: CodeBuilder ->
                    cob
                        .ldc(clb.constantPool().intEntry(67895))
                        .ireturn()
                    }
                }
            }
    }
File(resourcePath, "pt/isel/Bar.class")
    .also { it.parentFile.mkdirs() } // Create directories if they do not exist
    .writeBytes(bytes)

```

In the snippet from [Figure 6.6](#), the resulting byte array is saved as a `Bar.class` file inside the `pt/isel` folder, located at the path specified by the `resourcePath` variable. This variable `resourcePath` is initialized with the same path used by the class loader that will load the `Bar` class. Remember, a class loader searches for classes in the locations specified in the **classpath**. By saving the class file in the same location used by the class loader, we ensure that it will be found when we attempt to load it.

It is **not mandatory to save the byte array as a class file** in order to load it with a class loader. However, in our metaprogramming development cycle, **it is useful to include this step as an intermediate phase**, allowing us to verify that everything is correct, inspect the generated bytecode, and debug potential issues before dynamically loading and executing the class.

In the listing of [Figure 6.7](#), we first initialize `resourcePath` with the base path of the runtime resources, obtained from the location of the compiled classes via `getResource("/")`. If this lookup fails (i.e., returns `null`), we explicitly fall back to the current working directory using `System.getProperty("user.dir")`, ensuring that a valid filesystem path is always available.

Next, we create a `URLClassLoader` pointing to that path, allowing us to load classes dynamically from the filesystem. Finally, we load the class `pt.isel.Bar`, obtain its Kotlin reflection representation, create an instance using `createInstance`, retrieve the function `foo`, and invoke it on the created object, printing the result.

Figure 6.7. Loading the `Bar` class and invoking its instance method `foo`.

```
private val resourcePath = Unit::class.java
    .getResource("/")?.toURI()?.path
    ?: "${System.getProperty("user.dir")}/"

URLClassLoader(arrayOf(File(resourcePath).toURI().toURL()))
    .loadClass("pt.isel.Bar")
    .kotlin
    .also { klass ->
        val obj = klass.createInstance()
        val foo = klass.functions.first { it.name == "foo" }
        println(foo.call(obj)) // prints 67895
    }
```

Let's delve into a more complex example by dynamically constructing a `Counter` class. This class will feature an integer field and a method `add`, which computes the sum of that field with another integer provided as an argument. Additionally, this class should implement the `Sum` interface, which outlines the contract for the `add` method according to the definition of [Figure 6.8](#)

Figure 6.8. Interface `Sum` with a single method `add`.

```
package pt.isel;

interface Sum {
    fun add(other: Int): Int
}
```

To guide the implementation of the `buildAndSaveCounterClass()` function, which dynamically generates `Counter.class` using the Class-File API, it is important to **follow a reference implementation (i.e., a baseline) that closely resembles the desired output**. We achieve this by creating a Java class, `CounterBaseline`, based on the definition in [Figure 6.9](#).

Figure 6.9. CounterBaseline class equivalent to the desired dynamically generated implementation.

```
package pt.isel;

class CounterBaseline implements Sum {
    private final int nr;

    public CounterBaseline(int nr) { this.nr = nr; }

    public int add(int other) { return this.nr + other; }
}
```

Ignoring the use of `withCode` to emit the bytecode for the method implementations in the listing of Figure 6.10, we can outline the construction of the class structure, similar to the definition in Figure 6.9, including:

1. Implementation of the `Sum` interface on line 6.
2. Definition of the `nr` field on line 7.
3. Constructor definition on line 8, which accepts an `int` parameter.
4. Definition of the `add` method, which takes an `int` parameter and returns an `int`.

Figure 6.10. Layout of the `buildAndSaveCounterClass()` using the Class-File API.

```
1 private fun buildAndSaveCounterClass() {
2     val counterClassDesc = ClassDesc.of("pt.isel.Counter")
3     val sumClassDesc = ClassDesc.of("pt.isel.Sum")
4     val bytes = ClassFile.of().build(counterClassDesc) { clb: ClassBuilder ->
5         clb
6             .withInterfaces(Interfaces.ofSymbols(sumClassDesc).interfaces())
7             .withField("nr", CD_int, ACC_PRIVATE or ACC_FINAL)
8             .withMethod(INIT_NAME, MethodTypeDesc.of(CD_void, CD_int), ACC_PUBLIC) { mb ->
9                 mb.withCode { cob -> cob... }
10            }.withMethod("add", MethodTypeDesc.of(CD_int, CD_int), ACC_PUBLIC) { mb ->
11                mb.withCode { cob -> cob... }
12            }
13    }
14    FileOutputStream(File(resourcePath, "pt/isel/Counter.class")).write(bytes)
15 }
```

The rest of the implementation of the `buildAndSaveCounterClass()` could be quite tedious when it comes to emitting bytecode opcodes, and errors can be difficult to fix if we try to guess or mentally construct the bytecode. However, it can also be a straightforward task if we are systematic and follow a strict process. The recommended approach is **not to guess, but instead, emit bytecode based on the bytecode output of the desired baseline**, which in this case is the `CounterBaseline.class`.

Figure 6.11. CounterBaseline class output of javap.

```

class pt.isel.CounterBaseline implements pt.isel.Sum {
    public pt.isel.CounterBaseline(int);
    Code:
        0: aload_0
        1: invokespecial #1    // Method java/lang/Object."<init>":()V
        4: aload_0
        5: iload_1
        6: putfield      #7    // Field nr:I
        9: return

    public int add(int);
    Code:
        0: aload_0
        1: getfield      #7    // Field nr:I
        4: iload_1
        5: iadd
        6: ireturn
}

```

By examining the bytecode output for the CounterBaseline.class using javap, as shown in [Figure 6.11](#), we can easily mimic the opcodes using the CodeBuilder API, as demonstrated in the listing in [Figure 6.12](#).

Figure 6.12. buildAndSaveCounterClass() emitting bytecode through CodeBuilder API.

```

private fun buildAndSaveCounterClass() {
    ....
    .withMethod(INIT_NAME, ... ) { mb ->
        mb.withCode { cob ->
            cob
                .aload(0)
                .invokespecial(CD_Object, INIT_NAME, MTD_void)
                .aload(0)
                .iload(1)
                .putfield(counterClassDesc, "nr", CD_int)
                .return_()
            }
        }
    }.withMethod("add", ... ) { mb ->
        mb.withCode { cob ->
            cob
                .aload(0)

```

```

        .getField(counterClassDesc, "nr", CD_int)
        .iload(1)
        .iadd()
        .ireturn()
    }
}
}
FileOutputStream(File(resourcePath, "pt/isel/Counter.class")).write(bytes)
}

```

Finally, we can instantiate the generated Counter class and call its instance method add, as shown in the code in [Figure 6.13](#). We will follow the same approach to load the Counter.class as before, using a class loader. After that, we create an instance of the class by invoking its constructor and then call its add method. However, **instead of using reflection to invoke** the add method, we can directly call it because we have a reference of type Sum.

Figure 6.13. Instantiating the dynamically generated Counter class and invoking add.

```

val loader = URLClassLoader(arrayOf(File(resourcePath).toURI().toURL()))
val counter: Sum =
    buildAndSaveCounterClass()
        .let { loader.loadClass("pt.isel.Counter").kotlin }
        .constructors
        .first()
        .call(7) as Sum // <=> val count = new Count(7)
println(counter.add(11)) // > 18

```

6.3: Dynamic Mapper

Now we will implement a similar feature to that provided by the NaiveMapper in Chapter 3, but without using reflection in the implementation of its mapFrom(src: T): R method. To achieve this, we will define an interface called Mapper that specifies the contract for the mapFrom method according to definition of [Figure 6.14](#). This interface Mapper will be implemented by a dynamically generated class.

Figure 6.14. Kotlin interface Mapper implemented by dynamically generated classes.

```

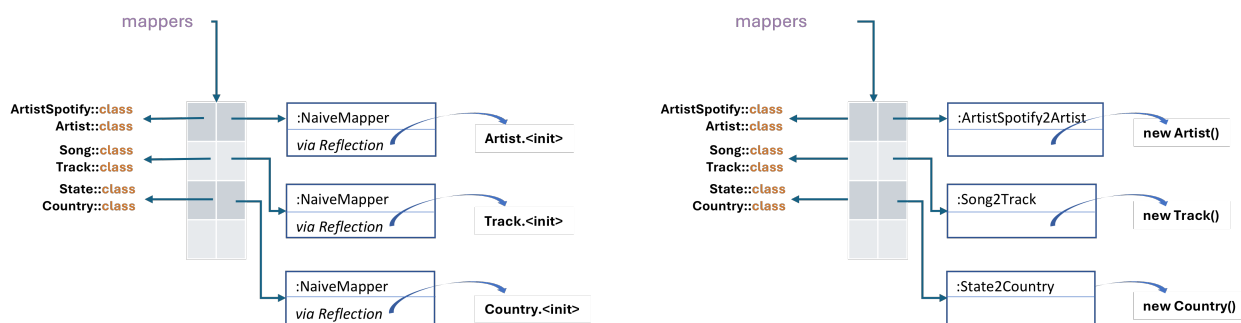
interface Mapper<T, R> {
    fun mapFrom(src: T): R

    fun mapFromList(src: List<T>): List<R> {
        return src.map { this.mapFrom(it) }
    }
}

```

The Mapper interface also provides a default implementation of the `mapFromList` method, which returns a new list with each element transformed by the `methodFrom`. Note that for older versions of the Kotlin compiler (prior to 2.2.0), non-abstract methods in Kotlin interfaces did not fully comply with the JVM's default method model. Since we are using a bytecode instrumentation library targeting the JVM, it is preferable to align with standard JVM semantics to simplify bytecode generation. In those earlier versions, the compiler option `-Xjvm-default=all` could be used to ensure that all non-abstract members of Kotlin interfaces are compiled as default methods in the generated bytecode, making them directly compatible with Java classes implementing those interfaces. **In recent Kotlin versions, the compiler adheres to the JVM default method model by default, and no additional configuration is required to obtain this behavior.**

One major difference from the former `NaiveMapper` approach using reflection is that now we will have a **distinct class implementing Mapper** for each pair of source and destination classes. With the `NaiveMapper` approach, we use the same type of mapper to map any class types. For each source-destination pair of classes, we need a new `NaiveMapper` instance, initializing different values for its `destInit` constructor and props container. However, all these instances were of the same type (i.e., `NaiveMapper`). In contrast, now we'll have a separate class for each mapping scenario. For example, to map from `ArtistSpotify` to `Artist`, we'll use an instance of a dynamically generated class `ArtistSpotify2Artist`, and to map from `Song` to `Track`, we'll use an instance of a dynamically generated class `Song2Track`, as presented in the example of Figure 6.15

Figure 6.15. A single class `NaiveMapper` versus many dynamically generated mappers.

Instead of having a single `DynamicMapper` class, we define a function `buildMapperClassfile` that returns a `KClass` representing the dynamically generated mapper class, as shown in the listing of [Figure 6.16](#). Following the approach used earlier, it first builds a byte array with the class definition, writes it to the corresponding `.class` file, and then loads the class using a class loader. Note that the name of the mapper class follows the pattern `src.simpleName2dest.simpleName`, where `src` and `dest` refer to the source and destination domain classes, respectively.

Figure 6.16. Building and loading the class for a dynamically generated mapper.

```
private const val PACKAGE_NAME = "pt.isel"

private val resourcePath = Unit::class.java
    .getResource("/")?.toURI()?.path
    ?: "${System.getProperty("user.dir")}/"

private val loader = URLClassLoader(arrayOf(File(resourcePath).toURI().toURL()))

fun <T : Any, R : Any> buildMapperClassfile(
    src: KClass<T>,
    dest: KClass<R>,
): KClass<out Any> {
    val className = "$PACKAGE_NAME.${src.simpleName}2${dest.simpleName}"
    buildMapperByteArray(className, src, dest)
    return loader.loadClass(className).kotlin
}

fun <T : Any, R : Any> buildMapperByteArray(...) { ... }
```

The `buildMapperByteArray` function, referenced in [Figure 6.16](#), is responsible for generating the byte array representing the class definition and saving it to a `.class` file. This function shown in [Figure 6.17](#) begins by gathering the constructor of the destination class and establishing the mapping between the properties in the source class and the parameters of that constructor. The algorithms for selecting the constructor and mapping properties are the same as those used in `NaiveMapper` in [Figure 3.19](#), using the predicate specified in [Figure 3.21](#). They are omitted in lines 8 and 9 of [Figure 6.17](#), where they are encapsulated in the auxiliary functions `findConstructor` and `findMatchingProperties`.

Figure 6.17. Building and saving the byte array definition of the generated class.

```

1 fun <T : Any, R : Any> buildMapperByteArray(
2     className: String,
3     src: KClass<T>,
4     dest: KClass<R>,
5 ) {
6     val destInit = findConstructor(src, dest)
7     val props: Map<KProperty<*>, KParameter> = findMatchingProperties(src, destInit)
8
9     // Descriptor of the generated class
10    val classDesc = ClassDesc.of(className)
11    val mapperInterface = Interfaces.ofSymbols(Mapper::class.descriptor())
12    ClassFile
13        .of()
14        .build(classDesc) { clb: ClassBuilder ->
15            clb
16                .withFlags(ACC_PUBLIC)
17                .withInterfaces(mapperInterface.interfaces())
18                .withMethod(INIT_NAME, MTD_void, ACC_PUBLIC) { mb: MethodBuilder ->
19                    mb.withCode { cob: CodeBuilder ->
20                        cob
21                            .aload(0)
22                            .invokespecial(CD_Object, INIT_NAME, MTD_void)
23                            .return_()
24                    }
25                }.withMethod(
26                    "mapFrom",
27                    MethodTypeDesc.of(CD_Object, CD_Object),
28                    ACC_PUBLIC,
29                ) { mb ->
30                    mb.withCode { cob -> cob.buildMapFrom(src, dest, props, destInit) }
31                }
32        }.also { bytes ->
33            File(resourcePath, "${className.replace(".", "/")}.class")
34                .also { it.parentFile.mkdirs() }
35                .writeBytes(bytes)
36        }
37 }

```

In the `buildMapperByteArray` function shown in Figure 6.17, we define the structure of the dynamically generated mapper, including its implementation of the `Mapper` interface (line 19), a constructor (line 20), and the `mapFrom` method (line 27). The constructor follows the same structure as

the parameterless constructor previously shown in [Figure 6.6](#), simply invoking the base constructor of `Object` in this case. The generation of the `mapFrom` bytecode is delegated to the auxiliary extension function `buildMapFrom` (called in line 32), which will be introduced later. Note the function type descriptor of `mapFrom`, which omits generics and uses `Object` as both the parameter and return type, represented by `CD_Object` (line 29). We will revisit this topic in a few paragraphs.

Throughout our implementation, we often need to obtain the `ClassDesc` instance for various types (e.g. line 13 of [Figure 6.17](#)). To facilitate this, we have created auxiliary extension functions named `descriptor`, shown in [Figure 6.18](#), which return the corresponding `ClassDesc` for a given `KType` or `KClass` from the Kotlin Reflection API.

Figure 6.18. `KClass` and `KType` extension functions to get the corresponding `ClassDesc` instance.

```
fun KClass<*>.descriptor(): ClassDesc =
    if (this.java.isPrimitive) {
        when (this) {
            Char::class -> CD_char
            Short::class -> CD_short
            Int::class -> CD_int
            Long::class -> CD_long
            Float::class -> CD_float
            Double::class -> CD_double
            Boolean::class -> CD_boolean
            else -> {
                throw IllegalStateException("No primitive type for ${this.qualifiedName}!")
            }
        }
    } else {
        ClassDesc.of(this.java.name)
    }

fun KType.descriptor(): ClassDesc {
    val klass = this.classifier as KClass<*>
    return klass.descriptor()
}
```

To guide the code generation by `buildMapFrom()`, **it's important to follow a baseline similar to an example of a class that we intend to generate**. This baseline should be written in Java, which is closer to generated bytecode and to the way we use the Class-File API. For example, the listing in the [Figure 6.19](#) presents a baseline implementation of a mapper that transforms `ArtistSpotify` objects into `Artist` objects. At this stage, we are only mapping properties that have the same type as the constructor parameters. These properties are directly copied from the source to the destination object, without the need for any transformation via an auxiliary mapper.

Figure 6.19. Baseline implementation of ArtistSpotify2Artist mapper.

```
public class ArtistSpotify2ArtistBaseline implements Mapper<ArtistSpotify, Artist> {
    public Artist mapFrom(ArtistSpotify src) {
        return new Artist(src.getKind(), src.getName());
    }
}
```

The result of the bytecode decompilation using javap is shown in Figure 6.20, where we include only the method body of the mapFrom(Object) method. It might seem odd to see the mapFrom method defined twice—once with the expected signature mapFrom(ArtistSpotify) and another version that takes an Object. This happens because of **type erasure** in Java.

The method mapFrom(ArtistSpotify) is the one we explicitly define in the source code, but it doesn't technically override the generic mapFrom(Object) method from the Mapper interface after compilation. To handle this, the compiler automatically generates a **bridge method**—a synthetic method that matches the erased signature. This bridge method takes an Object, casts it to ArtistSpotify, and then delegates the call to the strongly typed version (i.e. mapFrom(ArtistSpotify)).

Figure 6.20. Bytecode of baseline implementation of ArtistSpotify2Artist mapper.

```
public class ArtistSpotify2ArtistBaseline
    implements pt.isel.Mapper<ArtistSpotify, Artist> {

    public ArtistSpotify2ArtistBaseline();

    public Artist mapFrom(ArtistSpotify);

    public java.lang.Object mapFrom(java.lang.Object);
    Code:
        0: aload_0
        1: aload_1
        2: checkcast    #10  // class ArtistSpotify
        5: invokevirtual #21  // Method mapFrom:(LArtistSpotify;)LArtist;
        8: areturn
}
```

The checks for type safety that Kotlin and Java perform with generic declarations occur during compile time. However, at runtime, instances of generic types lack any information about their actual type arguments. This lack of type information is known as type erasure.

We can follow an approach in our buildMapperByteArray() generator that closely mimics the two mapFrom methods shown in Figure 6.20. However, this can be simplified by adopting a baseline

implementation that avoids generics altogether—mirroring the effect of **type erasure** seen in the generated bytecode.

Thus, for an `ArtistSpotify2Artist` mapper, we can consider the Java implementation shown in [Figure 6.21](#), written without generics. In this case, the generated bytecode includes just one method: `mapFrom(Object)`.

Figure 6.21. Baseline implementation of `ArtistSpotify2Artist` mapper without generics.

```
public class ArtistSpotify2ArtistBaseline implements Mapper {

    public Object mapFrom(Object src) {
        // 1. cast src to ArtistSpotify
        ArtistSpotify dto = (ArtistSpotify) src;

        // 2. For each property read its value from src
        // 3. Instantiate Artist with the values of src properties
        return new Artist(dto.getKind(), dto.getName());
    }
}
```

For the implementation in [Figure 6.21](#), we can observe the bytecode output shown in [Figure 6.22](#), obtained using the command `javap -c ArtistSpotify2Artist.class`. It begins by loading the input parameter (`aload_1`) and casting it to the expected source type `ArtistSpotify` (`checkcast #7`), storing it in a local variable (`astore_2`). A new instance of the target class `Artist` is then created (`new #9`). The `dup` instruction is used to duplicate the reference to the newly created object, preserving it on the stack for use as the target of the constructor call—since the `invokespecial` instruction will consume it as the first argument when calling `Artist`’s constructor. The method proceeds to load the constructor arguments by invoking `getKind()` and `getName()` on the source object (`invokevirtual #11` and `#15`). These values, along with the duplicated reference, are used in the `invokespecial #18` to instantiate the `Artist` object, which is finally returned (`areturn`).

Figure 6.22. Bytecode of baseline implementation of `ArtistSpotify2Artist` mapper.

```
public class ArtistSpotify2ArtistBaseline implements pt.isel.Mapper {
    public ArtistSpotify2ArtistBaseline();
    Code:
        0: aload_0
        1: invokespecial #1 // Method java/lang/Object."<init>":()V
        4: return

    public java.lang.Object mapFrom(java.lang.Object);
    Code:
        0: aload_1
```

```

1: checkcast      #7  // class ArtistSpotify
4: astore_2
5: new            #9  // class Artist
8: dup
9: aload_2
10: invokevirtual #11 // ArtistSpotify.getKind():()Ljava/lang/String;
13: aload_2
14: invokevirtual #15 // ArtistSpotify.getName():()Ljava/lang/String;
17: invokespecial #18 // Artist."<init>":(Ljava/lang/String;Ljava/lang/String;)V
20: areturn
}

```

The implementation of `buildMapFrom()` of Figure 6.23 mirrors the bytecode shown in Figure 6.22, where for each property in the source object, it emits the corresponding `aload_2` instruction to push the source object reference onto the stack, followed by an `invokevirtual` call to the property's getter method. Note that when emitting the `invokespecial` instruction to call the constructor, we need to specify the corresponding method descriptor. To do this, we build the descriptor with a void return type and a list of `ClassDesc` instances derived from the constructor parameters (i.e. `destInitParamsDesc` in line 24).

Figure 6.23. Code builder extension function `buildMapFrom` to emit bytecode of method `mapFrom`.

```

1 fun <T : Any, R : Any> CodeBuilder.buildMapFrom(
2     src: KClass<T>,
3     dest: KClass<R>,
4     props: Map<KProperty<*>, KParameter>,
5     destInit: KFunction<R>,
6 ) {
7     // e.g. ArtistSpotify dto = (ArtistSpotify) src;
8     aload(1)
9     checkcast(src.descriptor())
10    astore(2)
11    // e.g. new Artist
12    new_(dest.descriptor())
13    dup()
14    // e.g. dto.getKind()
15    props.forEach { (srcProp, destParam) ->
16        aload(2)
17        invokevirtual(
18            src.descriptor(),
19            srcProp.javaGetter?.name,
20            MethodTypeDesc.of(srcProp.returnType.descriptor()),
21        )

```

```

22     }
23     // e.g. Artist(...)
24     val destInitParamsDesc = destInit.parameters.map { it.type.descriptor() }
25     invokespecial(
26         dest.descriptor(),
27         INIT_NAME,
28         MethodTypeDesc.of(CD_void, destInitParamsDesc),
29     )
30     areturn()
31 }

```

The `buildMapFrom` emits a call to the constructor, ensuring that only non-optional parameters are passed. Unlike `callBy()` used with reflection, which deals with optional parameters by passing their default values, in this case, we are excluding some optional parameters from the generated constructor invocation (i.e., `invokespecial`). To achieve this, **the destination class needs to provide different constructors with a varying number of parameters based on the number of existing optional parameters**. Therefore, it is imperative that the definition of the destination domain class includes the `@JvmOverloads` annotation on its constructor to instruct the Kotlin compiler to generate overloads that substitute default parameter values. For example, the class `Artist` should be defined as shown in the following listing:

```

class Artist @JvmOverloads constructor(
    val kind: String, // "Group", "Person"
    val name: String,
    val country: Country = Country("", ""),
    val tracks: List<Track> = emptyList()
)

```

To support properties of complex types, such as other domain classes, we need to generate code capable of obtaining a `Mapper` for the corresponding type to transform the property's value. To achieve this goal, we adopt a method akin to the one utilized in [Figure 3.25 of Section 3.7](#), maintaining all instantiated mappers within an internal `Map`. This strategy prevents redundant instantiation of mappers for identical source and destination classes. Consequently, when a new dynamic mapper is needed, it can be obtained through the `loadDynamicMapper` function shown in [Figure 6.24](#).

Figure 6.24. `loadDynamicMapper` retrieves or creates a dynamic mapper instance for a given source and destination class.

```

1 private val mappers = mutableMapOf<Pair<Class<*>, Class<*>>, Mapper<*, *>>()
2
3 fun <T : Any, R : Any> loadDynamicMapper(
4     srcType: Class<T>,
5     destType: Class<R>,
6 ) = mappers.getOrPut(srcType to destType) {
7     buildMapperClassfile(srcType.kotlin, destType.kotlin)
8         .createInstance() as Mapper<*, *>
9 } as Mapper<T, R>
10
11 fun <T : Any, R : Any> loadDynamicMapper(
12     srcType: KClass<T>,
13     destType: KClass<R>,
14 ) = loadDynamicMapper(srcType.java, destType.java)

```

Note that in line 7 of Figure 6.24, the `buildMapperClassfile` function is invoked only if `getOrPut` does not find an existing `Mapper` instance for the given pair of `srcType` and `destType`. There is an overloaded version of `loadDynamicMapper` that accepts parameters of type `KClass` instead of `Class`. It converts the Kotlin `KClass<T>` and `KClass<R>` to their Java equivalents using `.java`, and delegates the call to the main `loadDynamicMapper(KClass<T>, KClass<R>)` function. This overload improves interoperability with Kotlin-based code, while the `Class`-based version handles the actual logic of generating and caching the dynamic mapper.

When mapping an `ArtistSpotify` to an `Artist`, as detailed in Figure 3.23, auxiliary mappers are needed to handle the conversion of `State` to `Country` and `Song` to `Track`. The baseline `mapFrom` implementation that handles this scenario is presented in Figure 6.25.

Figure 6.25. Baseline implementation of `mapFrom` handling complex properties.

```

public class ArtistDto2ArtistBaseline implements Mapper {
    public Object mapFrom(Object src) {
        ArtistSpotify dto = (ArtistSpotify) src;
        return new Artist(
            dto.getKind(),
            dto.getName(),
            loadDynamicMapper(State.class, Country.class).mapFrom(dto.getOrigin()),
            loadDynamicMapper(Song.class, Track.class).mapFromList(dto.getSongs())
        );
    }
}

```

In the bytecode listing of Figure 6.26, we present a sample illustrating how the value of a property—such as origin of type `State`—is mapped using an auxiliary mapper.

Figure 6.26. Bytecode for mapping a property through an auxiliary mapper.

```

1 17: aload_2
2 18: invokevirtual #25    // Method ArtistSpotify.getOrigin:()LState;
3 21: astore_3
4 22: ldc          #27    // class State
5 24: ldc          #29    // class Country
6 26: invokestatic #35    // Method DynamicMapperClassfileKt.loadDynamicMapper:
7                        //      (Ljava/lang/Class;Ljava/lang/Class;)LMapper;
8 29: aload_3
9 30: invokeinterface #39 // Mapper.mapFrom:(Ljava/lang/Object;)Ljava/lang/Object;
10 35: checkcast   #29    // class pt/isel/Country

```

The bytecode in the listing of [Figure 6.26](#) starts by retrieving the property value, as seen earlier in [Figure 6.22](#). Afterward, at offset 21 (line 3), the value is stored in an auxiliary local variable with index 3. The bytecode then continues by loading the `State` and `Country` classes at offsets 22 and 24. It then calls `DynamicMapperClassfileKt.loadDynamicMapper()` at offset 26, passing in the `State` and `Country` classes as arguments to retrieve the auxiliary mapper. The `State` object, stored earlier in local variable 3, is then passed to the auxiliary mapper at offset 29 by invoking the `mapFrom` method. The result of this mapping is finally cast to the `Country` type at offset 35.

For the property `songs` of type `List<Song>`, the bytecode is similar, with the exception that we will invoke the default method `mapFromList` depicted in [Figure 6.14](#), instead of `mapFrom`. Also, there is no need for the final casting because the type of the property in this case will always be `List`.

To emit bytecode that supports mapping properties for these scenarios, we need to update the `props.forEach` block in lines 15-22 of [Figure 6.23](#). After retrieving the value of a property, if its type differs from the destination parameter type, we need to check whether it is neither a primitive type nor a `String`. In such cases, we assume that the property can be mapped through an auxiliary mapper, and we will emit the equivalent bytecode as shown in the listing of [Figure 6.26](#).

To achieve this, we rely on two auxiliary extension functions. One, the `toKClass()` extension function, is used to obtain the `KClass` of the type corresponding to the classifier itself or the generic argument of a `List`, if applicable. The other function checks whether a given class is a primitive type or a `String`.

Figure 6.27. Extension functions toKClass and isPrimitiveOrString.

```

fun KType.toKClass() =
    if (classifier == List::class) {
        arguments[0]
            .type
            ?.classifier as KClass<*>
    } else {
        classifier as KClass<*>
    }

fun KClass<*>.isPrimitiveOrString() = this.java.isPrimitive || this == String::class

```

In the listing of Figure 6.28, we present a sample of code that will be interleaved in the `props.forEach` block between lines 15-22 of Figure 6.23, after retrieving the value of a property.

Figure 6.28. Code snippet for mapping properties with complex values.

```

if (srcProp.returnType != destInit.returnType &&
    !srcProp.returnType.toKClass().isPrimitiveOrString()
) {
    // stores the property value in a local variable (index3)
    astore(3)
    // Put arguments on Stack: e.g. State.class and Country.class
    ldc(constantPool().classEntry(srcProp.returnType.toKClass().descriptor()))
    ldc(constantPool().classEntry(destParam.type.toKClass().descriptor()))
    // call DynamicMapperClassfileKt.loadDynamicMapper(Class, Class)
    invokestatic(
        ClassDesc.of("pt.isel.DynamicMapperClassfileKt"), // owner,
        "loadDynamicMapper",
        MethodTypeDesc.of(
            Mapper::class.descriptor(), // return type
            Class::class.descriptor(), // 1st parameter
            Class::class.descriptor(), // 2nd parameter
        ),
    )
    // Loads the property value to be passed as an argument to mapFrom or mapFromList
    aload(3)
    if (srcProp.returnType.classifier != List::class) {
        invokeinterface(
            Mapper::class.descriptor(),
            "mapFrom",
            MethodTypeDesc.of(CD_Object, CD_Object),
        )
    }
}

```

```

    )
    checkcast(destParam.type.descriptor())
  } else {
    invokeinterface(
      Mapper::class.descriptor(),
      "mapFromList",
      MethodTypeDesc.of(CD_List, CD_List),
    )
    // No need of casting since mapFromList returns a type erased List
  }
}

```

The code snippet of [Figure 6.28](#) first checks if the property type of `srcProp` is neither a primitive nor a `String`. If so, it proceeds to map the property using a dynamic mapper. It stores the value of the property in a local variable (index 3) with `astore(3)`. Then, it pushes the class types of both the source property and the destination parameter onto the stack using `ldc`, referencing their descriptors. After this, the code calls the `loadDynamicMapper` function from the `DynamicMapperClassfileKt` class, passing in the class types as arguments. It then loads the stored value with `aload(3)`. If the property is not a list, it calls the `mapFrom` method from the `Mapper` interface using `invokeinterface`, and the result is cast to the appropriate type with `checkcast`. If the property is a list, it calls the `mapFromList` method instead, and no casting is needed because `mapFromList` returns a type-erased list.

6.4: Cojen Maker

6.4.1: Cojen Maker Features

The **Cojen Maker** library describes itself as a “*lightweight, full-featured, low-level dynamic Java class generator which is designed for ease of use*”.

As we have seen before with the Class-File API, the Cojen Maker API resembles the same organization as the Reflection Types. Hence, for each `Class`, `Field`, and `Method` in the Java Reflection API, we can find corresponding classes named `ClassMaker`, `FieldMaker`, and `MethodMaker` in the Cojen Maker API.

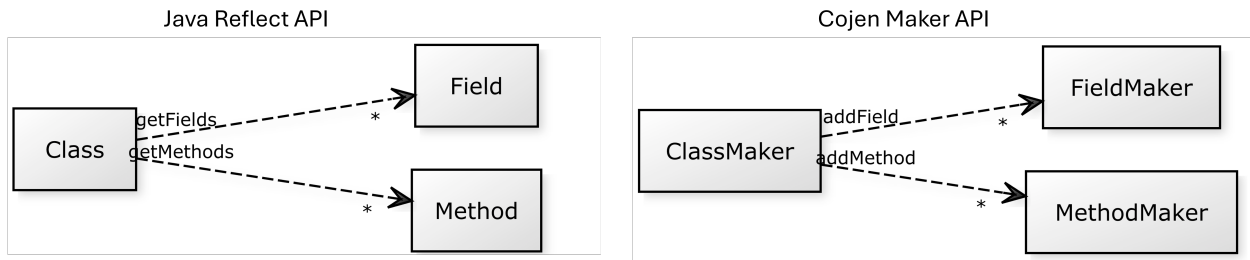


Figure 6.29. Java reflect API side by side with Cojen Maker API.

Furthermore, Cojen Maker relies on the Java Reflection API to refer to Java types. Whenever we need to identify a type, such as declaring a method return type or a field type, we can use a `Class` instance to specify that type. However, Cojen Maker also permits the use of type names as String literals. Despite this convenience, we will refrain from using this approach due to its lack of type safety. Because we will demonstrate our examples using the Kotlin programming language, when obtaining a representation of a class using `::class`, we then **chain the `.java` property to obtain the corresponding `Class` instance**.

Finally, Cojen Maker offers a fluent API that allows chaining method calls one after another, where the result of one method becomes the receiver of the next method call. Many of these methods have names similar to existing Java keywords, indicating that they serve as **builders** for those features. However, since these names are reserved keywords in Java, Cojen Maker appends an underscore suffix (i.e., `_`) to these method names to avoid name collisions. For example, builder methods such as `return_()` or `new_()` emit the corresponding opcodes for `return` and `new`, respectively.

6.4.2: Cojen Maker Examples

Let's consider that we want to dynamically build a class named `Bar`, which includes a parameterless method `foo` that returns the integer number 67895. The snippet in [Figure 6.30](#) demonstrates how to generate such a class.

Figure 6.30. A Kotlin program to generate a `Bar` class dynamically with Cojen Maker.

```

val cm: ClassMaker = ClassMaker
    .begin("Bar")
    .public_()
    .also { it
        .addConstructor()
        .public_()
        .invokeSuperConstructor()
    }
    .also { it
        .addMethod(Int::class.java, "foo")
        .public_()
    }

```

```

        .return_(67895)
    }
    cm.finishTo(FileOutputStream("${cm.name()}.class"))

```

Note that in the snippet shown in [Figure 6.30](#), `also` is not a Cojen builder method; instead, it is a **Kotlin scope function** that allows us to chain the creation of the constructor and the method `foo` without breaking the statement. In this case, the lambda argument it refers to the receiver of the call to `also`, which is the `ClassMaker`.

According to the JVM Type System, if we need to instantiate such a class, it is mandatory to provide an instance constructor. In turn, this constructor should call the constructor of its superclass.

In the end, the resulting class is saved into a file conforming to the `.class` file format. Despite naming the class `Bar`, the `ClassMaker` `begin` method appends a suffix with a random integer to ensure uniqueness. Therefore, we may find our class named, for example, `Bar-5.class`, or with any other suffix number.

Inspecting the resulting class with `javap -c` we may observe an output similar to:

```

public class Bar-5 {
    public Bar-5();
        Code:
            0: aload_0
            1: invokespecial #8    // Method java/lang/Object."<init>":()V
            4: return

    public int foo();
        Code:
            0: ldc          #13    // int 67895
            2: ireturn
}

```

Alternatively, instead of saving the resulting class, we may load it and obtain an instance of `Class` representing the dynamically generated class. To achieve this, we use the builder method `finish()` rather than `finishTo()`. In the following listing, we call `finish()` and chain the call to `.kotlin` to obtain the corresponding `KClass` in the Kotlin Reflect API. After that, we retrieve the `KFunction` for the method `foo` and invoke it with an instance of `kclass` as the receiver.

```

...
val klass: KClass<*> = cm.finish().kotlin
val result = klass
    .memberFunctions
    .first { it.name == "foo" }
    .call(klass.createInstance())
println(result)

```

Now, consider the previous example of dynamically constructing a Counter class that implements the Sum interface, as shown in [Figure 6.8](#). The function `buildAndLoadCounterClass()` of [Figure 6.31](#) dynamically constructs the Counter class according to the previously outlined specifications. Note that in this case, since our class is implementing the interface Sum, we may safely cast the resulting KClass to KClass<Sum>.

Figure 6.31. A Kotlin function to generate a Counter class dynamically.

```

fun buildAndLoadCounterClass(): KClass<Sum> {
    return ClassMaker
        .begin("Counter")
        .public_().implement(Sum::class.java)
        .also { it
            .addField(Int::class.java, "nr")
            .private_().final_()
        }
        .also { it
            .addConstructor(Int::class.java)
            .public_()
            .also { mm: MethodMaker ->
                mm.invokeSuperConstructor()
                mm.this_().field("nr").set(mm.param(0))
            }
        }
        .also { it
            .addMethod(Int::class.java, "add", Int::class.java, )
            .public_().override()
            .also { mm: MethodMaker ->
                val res: Variable = mm.this_().field("nr").add(mm.param(0))
                mm.return_(res)
            }
        }
        .finish()
        .kotlin as KClass<Sum>
}

```

In Cojen Maker, method parameters are identified by an index, similar to how it's done in Java bytecode. However, for instance methods, **parameter 0 in Cojen refers to the first formal parameter and not to this**, as is the case in bytecode. Instead, to access `this`, Cojen Maker provides the special builder `this_()`. The builder `params(int)` is used to refer to the formal parameters.

An instance of the `Variable` class in the Cojen Maker API represents an intermediate result within a method implementation, corresponding to the result of an expression, a method call, or other computations. In the statement `val res = mm.this_().field("nr").add(mm.param(0))` from [Figure 6.31](#), each method builder returns a `Variable`, and we can decompose it into the following statements:

```
val self: Variable = mm.this_()
val nr: Variable = self.field("nr")
val other: Variable = mm.param(0)
val res: Variable = nr.add(other)
mm.return_(res)
```

In this breakdown, `self`, `nr`, `other`, and `res` are all instances of the `Variable` class, representing different intermediate results. Note that the number of resulting local variables in bytecode may not necessarily be equal to the number of instances of `Variable` manipulated in Cojen Maker.

Finally, we can instantiate the generated class `Counter` and call its instance method `add`. Since `buildAndLoadCounterClass()` returns a `KClass<Sum>`, its reflective constructor will be an instance of `KFunction<Sum>`, which will return an instance of `Sum` rather than `Any` or `Object`. Note that in the statement `println(counter.add(11))` of [Figure 6.32](#), we call the method `add` directly without using reflection.

Figure 6.32. Instantiating the dynamically generated `Counter` class and invoking `add`.

```
val counter: Sum = buildAndLoadCounterClass()
    .constructors
    .first()
    .call(7)           // <=> val counter = new Counter(7)
println(counter.add(11)) // > 18
```

6.4.3: Dynamic Mapper with Cojen Maker

We now present an equivalent implementation of the dynamic mapper from [Section 6.3](#) using Cojen Maker instead of the Class-File API. The function `buildDynamicMapper` of [Figure 6.33](#) returns the `ClassMaker` representing the dynamically generated mapper. This function begins by gathering the constructor of the destination class and establishing the mapping between the properties in the source class and the parameters of that constructor. The criteria for selecting that constructor and mapping properties are exactly the same as those used in the `NaiveMapper` of [Figure 3.19](#), and are omitted in [Figure 6.33](#).

Figure 6.33. Building the ClassMaker for a dynamically generated mapper.

```

fun <T : Any, R : Any> buildDynamicMapper(srcType: KClass<T>, destType: KClass<R>)
    : ClassMaker
{
    val destInit = findConstructor(src, dest)
    val props: Map<KProperty<*>, KParameter> = findMatchingProperties(src, destInit)
    return ClassMaker
        .begin("Mapper${src.simpleName}2${dest.simpleName}")
        .public_()
        .implement(Mapper::class.java)
        .apply {
            addConstructor()
                .public_()
                .invokeSuperConstructor()
            addMethodMapFrom(src, dest, props)
        }
}

```

In the `buildDynamicMapper` method, the `apply` function is a Kotlin scope function, not a Cojen Maker builder. We use it to make further calls to the `ClassMaker` instance, which acts as the context object available inside the lambda as the implicit `this`.

The `addMethodMapFrom` of Figure 6.34 is an extension function of `ClassMaker` that implements the `mapFrom` method. Each step simulated in the `mapFrom` method of the `ArtistSpotify2Artist` class, shown in Figure 6.21, is generated by the following implementation of the `addMethodMapFrom` builder. For now, we are only mapping properties that have the same type as the constructor parameters. For each property in `props` we are iterating through `mapNotNull` (line 12), and whenever a property does not match a `destParam` in name or type we return null (line 17), thereby excluding it from the mapping process. Note that the Kotlin scope function `apply` is utilized on a `MethodMaker` (line 8), and within its lambda, the calls to `new_` and `return_` are made with the `MethodMaker` as the receiver, i.e. implicit `this` (line 19).

Figure 6.34. Dynamic builder of the `mapFrom` method.

```

1 private fun <T : Any, R : Any> ClassMaker.addMethodMapFrom(
2     src: KClass<T>,
3     dest: KClass<R>,
4     props: Map<KProperty<*>, KParameter>
5 ) {
6     addMethod(Any::class.java, "mapFrom", Any::class.java)
7     .public_()
8     .apply {
9         val from = param(0).cast(src.java) // var from = (ArtistSpotify) src

```

```

10
11     val propValues: List<Variable> = props
12     .mapNotNull { (prop, destParam) ->
13         if (prop.returnType.classifier != List::class
14             && prop.returnType.classifier == destParam.type.classifier
15         ) {
16             from.invoke(prop.javaGetter?.name) // var kind = from.getKind()
17         } else { null }
18     }
19     return_(new_(dest.java, *propValues.toTypedArray())) // return new Artist(..)
20 }
21 }

```

To support properties of complex types, such as other domain classes, we need to generate code capable of obtaining a Mapper for the corresponding type to transform the property's value. To achieve this goal, we adopt a method akin to the one utilized in [Figure 3.25](#) of [Section 3.7](#), maintaining all instantiated mappers within an internal Map. This strategy prevents redundant instantiation of mappers for identical source and destination classes. Consequently, when requiring a new dynamic mapper, it is accessible via the indexed access operator of the NaiveMapper object presented in [Figure 6.35](#).

Figure 6.35. Cache dynamic mapper instances in DynamicMapper object

```

object DynamicMapper {
    private val mappers = mutableMapOf<Pair<KClass<*>, KClass<*>>, Mapper<*, *>>()

    operator fun <T : Any, R : Any> get(srcType: Class<T>, destType: Class<R>) =
        get(srcType.kotlin, destType.kotlin)

    operator fun <T : Any, R : Any> get(srcType: KClass<T>, destType: KClass<R>) =
        mappers.getOrPut(srcType to destType) {
            buildDynamicMapper(srcType, destType)
                .finish()
                .kotlin
                .createInstance() as Mapper<*, *>
        } as Mapper<T, R>
}

```

Given that we are generating code using Cojen Maker to access the indexed operator, we have provided an overloaded version of this operator that accepts parameters of type `Class` instead of `KClass`. This decision was made because Cojen allows us to specify a literal value for a parameter of type `Class`, eliminating the need to retrieve that `Class` at runtime, thus minimizing overhead. As a result, rather than generating code to obtain the `KClass` at runtime, we generate bytecode using

instructions such as `ldc` to push a constant index from a constant pool. For instance, when mapping an `ArtistSpotify` to an `Artist` as detailed in [Figure 3.23](#), an auxiliary mapper is required to map `Song` to `Track`. In this scenario, bytecode similar to the following sample needs to be generated:

```
getstatic      #31 // Field DynamicMapper.INSTANCE:LDynamicMapper;
ldc            #58 // class Song
ldc            #60 // class Track
invokevirtual  #35 // DynamicMapper.get:(Ljava/lang/Class;Ljava/lang/Class;)LMapper;
```

When dealing with lists, we must determine the type of the list's elements to obtain the appropriate Mapper. Once we have that mapper, we can use the `mapFromList` method to transform the entire list.

We have now implemented the auxiliary function `convertPropToParam` according to [Figure 6.36](#), which can convert the value of a property from the source object to the value of a parameter in the destination constructor. This function will be invoked on line 17 of [Figure 6.34](#), replacing the previous `null` value.

Figure 6.36. Auxiliary function to map values of different types.

```
1 fun convertPropToParam(
2     mapFromBuilder: MethodMaker,
3     propVal: Variable,
4     prop: KProperty<*>,
5     destParam: KParameter)
6 : Variable {
7     val destKlass = (destParam.type.classifier as KClass<*>).java
8     return if(destKlass != List::class.java) {
9         mapFromBuilder
10            .`var`(DynamicMapper::class.java)
11            .field("INSTANCE")
12            .invoke("get", prop.javaGetter?.returnType, destKlass)
13            .invoke("mapFrom", propVal)
14            .cast(destKlass)
15     } else {
16         val propElemType = prop.returnType.arguments[0].type?.classifier as KClass<*>
17         val paramElemType = destParam.type.arguments[0].type?.classifier as KClass<*>
18         mapFromBuilder
19            .`var`(DynamicMapper::class.java)
20            .field("INSTANCE")
21            .invoke("get", propElemType.java, paramElemType.java)
22            .invoke("mapFromList", propVal.cast(List::class.java))
23            .cast(destKlass)
24     }
25 }
```

In [Figure 6.36](#), the `propVal` parameter (line 3) is a `Variable`, representing the property value retrieved from the source object. Initially, we validate whether the parameter class in the destination (i.e., `destClass`) is not a `List`. In such cases, we acquire a dynamic mapper for the pair property type and destination parameter class, denoted as `prop.javaGetter?.returnType` and `destClass` respectively (line 12). Conversely, if dealing with lists, we extract the argument type of the `List` to determine the class of its elements (lines 16 and 17). For additional insights into Reflection of generic types, please refer to [Section 3.7](#).

After obtaining the auxiliary dynamic mapper, we invoke either `mapFrom` or `mapFromList` based on whether we are mapping a single value or a list, respectively (lines 13 and 22). Subsequently, we conclude with a cast to the destination type. It's important to note that `mapFrom` and `mapFromList` return `Any` and `List<Any>` respectively due to **type erasure** on the JVM. While the cast is implicit at the source code level, it is essential at the bytecode level to obtain a reference of the desired type.

Chapter 7: Sequences, Generators and Suspending functions

Sequences, also known as **streams** in some programming languages, are a key feature for software development. In the Java ecosystem, there are many libraries available as alternatives to the standard Java API, including Vavr, JOOλ, StreamEx, Eclipse Collections, Guava, among others.

In common, these libraries provide the capability of building a **collection pipeline**, as defined in Martin Fowler's renowned article [Fowler, 2015]:

Collection pipelines are a programming pattern where you organize some computation as a sequence of operations which compose by taking a collection as output of one operation and feeding it into the next. (Common operations are filter, map, and reduce.)

DISCLAIMER: In this context, we are **excluding reactive sequences**, such as *reactive streams*, Kotlin Flow, and similar asynchronous APIs. Our focus is solely on synchronous and blocking access to sequences.

Most sequence libraries are constructed solely with programming language features. However, for the purpose of enhancing extensibility, many environments offer the **generators** feature and its **yield primitive**, which require some level of support from the managed runtime environment.

This idea was first introduced in the CLU programming language in 1975 and was key to expression evaluation in the Icon programming language in 1977. Its recent popularity may be attributed to its use in C# 2.0 and later in Ruby 1.9. In CLU and C#, **generators** are known as **iterators**, and in Ruby, they are called **enumerators**. Python, PHP, JavaScript, Scala, Dart, and Kotlin provide variants of the yield operator.

In the Kotlin programming language, `yield` is implemented through **suspending functions**. Suspending functions have a broader scope beyond generators, particularly in concurrent programming and providing coroutine support at the language level. Coroutines offer a structured approach to managing concurrent tasks within a specific scope and are a fundamental aspect of **structured concurrency**. There is extensive literature in the field of concurrent programming exploring the intricate details of structured concurrency, and we have chosen to omit this topic from the scope of the book. Instead, we will focus on utilizing only a small fraction of the capabilities of **suspend functions** to demonstrate how they enable the implementation of the `yield` functionality.

7.1: Sequence Alternatives

Sequence pipelines let programmers compose transformations over data, where the result of **each** computation serves as the input for the next transformation in the pipeline. One of the

advantages of this programming idiom is its **readability**, as each operation in the pipeline clearly expresses its specific step and purpose.

For example, given a variable `pastWeather` that refers to a list of `Weather` objects (i.e., `List<Weather>`), each containing daily weather information, we can filter for sunny days, create a sequence of temperatures, and finally select the first five elements. This is achieved through a chain of *filter*, *map*, and *take* operations, resulting in the pipeline shown in [Figure 7.1](#) in Kotlin. In this case, we need to provide certain operations (i.e., *filter* and *map*) with functions that specify what we want to do with each `Weather` object (i.e., `Weather::isSunny` and `Weather::celsius`).

Figure 7.1. Example of sequence pipeline to take 5 temperatures in sunny days.

```
val top5temps = pastWeather
    .filter(Weather::isSunny)
    .map(Weather::celsius)
    .take(5)
```

On the other hand, the example in [Figure 7.2](#) demonstrates an alternative implementation that avoids using auxiliary functions like *filter*, *map*, or *take*. Instead, it leverages the control flow constructs provided by the host language. This imperative approach manually processes the items from the data source, checking their properties (i.e., `isSunny` and `celsius`) to validate and transform the items into a new sequence.

Figure 7.2. Imperative approach to take 5 temperatures in sunny days.

```
val top5temps = mutableListOf<Int>()
for (w in pastWeather) {
    if (w.isSunny) { // ~ filter
        top5temps.add(w.celsius) // ~ map
        if (top5temps.size >= 5) { // ~ take
            break
        }
    }
}
```

Ignoring performance or efficiency issues for now and focusing solely on readability, in [Figure 7.2](#), we need to mentally parse the control flow to understand what is happening. In contrast, [Figure 7.1](#) is not only less verbose but also clearly conveys the purpose of each line, with the name of each operation corresponding directly to the action performed on the data.

Despite having similar goals, sequence APIs may vary in different characteristics depending on the technological environment, such as:

- Operation names.

- Composability: *method chaining* versus *nested functions*.
- *Eager* versus *lazy* evaluation.
- Access approach: *pull* versus *push*.
- Extensibility.

7.1.1: Operation names

Although **operation names** like *filter* and *map* are fairly consistent for sequence operations, different environments may use varying terminology. For example, in C#, operations similar to *filter* and *map* are called *Where* and *Select*, respectively, while *flatMap* is known as *SelectMany* in C# and *expand* in Dart. The equivalent example to [Figure 7.1](#) in C# is shown in [Figure 7.3](#). So, maybe in other technological environments, you will need to look up the correct name for the operation you are seeking.

Figure 7.3. Example of sequence pipeline in C#.

```
var top5temps = pastWeather
    .Where(weather => weather.IsSunny)
    .Select(weather => weather.Celsius)
    .Take(5)
```

7.1.2: Composability: *method chaining* versus *nested functions*

The technique of composing a pipeline, as shown in Listings [7.1](#) and [7.3](#), is known as **method chaining**. In this approach, the **receiver object** (the object on which the method is called) is implicitly passed as an argument to each method call, allowing subsequent methods to be invoked on the result of the previous method. While this idiom may appear to be the most natural way to chain operations into a pipeline, it might not be intuitive for all developers, particularly those not familiar with object-oriented programming.

For example, Scheme or Clojure developers might prefer using the **nested function** idiom, where functions are combined by making function calls the arguments of higher-level function calls. In this approach, the sequence of nested functions is evaluated from the innermost function outward, meaning that arguments are evaluated before the function is called. Consequently, operations in a pipeline need to be composed in the reverse order of their execution. However, this is typically not an issue for functional programmers, who are accustomed to writing such pipelines in functional programming languages like Clojure, as demonstrated in [7.4](#).

Figure 7.4. Example of sequence pipeline in Clojure.

```
(take 5
  (map :celsius
    (filter :isSunny pastWeather)))
```

7.1.3: Eager versus lazy evaluation

Another design difference between the technologies and idioms presented in the previous listings relates to **evaluation time**. The operations used in the approach of Figure 7.3 (e.g., Where, Select, and Take) and in Figure 7.4 (e.g., map, filter, and take) are characterized by **lazy evaluation**. This means they do not process the source elements (e.g., pastWeather) immediately when called. Instead, calling these methods merely adds another step to the sequence pipeline. It's important to note that sequences are **immutable**, so each query method returns a new sequence that results from composing the previous sequence with an additional operation.

This contrasts with **collections**, which are **in-memory data structures** that store all values they contain. In the imperative approach shown in Figure 7.2, we are **eagerly** instantiating the resulting list top5temps, which holds the outcome of the eager pipeline processing. Although it may not be immediately obvious, the pipeline in Figure 7.1 also processes eagerly. Each Kotlin function in Figure 7.1 generates a new list containing the intermediate elements resulting from each operation, as depicted in Figure 7.5.

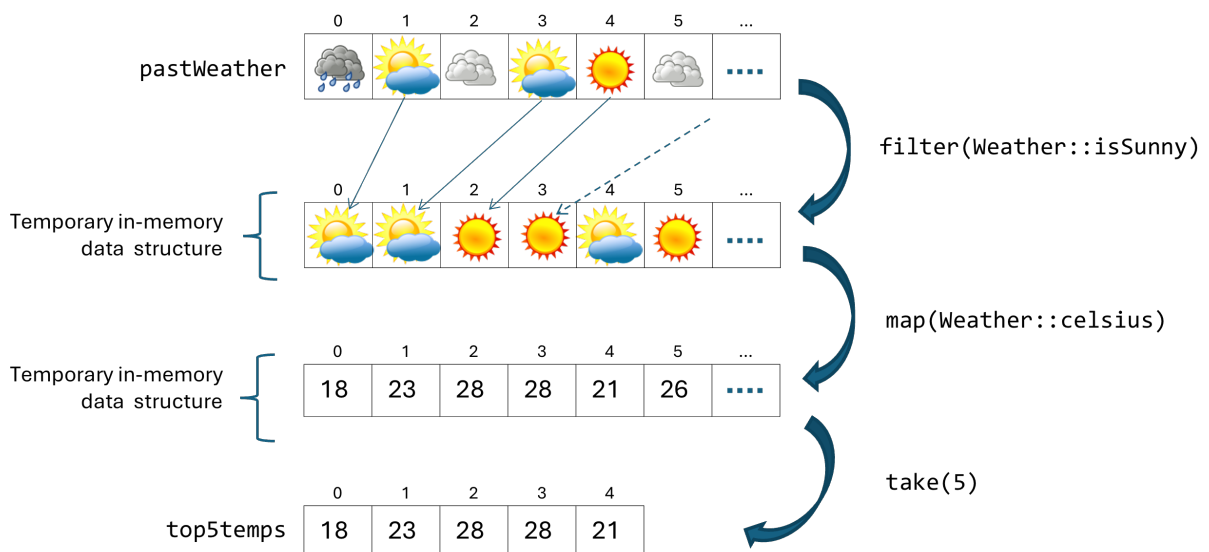


Figure 7.5. Eager evaluation of pastWeather data items.

Examining the implementation of such operations in Kotlin, we find extension functions for the `Iterable<T>` type, similar to those shown in [Figure 7.6](#), albeit with some simplifications. All these functions begin by creating a destination list to store the elements resulting from the computation. In the example of selecting the first 5 temperatures on sunny days, the `filter` and `map` operations generate intermediate results in the pipeline that are eventually discarded and later reclaimed by garbage collection.

Figure 7.6. Sample of Kotlin standard library extensions for `Iterable`.

```
public inline fun <T> Iterable<T>.filter(predicate: (T) -> Boolean): List<T> {
    val destination = ArrayList<T>()
    for (element in this)
        if (predicate(element))
            destination.add(element)
    return destination
}

public inline fun <T, R> Iterable<T>.map(transform: (T) -> R): List<R> {
    val destination = ArrayList<R>(collectionSizeOrDefault(10))
    for (item in this)
        destination.add(transform(item))
    return destination
}

public inline fun <T> Iterable<T>.take(n: Int): List<T> {
    val destination = ArrayList<T>(n)
    var count = 0
    for (item in this) {
        destination.add(item)
        if (++count == n)
            break
    }
    return destination.optimizeReadOnlyList()
}
```

Note that all operations in [Figure 7.6](#) are extension functions of the `Iterable<T>` interface, which is the base type for all collections in Kotlin and on the JVM. The `Iterable<T>` interface provides the minimal functionality necessary for traversing and accessing elements, which is required for implementing these operations. This design allows the use of these operations with any collection type, as all collections either directly or indirectly inherit from `Iterable<T>`.

In Kotlin, to differentiate between *eager* and *lazy* processing, there is the `Sequence<T>` interface, which has similar behavior to `Iterable<T>`, but with different semantics. The documentation for `Sequence<T>` states:

The values are evaluated lazily, and the sequence is potentially infinite.

To take advantage of lazy processing in Kotlin, we can convert any collection into a sequence using the utility function `asSequence()`. From this point, all sequence operations will use lazy evaluation. These operations have the same names as the `Iterable` extensions but with different implementations. Instead of creating an auxiliary destination list, these lazy operations return an instance of a new implementation of `Sequence<T>`. When sequence operations are invoked, the only computation performed is the instantiation of a new object that encapsulates the operation's logic. In [Figure 7.7](#), we present an example of such an implementation of the `map` operation, taken from the Kotlin standard library for JVM. This function instantiates the `TransformingSequence` class, which implements the `Sequence` interface by providing an `iterator()`. The actual instantiation of an anonymous `Iterator` happens only when the `iterator()` method is called. This `Iterator` then enables the traversal and transformation of each element from the original sequence.

Figure 7.7. Lazy implementation of `map` in Kotlin.

```
public fun <T, R> Sequence<T>.map(transform: (T) -> R): Sequence<R> {
    return TransformingSequence(this, transform)
}

internal class TransformingSequence<T, R>
constructor(
    private val sequence: Sequence<T>,
    private val transformer: (T) -> R
): Sequence<R> {
    override fun iterator(): Iterator<R> = object : Iterator<R> {
        val iterator = sequence.iterator()

        override fun next() = transformer(iterator.next())

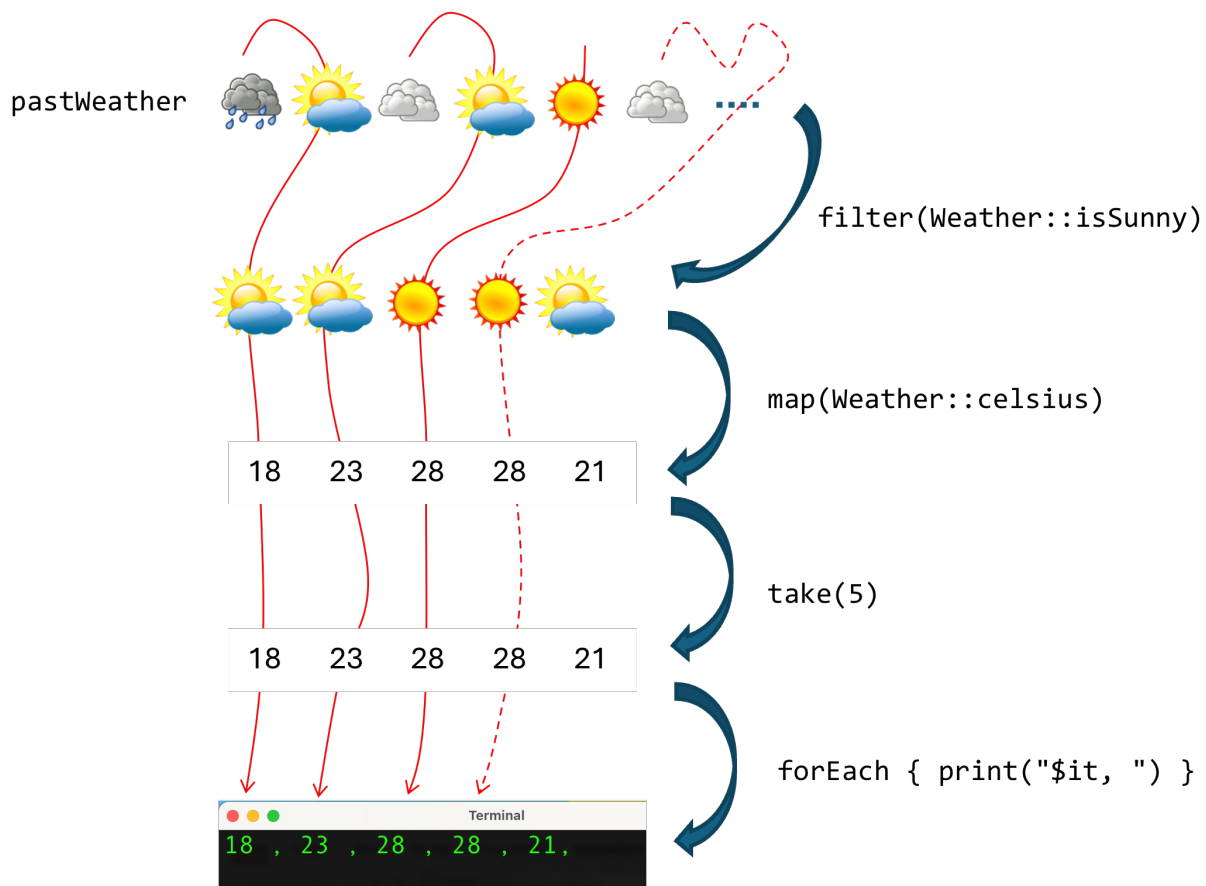
        override fun hasNext() = iterator.hasNext()
    }
}
```

If we interleave `asSequence()` between `pastWeather` and the first operation of the pipeline, i.e., `filter`, we obtain a lazy pipeline where nothing is produced until a *terminal operation* is chained. In contrast to *intermediate operations* like `filter`, `map`, or `take`, which produce a new sequence, a *terminal operation* either returns void (i.e. Kotlin `unit`) or something different from a sequence, preventing further chaining of operations in the pipeline. In this case, the elements of the resulting sequence (i.e., 18, 23, 28, 28, and 21) are computed on demand lazily, only when the sequence is iterated, for example, by a `forEach` operation, as depicted in [Figure 7.8](#).

Figure 7.8. Example of lazy pipeline to take 5 temperatures in sunny days.

```
pastWeather
    .asSequence()
    .filter(Weather::isSunny)
    .map(Weather::celsius)
    .take(5)
    .forEach { print("$it, ") }
```

The processing resulting from the *lazy evaluation* of a sequence pipeline is called **vertical processing** in some contexts, in contrast to **horizontal processing** performed during *eager evaluation*. When a pipeline is processed *eagerly*, **all the items of a sequence are traversed by each operation**. Conversely, with *lazy* evaluation, **each item traverses all operations of the pipeline**. If data items are laid out horizontally within a sequence, as depicted in [Figure 7.5](#), then processing all items *eagerly* means that each operation traverses all items horizontally, hence the term **horizontal processing**. On the other hand, when the operations are arranged vertically, as shown in [Figure 7.9](#), and each item in a *lazy* evaluation traverses all operations, we refer to it as **vertical processing**.

Figure 7.9. Lazy evaluation of `pastWeather` data items.

There is no universal rule that dictates lazy evaluation is always more efficient than an eager approach, or vice versa. However, for certain use cases, such as the example in this section, the lazy version may perform fewer data accesses and computations than the eager version. Consider `pastWeather` has 31 items, representing the 31 days of May, with 9 sunny days. In the eager approach, the `filter` operation calls the predicate 31 times, and the `map` operation performs 9 transformations, totaling 40 data accesses. If the 5th sunny day occurs at the 12th position in `pastWeather` (index 11), a lazy evaluation would perform only 12 data accesses for `filter` and 5 transformations for `map`, as only the 5 items requested by the `take` operation are processed, totaling 17 data accesses.

In addition, we must also consider the overhead and waste of creating intermediate collections through eager evaluation that are discarded at the end of the pipeline processing. This issue was highlighted by Roman Elizarov, the former project lead for Kotlin, who proposed an “*inspection to convert non-lazy chains of collection functions into sequences*”¹. He claimed that:

Code like this will conceptually work faster and produce less garbage (thus improving overall performance) when replaced with a lazy version using `.asSequence()`.

¹<https://youtrack.jetbrains.com/issue/KTIJ-6827/Inspection-to-convert-non-lazy-chains-of-collection-functions-into-sequences>

In the JVM standard library, Kotlin Sequence is not the only option for lazy evaluation of sequences. Before Kotlin, the Java standard library already provided a sequences API in the package `java.util.stream` through its core interface `Stream`. Therefore, it is possible to get a lazy stream over a collection and perform a pipeline, as depicted in [Figure 7.10](#), to get the first 5 temperatures on sunny days.

Figure 7.10. Example of Java stream pipeline to take 5 temperatures in sunny days.

```
var count = 0
pastWeather
    .stream()
    .filter { count++; it.isSunny }
    .map { count++; it.celsius }
    .limit(5)
    .forEach { print("$it, ") }
assertEquals(17, count)
```

For the example where the 5th sunny day occurs at the 12th position in the `pastWeather` stream, the count of filtering and transformations in the streams pipeline will match the count observed for the Kotlin sequences version. This can be illustrated by using a count variable incremented within each lambda, as shown in [Figure 7.10](#).

7.1.4: Extensibility

Implementing new custom operations for a sequences API requires adherence to the interface of its internal *traversal* mechanism. Each sequence technology has an interface that specifies how elements are **traversed** and **accessed**. Despite slight differences, many iterator protocols, such as `Iterator` in JVM, `Enumerator` in C#, or `Iterator` in Dart, provide at least one method to *advance* to the next element and one property to *access* the current element. In Java's standard library `Iterator`:

- The `next()` method serves both roles of advancing and accessing the next element.
- The `hasNext()` method indicates whether there are more items to iterate over.

`Enumerator` in C# has a similar API to `Iterator` in Dart, providing:

- A `moveNext()` method to advance, which returns `true` or `false` depending on whether it moves successfully or not.
- A `current` property that returns the current element.

With the exception of Java, many programming languages such as Kotlin, C#, Dart, and others, offer extension methods that allow developers to add new functionalities to existing interfaces like `Iterator`. We will demonstrate how to implement a new `interleave` method that intersperses

elements from two sequences. By chaining this `interleave` method in the pipeline, we can add a label like `($position)` next to each temperature, producing results such as 18 (1), 23 (2), 28 (3), 28 (4), 21 (5). Although similar outcomes can be achieved through various methods and operations, our focus here is to illustrate the use of a new custom `interleave` method, as shown in [Figure 7.11](#).

Figure 7.11. Example of Kotlin sequence pipeline with custom `interleave`.

```
pastWeather
    .asSequence()
    .filter(Weather::isSunny)
    .map(Weather::celsius)
    .interleave(generateSequence(1) { it + 1 }.map { " ($it) " })
    .take(10)
    .forEach { print(it) }
```

In the pipeline of [Figure 7.11](#), the `interleave` function is used to combine two sequences: one derived from `pastWeather` and another generated on-the-fly. It interleaves the sequence of temperatures with a sequence of positional labels generated by `generateSequence`, where each label is formatted as `" ($it) "`. The `interleave` function alternates elements from both sequences, pairing each temperature with its corresponding positional label.

The `interleave` function for Kotlin Sequence can be created by providing a custom implementation of the `Iterator` interface, as illustrated in [Figure 7.12](#). Later in [Section 7.2](#), we present a more compact implementation using generators.

Figure 7.12. Implementation of Kotlin extension function `interleave`.

```
fun <T> Sequence<T>.interleave(other: Sequence<T>) : Sequence<T> {
    return InterleavingSequence(this, other)
}

class InterleavingSequence<T>(self: Sequence<T>, other: Sequence<T>) : Sequence<T> {
    var selfIter = self.iterator()
    var otherIter = other.iterator()
    override fun iterator(): Iterator<T> {
        return object : Iterator<T> {
            override fun hasNext() = selfIter.hasNext()

            override fun next() = selfIter
                .next()
                .also { // Swap iterators so we pull from the other one next time.
                    selfIter = otherIter.also { otherIter = selfIter }
                }
        }
    }
}
```

```

    }
}

```

The `InterleavingSequence` class of [Figure 7.12](#) implements the `Sequence` interface and manages two iterators, `selfIter` and `otherIter`, for the respective sequences. The `iterator()` method of `InterleavingSequence` returns a custom iterator that alternates between the two sequences. It checks if there are more elements in the current iterator. After yielding an element, the `next()` method swaps the iterators to pull the next element from the other sequence, thus achieving the interleaving effect.

7.1.5: Access approach: *pull* versus *push*

The iteration protocol described in the previous section follows a **pull-based** approach, where elements are accessed from the sequence and then processed. Developers can check if the sequence has more elements by calling the `hasNext` method, and they can obtain the next element using the `next` method. Once an element is accessed, it can be processed according to the developer's code. The `Enumerator` in C# and the `Iterator` in Dart both use a similar *pull-based* approach.

Java streams use a different access approach known as **push-based**. Instead of pulling items from the sequence (i.e., requesting elements), the developer specifies what to do with the items (i.e., providing instructions). In a push-based approach, the equivalent method to `moveNext()`, such as `tryAdvance()`, takes a function that defines what to do with the next element, rather than returning the element itself. The `tryAdvance()` method returns a boolean: `false` if there are no remaining elements, and `true` if there are more elements to process.

For instance, if `pastWeather` is a Java Stream, we can iterate over its elements using a *push-based* approach, as demonstrated in [Figure 7.13](#). In this example, `tryAdvance` takes a lambda that prints the given item. The `while` block remains empty as there is no additional logic required within it.

Figure 7.13. Traversing items through a `Splitter` using a push-based approach.

```

val iter = pastWeather.splitter()
while (iter.tryAdvance { w ->
    println(w)
}) {
    /* while block */
}

```

The `tryAdvance` method is part of the `Splitter` interface in Java streams. Although `Splitter` provides additional functionality for parallel processing, we are focusing only on sequential processing here. With that in mind, we can implement a custom interleave operation for Java streams through the explicit implementation of the `AbstractSplitter` interface, as shown in [Figure 7.14](#).

Figure 7.14. Implementation of Kotlin extension function `interleave` for Java streams.

```

fun <T> Stream<T>.interleave(other: Stream<T>) : Stream<T> {
    val res = object : AbstractSpliterator<T>(Long.MAX_VALUE, ORDERED) {
        var selfIter = this@interleave.spliterator()
        var otherIter = other.spliterator()

        override fun tryAdvance(cons: Consumer<in T>): Boolean {
            return selfIter
                .tryAdvance(cons)
                .also { // Swap iterators so we pull from the other one next time.
                    selfIter = otherIter.also { otherIter = selfIter }
                }
        }
    }
    return StreamSupport.stream(res, false)
}

```

The logic behind the implementation in Figure 7.14 is quite similar to the Figure 7.12. The `spliterator` alternates between two internal spliterators, `selfIter` and `otherIter`, to push elements from the respective streams to given consumer `cons`. It does so by creating an anonymous `AbstractSpliterator` with an effectively unlimited size (`Long.MAX_VALUE`) and ordered characteristics. The `tryAdvance` method attempts to advance the current iterator and, after yielding an element, swaps the iterators to alternate the sequence of elements. Finally, the resulting interleaved elements are provided through the `StreamSupport.stream` method, returning a new stream that reflects the interleaving behavior.

Note that a similar implementation for Java streams can be achieved by implementing a **pull-based** iterator such as `Iterator<T>`. However, for Java streams extensibility, it is recommended to extend `Spliterator` rather than `Iterator`. As Brian Goetz (Architect for the Java Language at Oracle Corporation) stated in response to the Stack Overflow question [Iterator versus Stream of Java 8](https://stackoverflow.com/a/31212695/1140754)²:

Spliterator has fundamentally lower per-element access costs than Iterator, even sequentially.

7.2: Generators and yield pattern

Simply put, a **generator** is like a function that generates a sequence of values. However, instead of building the entire sequence at once (like an array or list), a **generator yields values one at a time**, returning a “new” value each time it is iterated over or called.

A **generator** refers to a computation that:

²<https://stackoverflow.com/a/31212695/1140754>

1. **yields values** to the caller.
2. **is resumed** after the yielded value has been consumed by the caller.

When a generator yields, it pauses its execution and does not need to be resumed by its caller, even though it may have pending computations to perform.

In summary a generator acts like a subroutine encompassing a special computation, that is restricted to communicate with its caller through the **yield** primitive. During its computation, a generator can yield out many values.

This characteristic makes generators useful for simplifying the implementation of lazy iterators. As we have seen, extending sequences with new custom operations in a lazy manner is not as straightforward as implementing them in an eager way. The verbosity of the lazy map operation implementation in [Figure 7.7](#) is evident when compared to the eager counterpart in [Figure 7.6](#), which only required four lines of code. This is especially true in object-oriented languages where following the iterator design pattern substantially increases the complexity and verbosity of those implementations, often requiring the implementation of two interfaces: `Iterable/Sequence` and `Iterator`.

With generators, programming languages provide a way to implement operations in a concise manner similar to eager approaches, but without incurring their drawbacks and maintaining a lazy behavior. The compiler then translates that implementation into a lazy form by implementing the required traversal interfaces.

In Kotlin, generators and the `yield` primitive are implemented through a lambda with a receiver of type `SequenceScope`. To avoid delving into these internal details immediately, we will first introduce the concept of generators using JavaScript. As a widely understood language, JavaScript uses native keywords for generators, making it easier to explain the concept without getting into the complexities behind the scenes.

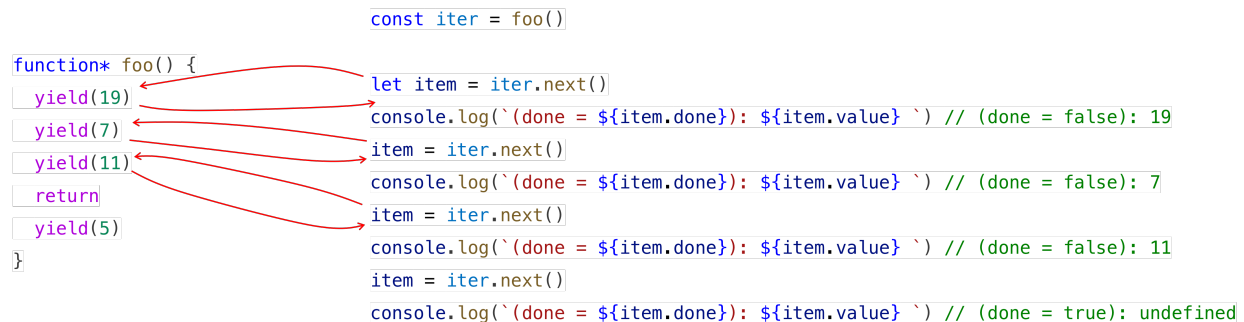
In JavaScript, a generator function is defined with `function*` rather than just `function`. The asterisk signifies that the function can yield multiple values, as opposed to returning a single value. Within the generator, using `return` terminates the sequence, while `yield` produces an item in the sequence.

In [Figure 7.15](#), the generator `foo` creates an iterator representing a sequence of three items: 19, 7, and 11. Note that the last yielded item, 5, is not included in the sequence because it occurs after the generator body has completed and returned.

Figure 7.15. JavaScript generator `foo` yielding numbers.

```
function* foo() {  
  yield 19  
  yield 7  
  yield 11  
  return  
  yield 5  
}
```

In [Figure 7.16](#), we illustrate an example using the generator `foo`. When `foo()` is called, none of its body's statements are executed immediately. Instead, it simply creates an instance of an iterator that encapsulates those statements. Execution of the statements begins only when we start advancing through the iterator by calling `next()`, at which point the generator's body will execute until it reaches a `yield` instruction. On subsequent call to `next()`, the generator resumes execution from the point where it was last suspended. In JavaScript, the `next()` method returns an object with two properties: `value`, which holds the current item, and `done`, which is `true` if the iterator has reached the end of the sequence.



```
const iter = foo()

function* foo() {
  yield(19)
  yield(7)
  yield(11)
  return
  yield(5)
}

let item = iter.next()
console.log(`(done = ${item.done}): ${item.value} `) // (done = false): 19
item = iter.next()
console.log(`(done = ${item.done}): ${item.value} `) // (done = false): 7
item = iter.next()
console.log(`(done = ${item.done}): ${item.value} `) // (done = false): 11
item = iter.next()
console.log(`(done = ${item.done}): ${item.value} `) // (done = true): undefined
```

Figure 7.16. Example of Javascript generator.

In Kotlin, a generator is created using the `sequence` function, which takes a lambda defining the generator's body. The Kotlin version of the `foo` generator of [Figure 7.17](#) returns a `Sequence` produced by calling `sequence`.

Figure 7.17. Kotlin generator `foo` yielding numbers.

```
fun foo() : Sequence<Int> {
    return sequence {
        yield(19)
        yield(7)
        yield(11)
        return@sequence
        yield(5)
    }
}
```

The `sequence` function receives a lambda where items are emitted using the `yield` function. Unlike JavaScript, `yield` in Kotlin is not a keyword but a function. It is a method of the `SequenceScope` instance, which is passed as the receiver of the lambda argument to `sequence`, as depicted in its signature:

```
fun <T> sequence(block: suspend SequenceScope<T>().() -> Unit): Sequence<T>
```

Just like in JavaScript, the behavior of `yield` is similar in Kotlin. When we call `foo`, it only creates an instance of a `Sequence` that encapsulates the function provided in the lambda; none of its instructions are executed at that time. Instead, execution starts only when we advance through the iterator by calling the `next()` method. The function runs until it reaches a `yield` statement, which pauses execution and returns a value. This process alternates between calls to `next()` from the caller and `yield` from the generator. In the following sections, we will explain how `yield` in Kotlin suspends execution and how it resumes.

Thus, the generator and `yield` functionality simplify the implementation of lazy iterators by abstracting away the details of interface implementations. This approach consolidates the iterator's logic into a single method, rather than requiring separate methods like `next()` and `hasNext()`. In [Figure 7.18](#), we present the equivalent lazy `interleave` implementation using Kotlin's `sequence` utility.

Figure 7.18. Kotlin `interleave` implementation with `sequence` and `yield`.

```
fun <T> Sequence<T>.interleave(other: Sequence<T>) : Sequence<T> {
    return sequence {
        var selfIter = this@interleave.iterator()
        var otherIter = other.iterator()
        while(selfIter.hasNext()) {
            yield(selfIter.next())
            selfIter = otherIter
            .also { otherIter = selfIter }
        }
    }
}
```

7.3: Suspend functions

A suspend function differs from a “normal” function (i.e., non-suspend function) by potentially having zero or more **suspension points**. Whenever a suspend function calls another suspend function, it creates a **suspension point** — statements in its body that may **pause** the function's execution to be **resumed** at a later moment in time.

In the JVM, a **suspend function** is translated by the Kotlin compiler to a method with an **additional parameter of type `Continuation<T>`**. This continuation parameter serves two purposes:

1. It receives the result of the suspend function being invoked, conforming to the **continuation-passing style**.
2. It manages the suspension implementation, effectively **transforming the suspension point into a continuation that can later resume the execution flow**.

CPS, or Continuation-Passing Style, contrasts with the **direct style**, where the **result of a function aligns with its returned value**. Conversely, in CPS, the **result is handed off to a continuation function**.

CPS initially gained prominence within the Scheme programming language community. This technique later found widespread adoption in JavaScript due to its suitability for handling asynchronous operations in the language's event-driven, single-threaded environment.

CPS may manifest in various forms depending on the implementation technology. In JavaScript, a continuation may often appear as a callback function, typically defined with the signature `(error, result) => void`, where `error` represents an error object if an error occurred during the operation. It is usually `null` or `undefined` if no error occurred. On the other hand, `result` contains the outcome or result of the asynchronous operation.

In Kotlin, the continuation is represented by the following interface:

```
interface Continuation<in T>
    abstract val context: CoroutineContext
    abstract fun resumeWith(result: Result<T>)
```

Ignoring the `context` property for the moment, the `resumeWith` function is equivalent to a callback, handling the return value of the invoked suspending function, whether it completes successfully or with an error. Instead of receiving two arguments, the `resumeWith` function has a single parameter of type `Result<T>`, which encapsulates either a successful outcome with a value of type `T` or a failure with a `Throwable` exception.

It is possible to explicitly cast a suspend function to its normal version counterpart, which includes an additional parameter `Continuation`. For example, consider the following suspending function with the signature: `suspend fun fetchSuspend(url: String): String` that performs an HTTP request for the given URL and returns the response body as a `String`. We call it `fetchSuspend` to distinguish it from a different `fetch` function that we will use later in this chapter. We can get a reference to that function of type `(String, Continuation<String>) -> Any` using the following statement:

```
val fetchHandle = ::fetchSuspend as (String, Continuation<String>) -> Any
```

Note that `Continuation` is parametrized with `String`, which is the type of the result of the `suspendFetch` function. The `Any` returned by the `fetchHandle` does not represent the result of the function but rather the execution state of that function. We will revisit this topic later. Given the `fetchHandle`, we can invoke it using the continuation-passing style. In the example of [Figure 7.19](#), we perform a fetch to `https://github.com` and print the response body.

Figure 7.19. Calling a suspend function using continuation-passing style.

```
val fetchHandle = ::fetchSuspend as (String, Continuation<String>) -> Any
fetchHandle("https://github.com", object : Continuation<String> {
    override val context = EmptyCoroutineContext
    override fun resumeWith(result: Result<String>) {
        println(result.getOrNull())
    }
})
```

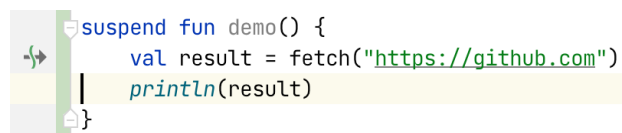
The coroutine context encompasses various elements, such as the coroutine's job and its dispatcher, which determine the thread or threads used for execution by the corresponding coroutine. Given that we are using these functions synchronously and not engaging in concurrent programming, we leave those details out of the scope of this book. Therefore, we initialize the context with an `EmptyCoroutineContext`.

The code snippet in [Figure 7.19](#) is equivalent to a portion of the work performed by the Kotlin compiler when we invoke a suspend function like `fetchSuspend`. This code could be the result of the translation of a suspending function call, such as the snippet of [Figure 7.20](#):

Figure 7.20. Calling a suspend function.

```
val result = fetchSuspend("https://github.com")
println(result)
```

When we call `fetch("https://github.com")`, the Kotlin compiler transforms it into the continuation-passing style similar to what we've shown with [Figure 7.19](#). The code snippet of [Figure 7.20](#) presents a direct style despite being translated to a continuation-passing style. Some IDEs, such as IntelliJ, mark suspension points with a distinctive icon, indicating that invocations may be paused and resumed later.



```
suspend fun demo() {
    val result = fetch("https://github.com")
    println(result)
}
```

Figure 7.21. IntelliJ highlighting suspension points.

We can also make the reverse transformation by casting a normal function that uses the continuation-passing style to a suspending function. For instance, consider the following definition of `fetchCps`:

```
fun fetchCps(url: String, onCompletion: Continuation<String>) { ... }
```

In this case, `fetchCps` is a function that takes a URL and a continuation as parameters, and later invokes that continuation with the response body of the HTTP request. If we cast this function to its suspending counterpart, we can then invoke it as a suspend function, such as the snippet of [Figure 7.22](#).

Figure 7.22. Calling CPS function as a suspend function

```
val fetchCpsHandle = ::fetchCps as (suspend (String) -> String)
val result = fetchCpsHandle("https://github.com")
println(result)
```

Since `fetchCpsHandle` is a suspend function, the call `fetchCpsHandle("https://github.com")` corresponds to a suspension point. Additionally, **note that this code is only valid within a suspend function**, as only suspend functions may call other suspend functions.

7.4: Deconstructing yield and generators

Generators and `yield` in Kotlin are implemented by means of **suspend functions**. The semantics of **suspending functions** and **suspension points** align closely with the concepts of **generators** and the **yield primitive**. In this analogy, a **generator** is akin to a suspending function, and a **call to yield** creates a **suspension point** that can be resumed later.

7.4.1: Translating a Suspend Function into a State Machine

The Kotlin compiler translates a **suspending function** into a **state machine**, where each state represents a **continuation**—i.e., the code to be executed after a suspension point.

To illustrate this translation process, we will manually implement a function with suspension points supported by the explicit implementation of a `Continuation<T>`. In our use case, we will implement a `fetchMany` function that receives a list of URLs and returns a list of strings (`List<String>`) containing the HTTP response body of each request for the given URLs.

We will incrementally evolve this example until we reach the final version, where we will fully utilize the existing suspend function mechanism. This development proceeds through the following phases:

1. `fetchMany`
2. `fetchManyCps`
3. `fetchManyCpsLazy`
4. `fetchManySuspendingLazy`

In the first phase, we implement `fetchMany` as a ‘normal’ (i.e., non-suspend) function, which will be transformed in the second phase into `fetchManyCps` to adhere to continuation-passing style. Additionally, `fetchManyCps` breaks the original algorithm into states, with each state fetching one of the given URLs. In the third phase, instead of returning a `List<String>`, the `fetchManyCpsLazy` function returns a `Sequence<String>`, which fetches each URL lazily on demand only when we advance the resulting iterator (i.e., call `next()`). This implementation might not fetch all provided URLs if the resulting iterator is not traversed to the end. Finally, the `fetchManySuspendingLazy` function implements the same behavior as `fetchManyCpsLazy` but with the support of suspending functions.

At some points in the intermediate stages, the code may seem more complex than necessary. However, it’s important to keep in mind that we’re essentially simulating the behavior of the compiler-generated code. Ultimately, in the final version, this complexity will be abstracted away by the use of suspending functions.

While this may seem limiting, instead of receiving a `vararg` parameter or any other form of list of strings, we will restrict the arguments to three URLs: `ur11`, `ur12`, and `ur13`. This approach allows us to more easily illustrate the state machine and reason about each state in the generated bytecode.

1. `fetchMany`

The `fetchMany` function utilizes an auxiliary `fetch` function responsible for performing the HTTP request and returning the response body. This `fetch` is a normal function (i.e. non-suspending). To observe the eager versus lazy behavior of each implementation, we include a print message in the `fetch` function. In [Figure 7.23](#), we provide the implementations of both the `fetchMany` and `fetch` functions.

Figure 7.23. `fetchMany` implementation in a “normal” function.

```
fun fetch(path: String): String {
    println("Fetching $path")
    return URI(path).toURL().readText()
}

fun fetchMany(ur11: String, ur12: String, ur13: String): List<String> {
    val bodies = mutableListOf<String>()
    bodies.add(fetch(ur11))
    bodies.add(fetch(ur12))
    bodies.add(fetch(ur13))
    return bodies
}
```

The `fetchMany` function begins by initializing an empty mutable list called `bodies` to store the response bodies. It then calls the `fetch` function for each URL (`ur11`, `ur12`, and `ur13`) and adds the

returned response body to the bodies list. Finally, it returns the bodies list containing the response bodies fetched from all three URLs.

2. fetchManyCps

At this stage, we will adapt the previous `fetch` and `fetchMany` functions to the continuation-passing style in two new implementations, named `fetchCps` and `fetchManyCps`. This new implementation with CPS behaves in the same manner as the former direct style and offers no advantages beyond increased complexity. Nevertheless, it serves as a necessary step to comprehend the suspension points generated by the invocation of suspending functions, such as `yield`.

First, we will introduce a new function `fetchCps` (Figure 7.24) adhering to the continuation-passing style. In comparison to the previous `fetch` function, `fetchCps` receives an additional parameter of type `Continuation<String>`. This continuation is responsible for handling the result of the `fetchCps` function. It will either resume with the response body upon successful completion of the HTTP request or with an exception in case of failure.

Figure 7.24. `fetchCps` adhering to the continuation-passing style.

```
fun fetchCps(path: String, onComplete: Continuation<String>) {
    println("Fetching $path")
    try {
        val body = URI(path).toURL().readText()
        onComplete.resume(body)
    } catch (err: Throwable) {
        onComplete.resumeWithException(err)
    }
}
```

Now, we will break down the `fetchMany` algorithm into a state machine within the implementation of a `Continuation<String?>`. In this setup, the `resumeWith` function will sequentially handle the results of the calls to `fetchCps`, following the definition outlined in Figure 7.25.

Figure 7.25. Function `resumeWith` in the implementation of the `FetchManyCps` class.

```
override fun resumeWith(result: Result<String?>) {  
    result.getOrThrow()?.let { bodies.add(it) }  
    when(state++) {  
        0 -> fetchCps(url1, this)  
        1 -> fetchCps(url2, this)  
        2 -> fetchCps(url3, this)  
        3 -> onComplete.resume(bodies)  
    }  
}
```

In this implementation, the list `bodies`, the `state`, and the `onComplete` are properties within the class `FetchManyCps`, which implements the `Continuation` interface. On each `fetchCps` call, we reuse the same `Continuation` object (`this`), ultimately resuming to the `resumeWith` function, but in a different state. The `fetchManyCps` function instantiates the `FetchManyCps` class and initiates the flow by invoking the `resumeWith` method with `null`, as depicted in [Figure 7.26](#).

Figure 7.26. `fetchManyCps` function starting the execution flow.

```
fun fetchManyCps(  
    url1: String,  
    url2: String,  
    url3: String,  
    onComplete: Continuation<List<String>>  
) {  
    FetchManyCps(url1, url2, url3, onComplete)  
        .resume(null)  
}
```

Beyond the implementation of `resumeWith` and the previously mentioned properties, the `FetchManyCps` class also overrides the `context` property of the `Continuation` interface, in accordance with the definition provided in [Figure 7.27](#).

Figure 7.27. FetchManyCps class

```
class FetchManyCps(
    private val url1: String,
    private val url2: String,
    private val url3: String,
    private val onComplete: Continuation<List<String>>
)
    : Continuation<String?>
{
    private val bodies = mutableListOf<String>()
    var state = 0

    override val context = EmptyCoroutineContext

    override fun resumeWith(result: Result<String?>) {...}
}
```

The implementation of `fetchManyCps` follows CPS but lacks the ability to be paused and resumed. When `fetchManyCps` is invoked, all HTTP requests are executed for the given URLs.

For example, when making the call depicted in the snippet from [Figure 7.28](#), it promptly prints the following output to the terminal:

```
Fetching https://stackoverflow.com/
Fetching https://github.com/
Fetching https://developer.mozilla.org/
```

Figure 7.28. Calling `fetchManyCps` with 3 urls

```
val res = fetchManyCps(
    "https://stackoverflow.com/",
    "https://github.com/",
    "https://developer.mozilla.org/")
```

3. `fetchManyCpsLazy`

To make it lazy, we will modify the function to return a `Sequence<String>` instead of a list. This sequence will progress through the state machine only when the `next()` function is invoked.

To achieve this, the `FetchManyCpsLazy` class will implement the `Iterator` interface. It will deliver each item whenever the `next()` function is called, rather than collecting all items into a mutable list

bodies that is later delivered to the `onComplete` continuation. Therefore, in the `FetchManyCpsLazy` class, the `bodies` and `onComplete` properties are replaced with the `nextItem` property. This property will hold the next element delivered by the `next()` method. In the code provided in [Figure 7.29](#), only the new parts that differ from the former implementation of the `FetchManyCps` class are included.

Figure 7.29. `FetchManyCpsLazy` implementing Continuation and Iterator.

```
class FetchManyCpsLazy(...) : Continuation<String?>, Iterator<String> {
    var state = 0
    var nextItem: String? = null
    ...
    override fun resumeWith(result: Result<String?>) {
        nextItem = result.getOrNull()
    }
    private fun block() {
        when(state++) {
            0 -> fetchCps(url1, this)
            1 -> fetchCps(url2, this)
            2 -> fetchCps(url3, this)
            else -> throw NoSuchElementException()
        }
    }

    override fun hasNext() = state <= 2

    override fun next(): String {
        block()
        return nextItem as String
    }
}
```

It's worth noting that the original state machine, initially presented in [Figure 7.25](#), is now encapsulated within the `block` function. Additionally, the `resumeWith` implementation of Continuation solely stores the next item. In essence, the `next()` function progresses through the state machine by invoking the `block()` function, which may pause during a `fetchCps` call and resume during the execution of the `resumeWith` function.

The new implementation of `fetchManyCpsLazy` simply needs to construct a sequence that returns a new instance of `FetchManyCpsLazy` when the iterator is called, as demonstrated in the following listing:

```
fun fetchManyCpsLazy(url1: String, url2: String, url3: String)
: Sequence<String> {
    return object : Sequence<String> {
        override fun iterator() = FetchManyCpsLazy(url1, url2, url3)
    }
}
```

Now, invoking the `fetchManyCpsLazy` function will not trigger any actions until we iterate over the resulting sequence. Running the snippet of Figure 7.30 will only print the messages "Fetching" after the "Call to `fetchManyCpsLazy` finished!" message appears in the terminal, as shown in the following output:

```
Call to fetchManyCpsLazy finished!
Fetching https://developer.mozilla.org/
Fetching https://stackoverflow.com/
```

Figure 7.30. Getting only 2 items of the Iterator from the `fetchManyCpsLazy`.

```
val seq = fetchManyCpsLazy(
    "https://stackoverflow.com/",
    "https://github.com/",
    "https://developer.mozilla.org/")
val iter = seq.iterator()
println("Call to fetchManyCpsLazy finished!")
iter.next()
iter.next()
```

Additionally, we are not obligated to fetch all given URLs unless we traverse the iterator to the end. In this scenario, if we do not consume the last item, we leave the iterator in a suspended state indefinitely.

4. `fetchManySuspendLazy`

At this stage, we are approaching the final implementation of the `fetchManySuspend` function, defined as follows:

```

fun fetchManySuspend(url1: String, url2: String, url3: String) : Sequence<String> {
    return sequence {
        val body1 = fetch(url1)
        yield(body1)
        val body2 = fetch(url2)
        yield(body2)
        val body3 = fetch(url3)
        yield(body3)
    }
}

```

The lambda passed to the sequence builder function will be translated by the Kotlin compiler into an anonymous function that implements a state machine similar to the one presented in [Figure 7.29](#) and can be denoted as:

```

private fun block() {
    when (state++) {
        0 -> fetchCps(url1, this)
        1 -> fetchCps(url2, this)
        2 -> fetchCps(url3, this)
        else -> throw NoSuchElementException()
    }
}

```

Thus, we will leverage Kotlin's native support for suspending functions and suspension points to automatically generate an equivalent state machine. In the new version of the `FetchManySuspendLazy` class, the `block` function will be a suspending function defined as depicted in [Figure 7.31](#):

Figure 7.31. `block` function in `FetchManySuspendLazy` class.

```

private suspend fun block() {
    val body1 = fetch(url1)
    yield(body1)
    val body2 = fetch(url2)
    yield(body2)
    val body3 = fetch(url3)
    yield(body3)
}

```

Note that `yield` is a suspending function too, and each call creates a suspension point. Additionally, we replaced the usage of `fetchCps` with a non-suspending call to the normal `fetch` function. Otherwise, we would have three more suspension points compared to the former state machine example.

In [Figure 7.32](#), we present the bytecode resulting from the compilation of the `block` function described in [Figure 7.31](#). Here, local variable 2 will store the result of the invocation of `fetch()`. Local variable 6 refers to an instance of the anonymous inner class `FetchManySuspendLazy$block$1`, which implements `Continuation`. This object serves as the continuation for the calls to the suspending function `yield`. Additionally, `FetchManySuspendLazy$block$1` contains an integer field `label`, equivalent to our state property of [Figure 7.27](#), which holds the current state in the state machine. It has also a field `this$0` that is a reference to an instance of the outer class `FetchManySuspendLazy`.

Figure 7.32. Bytecode resulting from compilation of suspending block function.

```
private final block(Lkotlin/coroutines/Continuation;)Ljava/lang/Object;
...
    /*
     * loc2: String
     * loc6: FetchManySuspendLazy$block$1
     */
    ALOAD 6
    GETFIELD pt/isel/FetchManySuspendLazy$block$1.label : I
    TABLESWITCH
        0: L4
        1: L5
        2: L6
        3: L7
        default: L8
    L4:
        /*
         * <=> loc2 = FetchKt.fetch(this.url1)
         */
        ALOAD 0
        GETFIELD pt/isel/FetchManySuspendLazy.url1 : Ljava/lang/String;
        INVOKESTATIC pt/isel/FetchKt.fetch (Ljava/lang/String;)Ljava/lang/String;
        ASTORE 2
        /*
         * <=> loc6.label = 1
         */
        ALOAD 6
        ICONST_1
        PUTFIELD pt/isel/FetchManySuspendLazy$block$1.label : I
        /*
         * <=> this.yield(loc2, loc6)
         */
        ALOAD 0
        ALOAD 2
        ALOAD 6
```

```
INVOKESPECIAL pt/isel/FetchManySuspendLazy.yield (
    Ljava/lang/String;Lkotlin/coroutines/Continuation;)Ljava/lang/Object;
```

The bytecode presented in [Figure 7.32](#) offers a simplified version, focusing on the core instructions. It begins by evaluating the `label` field using `TABLESWITCH` and jumps to one of the corresponding states. Label `L7` represents the final state reached after the last `yield`, terminating the execution of the suspending function `block`. Label `L8` indicates an illegal state that should never occur, except in exceptional cases.

In [Figure 7.32](#), we present bytecode for label `L4`. However, this can be extended to cover other labels such as `L5` and `L6` with minimal changes. These changes include updating the `GETFIELD` instruction to access `ur12` and `ur13`, as well as updating the `label` field to the values 2 and 3 respectively.

Each state begins by executing the code corresponding to the current step, which involves fetching the given URL and storing the response body in local variable 2. Subsequently, it updates the state variable (the `label` field) and concludes with a call to the suspending function `yield`. The continuation, in this case, is represented by local variable 6 of type `FetchManySuspendLazy$block$1`, which is passed as the last argument in the `yield` call.

Looking now into `FetchManySuspendLazy$block$1` of [Figure 7.33](#) we can observe this class extends `ContinuationImpl` that redirects the `resumeWith` call to the function `invokeSuspend`. Despite other internal details, we are only highlighting that in this case the `invokeSuspend` will call again the `block` function when this continuation is resumed, but entering in a new state. The call to `block` receives this same instance of `FetchManySuspendLazy$block$1` as the continuation, which is equivalent to call `this$0.block(this)`.

Figure 7.33. Bytecode of continuation passed to `yield` that resumes to the outer function.

```
final class pt/isel/FetchManySuspendLazy$block$1
    extends kotlin/coroutines/jvm/internal/ContinuationImpl
{
    /*
     * field `label` holding the current state in the state machine.
     * field `this$0` that is a reference to the outer object.
     */
    I label
    final synthetic Lpt/isel/FetchManySuspendLazy; this$0
    ...
    public final invokeSuspend(Ljava/lang/Object;)Ljava/lang/Object;
    ...
    /*
     * <=> this$0.block(this)
     */
    ALOAD 0
    GETFIELD pt/isel/FetchManySuspendLazy$block$1.this$0
```

```

ALOAD 0
INVOKESTATIC pt/isel/FetchManySuspendLazy.access$block (
    Lpt/isel/FetchManySuspendLazy;Lkotlin/coroutines/Continuation;)

```

Now, let's create our custom implementation of the `yield` function, which will control the progress in the state machine. The `yield` function only needs to store the `nextItem` and update the `nextStep` property with a reference to the next continuation, which is responsible for resuming the execution when invoked. To access this continuation, we will deconstruct the definition of the suspending function `yield` using the same technique presented in [Section 7.3](#). The implementation of `yield` presented in [Figure 7.34](#) is part of the `FetchManySuspendLazy` class.

Figure 7.34. Custom implementation of an `yield` suspending function.

```

private suspend fun yield(item: String) = yieldHande(item)

private val yieldHande = ::yieldCps as (suspend (String) -> Unit)

private fun yieldCps(item: String, onComplete: Continuation<Unit>) : Any {
    nextItem = item
    nextStep = onComplete
    return COROUTINE_SUSPENDED
}

```

The `COROUTINE_SUSPENDED` value returned by `fetchCps` indicates that the suspending function has suspended execution and will not return any result immediately.

The remaining implementation of the class `FetchManySuspendLazy` is presented in [Figure 7.35](#). This class includes an auxiliary function `tryAdvance` that advances by either invoking the `nextStep` continuation (`nextStep?.resume(Unit)`) or starting the execution by invoking the `block` function (`blockHandle(this)`). Since `block` is a suspending function, we invoke it with a continuation to the same instance (i.e., `this`), where the `resumeWith` function will store the end result in the property `finish`. This property allows us to determine when the `block` function has terminated its execution.

Figure 7.35. `FetchManySuspendLazy` class

```

class FetchManySuspendLazy(...) : Continuation<Unit>, Iterator<String>
{
    var nextItem: String? = null
    var nextStep: Continuation<Unit>? = null
    var finish: Result<Unit>? = null
    ...
    private val blockHandle = ::block as ((Continuation<Unit>) -> Any)

    private fun tryAdvance() {

```

```

        nextStep
            ?.resume(Unit)
            ?:blockHandle(this)
    }
    override fun resumeWith(result: Result<Unit>) {
        finish = result
    }
    override fun hasNext() = finish == null

    override fun next(): String {
        tryAdvance()
        finish?.getOrThrow()
        if(finish != null) throw NoSuchElementException()
        return nextItem as String
    }
    ...

```

The behavior of `FetchManySuspendLazy` closely mirrors that of the `FetchManyCpsLazy` class. However, it relies on the continuations generated by the Kotlin compiler rather than explicitly implementing a state machine.

7.4.2: sequence builder

Now, let's build our custom `buildSequence` utility function, equivalent to the existing sequence builder in Kotlin. Our `buildSequence` function will create an instance of the `SequenceBuilderIterator` class, which is based on the previous implementation of `FetchManySuspendLazy`. However, the `SequenceBuilderIterator` class will receive the `block` function as a constructor parameter, allowing it to remain flexible and not tied to the specific implementation logic of the `block`.

However, clients of our library who implement this `block` function should have access to the internal `yield` function, which is crucial for retaining the continuation instances. To facilitate this, the `SequenceBuilderIterator` class provides a `Yieldable` object as the receiver of the `block` function, as denoted in [Figure 7.36](#). Additionally, the `SequenceBuilderIterator` class implements the `Yieldable` interface, allowing it to act as the receiver of the `block` function (`this.blockHandle(this)`) (line 23) and thereby receive the items yielded during the `block` function's execution.

Figure 7.36. SequenceBuilderIterator implementation with a block function parameter.

```

1  interface Yieldable<T> { suspend fun yield(item: T) }
2
3  enum class SequenceState { NotReady, Ready, Done, Failed}
4
5  private class SequenceBuilderIterator<T>(block: suspend Yieldable<T>.(()) -> Unit) :
6      Yieldable<T>,
7      Iterator<T>,
8      Continuation<Unit>
9  {
10     private var nextItem: T? = null
11     private var state = SequenceState.NotReady
12     private var finish: Result<Unit>? = null
13     private var nextStep: Continuation<Unit>? = null
14
15     override val context = EmptyCoroutineContext
16
17     private val blockHandle = block as (Yieldable<T>.(Continuation<Unit>) -> Any)
18
19     private fun tryAdvance() {
20         if(state == SequenceState.NotReady) {
21             nextStep
22                 ?.resume(Unit)
23                 ?: this.blockHandle(this)
24         }
25     }
26     override fun resumeWith(result: Result<Unit>) {
27         state =
28             if(result.isSuccess) {
29                 SequenceState.Done
30             } else {
31                 finish = result
32                 SequenceState.Failed
33             }
34     }
35     ...

```

There is also a further enhancement compared to the `FetchManyCpsLazy` regarding state control. The `hasNext()` function might need to advance the traversal depending on whether `next()` has been called before or not. Thus, we cannot simply check if the `finish` property is still null. We must improve it with a generic state machine that controls whether we have advanced or not. We follow a similar approach to the existing `SequenceBuilderIterator` of Kotlin in a simplified version,

including four states: `NotReady`, `Ready`, `Done`, and `Failed`. The iterator will proceed by transitioning between `NotReady` and `Ready`, and will reach a final state at either `Done` or `Failed`.

Our `SequenceBuilderIterator` implements the `yield` function in the same way as the `FetchManyCpsLazy` class by deconstructing the suspend function `yield` into a `yieldCps` function. However, now, whenever `yieldCps` is called, it should update the state to the `Ready` value.

Finally, in the listing of [Figure 7.37](#), we present the rest of the implementation of the `SequenceBuilderIterator`, including its `next()` and `hasNext()` methods.

Figure 7.37. `SequenceBuilderIterator` implementation of `next()` and `hasNext()`.

```
private class SequenceBuilderIterator<T>(...) : ...{
    ...
    override fun hasNext(): Boolean {
        tryAdvance()
        return state == SequenceState.Ready
    }

    override fun next(): T {
        tryAdvance()
        finish?.getOrThrow()
        if(state != SequenceState.Ready) throw NoSuchElementException()
        state = SequenceState.NotReady
        return nextItem as T
    }
    ...
}
```

Summary

This chapter begins by discussing the Collection Pipeline idiom, a term coined by Martin Fowler, and comparing its characteristics across different technological environments. It also details the various design options considered in the implementation of sequences, including naming, composability, evaluation timing, access methods, and extensibility.

Following that, we introduce generators and the `yield` primitive—one of the most widely adopted mechanisms for lazily implementing custom sequence operations, commonly found in most high-level programming languages, with the notable exception of Java.

Afterward, we delve into the internals of suspending functions in Kotlin, which form the foundation for supporting generators and `yield` functionality in this environment. We conclude by explaining how generators in Kotlin are translated into a state machine using suspending functions.

7.5: Exercises

Exercise 1

Consider the following Kotlin code:

```
arrayOf("abc", "isel", "super")
    .map { print("$it "); it.length }
    .filter { print("$it "); it == 4 }
    .first()
```

- What is the output of executing the given code?
 - If you used `sequenceOf` instead of `arrayOf`, would there be any difference? Justify your answer.
-

Exercise 2

Consider the following Kotlin code:

```
val res = arrayOf("abcdef", "super", "isel", "trio", "tri")
    .map { print("$it "); it.length }
    .filter { print("$it "); it % 2 == 0 }
    .distinct()
```

- What is printed in the output when this code is executed, and what is stored in the `res` variable?
 - What is printed in the output when this code is executed using `sequenceOf` instead of `arrayOf`? Justify your answer.
-

Exercise 3

Make two distinct implementations of the generic extension function `coalesce`: one for `Iterable` (eager) and another for `Sequence` (lazy). Both implementations should combine pairs of elements using a function passed as a parameter, producing a new sequence with half the number of elements.

For example, the expression:

```
sequenceOf(1, 2, 3, 4, 5, 6, 7, 8, 9).coalesce { a, b -> (a + b) / 2.0 }
```

produces the sequence 1.5, 3.5, 5.5, 7.5. If the sequence has an odd number of elements, the last element is not used. As in the example, the elements of the output sequence can be of a different type than those of the input sequence.

Exercise 4

The function `Sequence<String>.reversedWordsLongerThan(longSize: Int) : List<String>` returns a list of words longer than `longSize`, with their characters in reverse order. In the following example, the standard output will display: “lesi” and “repus”.

```
sequenceOf("isel", "abc", "super", "por")  
  .reversedWordsLongerThan(3)  
  .forEach(::println)
```

a. Given the two implementations below for the function `reversedWordsLongerThan`, indicate whether both produce the correct result. If they do, explain which one you would choose and why. If not, explain the difference in the results and identify the correct version.

- `return this.toList().map { it.reversed() }.filter { it.length > longSize }`
- `return this.toList().filter { it.length > longSize }.map { it.reversed() }`

b. In the previous examples, explain the difference between using `toList()` at the beginning or the end of the pipeline, or if it makes no difference.

Chapter 8: Garbage Collection and Cleanup Actions

Automatic memory management is one of the key features of the JVM and managed runtime environments in general. Despite the criticism it faced in the early days of the JVM, today it is almost unthinkable to envision a managed runtime without a garbage collector (GC). It not only prevents many developer bugs in software development, but it is also so optimized that its overhead is almost negligible in long-running applications, such as web applications.

That said, while there are many tuning configurations available for the GC, use them with caution. Most GC implementations are highly optimized, and the best approach is often to let them run freely.

This chapter provides insights into the core principles and internals of the GC. Tuning and configuring the GC is beyond its scope; for those topics, we refer you to official documentation, such as the [HotSpot Virtual Machine Garbage Collection Tuning Guide](https://docs.oracle.com/en/java/javase/17/gctuning)¹, or other relevant resources.

8.1: The Managed Heap

Reference types always produce **instances** that are allocated on the **heap**, whereas primitive types have their allocation determined by the context in which the variable is declared. For example, **local variables** are allocated on the **stack**, while **static fields** are allocated in the scope of their declaring class, and **instance fields** are allocated within the layout of the owning **object**, as described in [Section 2.1.3.1](#).

Reference type **variables** hold **object references** rather than instances of their declared types. A reference type variable merely contains the address of the object it points to.

One way of manually managing dynamic memory involves identifying **free memory blocks**, as illustrated in [Figure 8.1](#). When an object reference is released through a free primitive or an equivalent mechanism, the referenced memory is added to the list of free memory blocks.

¹<https://docs.oracle.com/en/java/javase/17/gctuning>

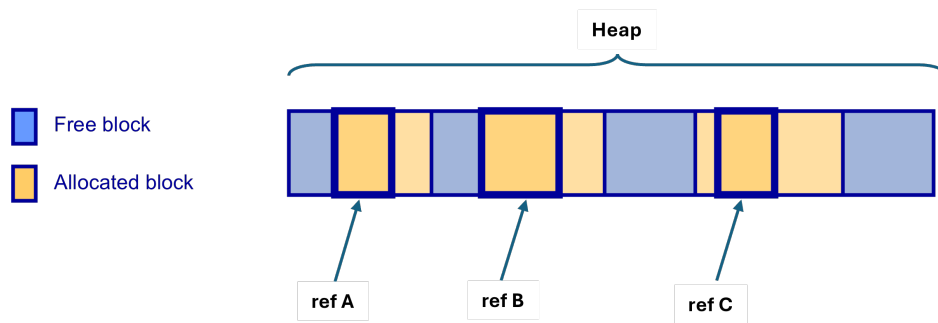


Figure 8.1. Manual management of heap.

However, this approach has several issues, some of which are highlighted in [Figure 8.1](#). First, some free memory blocks may not have enough space to hold the storage needed for a newly allocated object. As a result, we may encounter several small memory blocks that, despite being free, do not have sufficient space to be reallocated. This phenomenon results in what is known as **fragmentation**. Another drawback is that objects allocated consecutively, such as when an instance field holds a reference to another object, may end up in separate memory segments. If these objects are frequently accessed together, it may require two memory accesses to retrieve each object, which penalizes performance. This is known as **spatial locality**, which refers to the tendency to access data elements that are stored close to each other.

Another issue we may encounter is related to memory that is identified as allocated but has no references to any object located in that memory. This typically corresponds to a **memory leak**, which can occur when the programmer forgets to free a block of memory after clearing all references to it. Without any variable holding a reference to that block of memory, there is no way to add it to the list of free blocks.

All of these issues contribute to the increase in the virtual addressing space occupied by processes and enhances the increase in the working set of processes.

In addition to the inefficient use of the heap, there are performance issues to consider. This allocation process always involves searching the heap for a block of memory large enough to accommodate an object's storage. In contrast, as we will explore later in this chapter, a fundamental principle of automatic memory management is that allocation should be as fast as possible.

8.2: Garbage Collection

The role of a garbage collector is to relieve the application developer from the burden of manual dynamic memory management. This removes the need for developers to manually match allocations with deallocations and carefully manage the lifetimes of allocated memory. This approach mitigates certain types of common memory management errors. Automatic garbage collection consists of inspecting heap memory to distinguish between objects that are actively utilized and those that are not, subsequently eliminating the **unused objects**, also called **unreachable**.

One of the fundamental ideas in the design of garbage collection algorithms is to **make allocation as fast as possible**. For instance, unlike manual allocation, it should avoid searching for a free memory block that is large enough to accommodate the newly allocated object. If we assume *ad absurdum* that the memory heap size is infinite, then we would never need to free memory space for unreachable objects. Allocating a new object would simply involve moving the pointer to the next available address. Each time we allocate a new object, we just advance this pointer by the size of the newly allocated object as illustrated in Figure 8.2

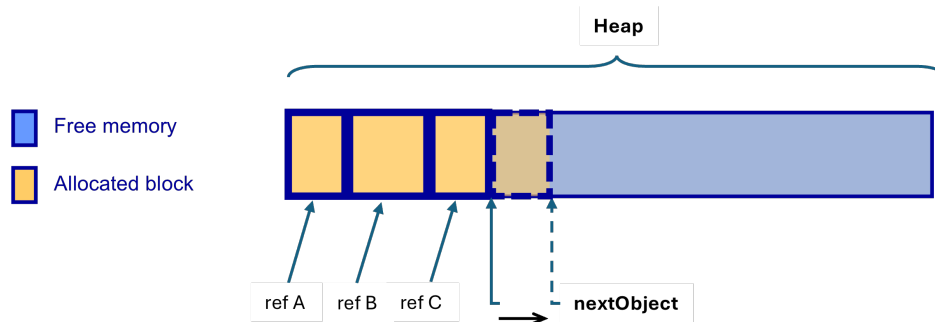


Figure 8.2. The nextObject pointer holds the reference to the beginning of the free space.

This approach not only reduces allocation latency but also addresses issues like fragmentation and enhances **spatial locality** by ensuring that **objects are allocated continuously in memory**. However, since memory space is not infinite, it is essential to have a garbage collection mechanism to **free up memory occupied by unused objects**.

Garbage collection algorithms have been the subject of extensive research for decades, striving to identify the most efficient way to manage memory with minimal overhead. At one point, manufacturers kept their internal garbage collection implementations secret, resulting in less optimized versions of their algorithms being publicly available. Today, most techniques are well-known, and most managed runtimes apply a combination of several techniques, including but not limited to: **marking**, **sweeping**, **compaction**, **generational** collection, and others.

Marking involves distinguishing which objects are still in use, i.e., **reachable**, from the unused objects, i.e., **unreachable**. An object becomes **unreachable**, and thus eligible for garbage collection, when there is **no path from a root reference to the object**. Root references include references from active threads, such as local variables or function parameters. Additionally, Java **static fields** are **root references**, and in Kotlin, top-level variables and companion object declarations, which may also produce static fields, will also act as root references. The marking process starts by marking objects from the root references. It then continues through each object's instance fields, marking the referenced objects, and so on.

Earlier approaches required a **stop-the-world** pause to perform the marking phase. Today, concurrent GC methods aim to reduce latency and minimize pauses by executing the marking phase in parallel. To achieve this, some algorithms take a virtual snapshot of the heap at the beginning of the marking phase. In this approach, all objects that were live at the start of marking

are considered live throughout the process. Consequently, objects that become unreachable during the marking phase are still treated as live, with some exceptions, and their memory space will not be reclaimed. This can result in some extra memory being retained compared to other collectors but offers potentially better latency.

The GC uses the header portion of the object's layout, as described in [Section 2.1.3.1](#), to mark each object. When the marking phase is complete, all objects that were **not marked** are considered **unreachable** and can be freed as garbage. Consider, for instance, a new example with a graph of objects as depicted in [Figure 8.3](#). When the marking phase starts, two references are no longer present: one reference from object B to another object has been cleared, and `ref C` is also gone. Starting from `ref A`, three objects will be marked as reachable, whereas starting from `ref B`, only two objects will be marked. Since `ref C` no longer exists, the object it referenced will not be marked. However, the other object, which is part of the graph starting from `ref C`, has already been marked as reachable because it is also part of the graph starting from the root `ref A`. Hence, at the end of the marking phase depicted in [Figure 8.3](#), there are two unmarked objects that can have their memory reclaimed by garbage collection.

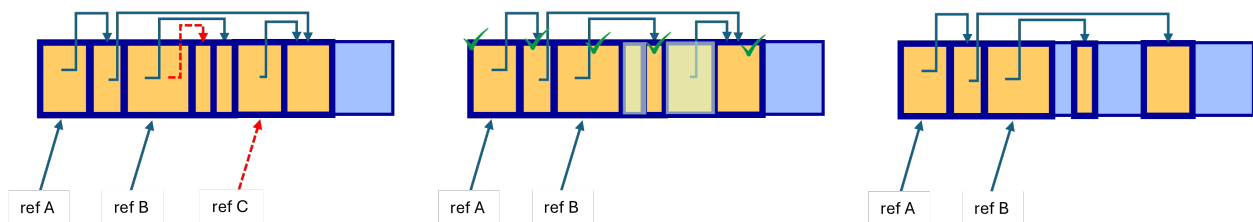


Figure 8.3. Marking process.

Following the marking process, in the **sweep** stage, all memory is scanned from start to finish, examining all blocks, whether free or used. Blocks that are **not marked** as reachable can be freed. For the objects that were **marked**, the flag is cleared, preparing for the next cycle.

The straightforward cleanup of that memory storage can lead to fragmentation, where free memory is broken into small, non-contiguous blocks. **Compaction** is one approach to address this issue. After the marking phase, objects are moved to the previously free space, as depicted in [Figure 8.4](#). However, this process invalidates all references to the objects that have been moved. Consequently, existing references to these relocated objects must be updated to reflect their new locations. **To perform compaction, a stop-the-world is often required**, during which all application threads are paused to ensure consistency while the memory is being reorganized.

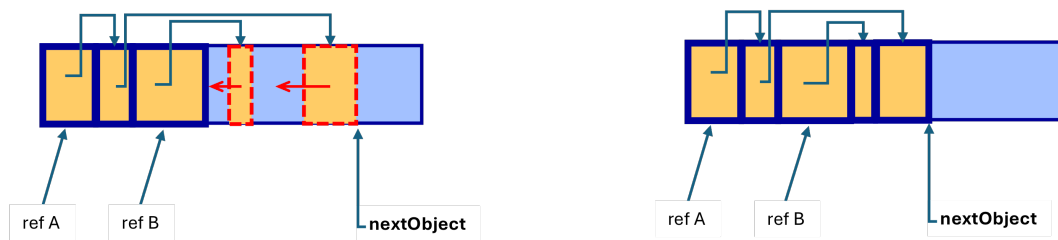


Figure 8.4. Compaction.

All described processes introduce latency, which is proportional to the number of allocated objects. The more objects that are allocated, the longer the garbage collector will take to run. One way to reduce this latency is by limiting the set of objects analyzed by the garbage collector. Instead of scanning the entire heap, the garbage collector can focus on a smaller portion of it. A critical question is which part of the heap should be examined? To optimize garbage collection, it should target the areas where garbage—i.e., unreachable objects—is most likely to be found. Empirical observations suggest that recently created objects are often the most likely to become unreachable quickly. Therefore, focusing on these younger objects can make the garbage collection process more efficient.

Object age can be tracked using a **generational** approach. A newly allocated object starts in generation 0. Each time an object survives a garbage collection cycle, its generation number is incremented. Therefore, an object with a higher generation number is older.

One way to organize object generations is through the *Generation Scavenging Algorithm* [Ungar, 1984], first integrated into the Smalltalk language. The heap is divided into two regions: the **young generation** and the **old generation** (or **tenured**). When the young generation becomes full, a **minor garbage collection** is triggered. The old generation is designated for long-surviving objects. Typically, objects in the young generation are moved to the old generation once they reach a certain age **threshold**. Periodically, the old generation also requires garbage collection, known as a **major garbage collection**.

The **young generation** is divided into two spaces:

- **Eden**, where **new objects** are allocated.
- **Survivor** space, where objects that survive minor collections reside.

The **Survivor** space is further divided into two areas: the **Past Survivor** space and the **Future Survivor** space. After a GC cycle, both the Eden and the Past Survivor spaces are emptied. The GC moves live objects from the Eden and Past Survivor spaces to the Future Survivor space, then swaps the roles of the Past and Future Survivor spaces.

The example in Figure 8.5 shows the layout of the heap on the left side after the marking phase, where two objects in the Eden space and three objects in the Past Survivor space are marked as reachable. One of the three reachable objects in the Past Survivor space, with an age equal to 8, reaches the threshold and is moved to the Tenured space. The other two objects are moved to the

Future Survivor space along with the two live objects from the Eden space. On the right side of Figure 8.5, the heap is shown after the completion of garbage collection. Note that the objects in the Survivor space have their ages incremented, and the roles of the Past and Future Survivor spaces have been swapped.

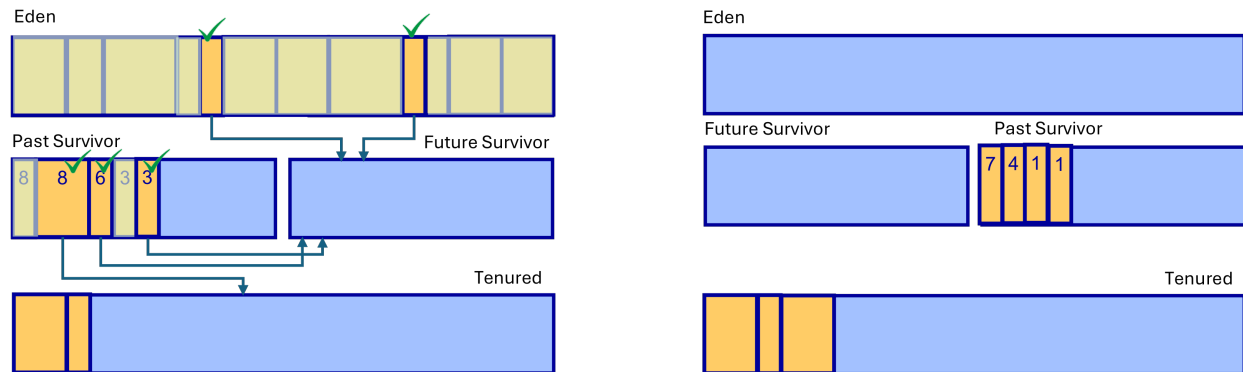


Figure 8.5. Generations and moving live objects.

8.3: Types Requiring Special Cleanup

While objects are managed by the garbage collection (GC) system, some classes create instances that hold **special resources not managed by GC**. Examples of these classes include those used for file system access, database connections, network communication via sockets, and others. Instances of these classes need to perform **special cleanup tasks** to release native resources at the end of their lifecycle. The code for this cleanup is specific to each class and must be implemented in a specific method. Typically, there are two different methods for this purpose in managed runtime environments:

1. `close/dispose` - this method should be called programmatically by the developer as soon as an instance is no longer needed.
2. A *finalizer* or *cleaner* method that is invoked by the runtime if the developer forgets to explicitly call `close`. This acts as a safeguard, ensuring that cleanup tasks are performed even if the programmer neglects to do so.

8.3.1: Using closeable types

Native resources are limited, and while a process holds an exclusive handle to such a resource, it prevents others from using it. Therefore, it is a good programming practice to release these resources as soon as they are no longer needed. To facilitate this, managed runtime environments

often define a reserved method, usually called `close` or `dispose`, sometimes specified by an interface like `Closeable` in Java or `Disposable` in .NET. For historical reasons, the JVM ecosystem includes two interfaces, `Closeable` and `AutoCloseable`, which are almost indistinguishable in their usage from Kotlin. A discussion of their differences is beyond the scope of this book. We will use the term *closeable* to refer to any type that provides a `close` method and may hold native resources.

The `close` method should be explicitly called by programmers when they no longer need to use an instance. Moreover, since exceptions can interrupt the execution of a block, it is advisable to place the `close` call inside a `finally` block to ensure that resources are properly released regardless of whether an exception occurs. Figure 8.6 shows an example of using a `FileWriter` with the `close` method invoked inside a `finally` block.

Figure 8.6. Explicitly call `close()` in `finally` block.

```
var writer: FileWriter? = null
try {
    writer = FileWriter("temp.txt")
    writer.write("...")
    ... // do other stuff
} finally {
    writer?.close()
}
```

To avoid the tedious task of correctly writing code to manage *closeable* types and to prevent programming mistakes such as forgetting to call `close`, some programming languages provide utilities such as `try-with-resources` in Java, `using` in C#, `use` in Kotlin, or other. These features alleviate the burden on programmers by allowing the compiler to generate the equivalent resource management code automatically and ensure its correct usage. For example, in Java, the code in Figure 8.7, or in Kotlin, the code in Figure 8.8 is compiled to something equivalent to the code written in Figure 8.6.

Figure 8.7. Example of `try-with-resources` in Java

```
try(Writer writer = new FileWriter("temp.txt")) {
    writer.write("...");
    ... // do other stuff
}
```

Figure 8.8. Example of use in Kotlin

```

FileWriter("temp.txt").use { writer ->
    writer.write("...")
    ... // do other stuff
}

```

When implementing your own type that wraps other `Closeable` types, you should also make your type `Closeable`. To achieve this, you can implement the `Closeable` interface, which requires your class to provide an implementation for the `close` method. In your `close` method implementation, you should propagate the close call by invoking `close` on all wrapped *closeable* instances.

Consider, for example, the use case of a `TempImage` class that wraps an instance of `BufferedImage`. The `TempImage` class is instantiated with a URL to an image, which is downloaded only if it does not already exist in the filesystem. The `close` method should delete the image file from the filesystem when it is invoked.

The code in [Figure 8.9](#) presents a possible implementation of the `TempImage` class. We included an additional `downloaded` property to indicate whether the image associated with the URL has been downloaded. Note that we use the `InputStream` from the connection to the given URL to download the image. Since an `InputStream` is also `Closeable`, we manage its instance within a `try-with-resources` block. The `close` method simply deletes the downloaded file if it exists.

Figure 8.9. `TempImage` class deleting file on close.

```

class TempImage(url: String) : Closeable {
    val img: BufferedImage
    val downloaded: Boolean
    private val file = File(url.substringAfterLast('/'))
    init {
        if(file.exists()) {
            img = ImageIO.read(file) // read from local file
            downloaded = false
        } else {
            val conn = URI(url).toURL().openConnection()
            val kind = conn.getHeaderField("Content-Type").substringAfterLast('/')
            conn.getInputStream().use { stream ->
                img = ImageIO.read(stream) // read from the connection stream
                ImageIO.write(img, kind, file)
                downloaded = true
            }
        }
    }
}

override fun close() {

```

```

        if(file.exists()) {
            file.delete()
        }
    }
}

```

In the following test, we can observe that `duke_star7.png` is downloaded once for the outer `TempImage` instance. Outside the scope of both `use` blocks, we can see that `duke_star7.png` has been deleted.

Figure 8.10. Using `TempImage` within `use` block.

```

TempImage("https://dev.java/assets/images/duke/duke_star7.png")
    .use {
        assertTrue(it.downloaded)
        TempImage("https://dev.java/assets/images/duke/duke_star7.png")
            .use { second ->
                assertFalse(second.downloaded)
            }
    }
assertFalse(File("duke_star7.png").exists())

```

8.3.2: Finalization and Cleaners

Although the idioms presented in [Section 8.3.1](#) assist programmers in correctly using *closeable* types, following these patterns is not mandatory. Moreover, there are scenarios where a *closeable* instance needs to be passed across different scopes, making the try-with-resources idiom unsuitable. So, how can you implement an escape path that enables the runtime to release resources, such as deleting a temporary file, even if the programmer does not call the `close` method at the end of the lifecycle of a *closeable* object?

Note that having garbage collection invoke `close` is not a suitable option. First, garbage collection should remain as efficient as possible and should not be slowed down by instances holding native resources. Second, the operation of the GC should not be affected if there are issues with releasing those resources.

For these reasons, managed runtime environments usually offer a separate mechanism for native resource cleanup. In older JVM versions and in .NET, this mechanism is provided through **Finalization**. Since Java 9, an alternative to finalization is available through the **Cleaner API**, which avoids many of the drawbacks of finalization. To prepare developers for the removal of finalization in a future version of Java, starting with Java 18, a new command-line option `--finalization=disabled` can be used to disable finalization. However, the complete removal of finalization will be a long process, and for a while, developers may still encounter this feature. Thus, we will first take a look at the finalization mechanism before delving into the Cleaners API.

Finalization enables an object to run code after it is considered unreachable but before its memory is freed from the managed heap. When the JVM identifies that an object is no longer reachable, the object can finalize itself by releasing the native resource it holds. Subsequently, the garbage collector will reclaim the object's memory from the managed heap. **Note that an object implementing finalization requires at least two GC cycles to free its memory.** One cycle is needed to identify the object as unreachable, and a second cycle is required to reclaim its memory after finalization has run. This approach is not ideal for memory consumption, which is why finalization should be avoided when possible. Additionally, if finalizable objects are promoted, any objects referenced by their fields are also promoted to ensure they continue to live.

An object is marked for finalization if its class overrides the `finalize` method from `java.lang.Object`. The JVM uses a dedicated thread to invoke these `finalize` methods. If a `finalize` method blocks, this thread cannot proceed to call other `finalize` methods. Additionally, if a `finalize` method throws an unhandled exception, it cannot be caught. Therefore, there are many drawbacks associated with `finalize` methods, and they should be used with caution.

Let's create a straightforward implementation of the `finalize` method in our `TempImage` class and analyze its effects. This implementation, shown in Listing 11, calls the existing `close` method. Overriding the `finalize` method in Kotlin suppresses the `override` keyword. The rest of the `TempImage` class remains the same as in [Figure 8.9](#), with the only addition being a `println` statement in the `close` method to highlight the effects of this new version of `TempImage`.

Figure 8.11. `TempImage` class deleting file on close.

```
class TempImage(url: String) : Closeable {  
    ...  
    override fun close() {  
        println("Try deleting file...")  
        if(file.exists()) {  
            file.delete()  
        }  
    }  
  
    protected fun finalize() = close()  
}
```

Using this new version of `TempImage`, even if we forget to call the `close` method, the GC may eventually determine its instances as unreachable, and when the finalization thread runs, it will call the `finalize` method, which will delete the temporary file. To observe this behavior, we can create an instance of `TempImage` in an auxiliary function, such as `loadImageAndForgetClose` depicted in [Figure 8.12](#). After the function completes, we can force garbage collection by calling `System.gc()`. If finalization occurs, the file will be deleted. Note that `System.gc()` is used here

only for demonstration purposes and should be avoided in production code. Additionally, since finalization runs in a separate thread, we include `Thread.sleep(100)` to give opportunity for finalization thread to execute.

Figure 8.12. Observing `TempImage` finalize effect.

```
fun loadImageAndForgetClose() {
    TempImage("https://dev.java/assets/images/duke/duke_star7.png")
        .also {
            assertTrue(it.downloaded)
        }
}
loadImageAndForgetClose()
assertTrue(File("duke_star7.png").exists())
/*
 * Once an object is eligible for GC, finalization may occur on a different thread,
 * so you might need to pause briefly to observe the changes.
 */
System.gc()
Thread.sleep(100)
assertFalse(File("duke_star7.png").exists())
```

Although this new version of `TempImage` appears to solve our issue, it introduces additional overhead. Instances will always require at least two GC cycles to have their memory reclaimed from the heap, even if the programmer ensures that the `close` method is correctly invoked. Furthermore, in this case, `close` will be called twice: once programmatically and once by the finalization thread through the `finalize` method.

In the code of Figure 8.12, if we replace the `also` scope function with `use`, which ensures that the `close()` method is called at the end of the block, we will see the message "Try deleting file..." appear twice in the console output. One instance of the message comes from the explicit call to `close()`, and the other comes from the `finalize` call during finalization. We could use a flag to indicate whether `close` has already been executed, but we cannot avoid the other issues associated with finalization.

This well-known problem has been addressed in other managed runtime environments for decades. For example, in .NET, the standard GC class provides a `SuppressFinalize` method [Box, 2003] that removes the specified object from the finalization queue, preventing redundant finalization.

The Java **Cleaner API** introduced in JDK 9, provides a more efficient and flexible alternative to finalization. Unlike finalization, the Cleaner API allows for more predictable and immediate cleanup

of resources. The cleanable class registers objects through a `Cleaner` object, specifying a *cleanup action* to be executed when the object becomes unreachable.

To do this, first, we need an instance of `Cleaner`, which either creates a new daemon thread to run the *cleaning actions* or uses a thread from a provided `ThreadFactory`. This `Cleaner` instance can be shared across different classes or used independently, depending on the developer's needs. Next, a *cleaning action* is registered in the `Cleaner` using an instance of a class implementing `Runnable` with the cleanup code. Registering a *cleaning action* through the `Cleaner.register(action: Runnable)` method returns an instance of `Cleanable`, which can be used to monitor the cleaning process. In the diagram of Figure 8.13, the `_` class represents an anonymous class implementing the `Runnable` interface, which encapsulates the cleanup code—such as an operation involving a `File` instance.

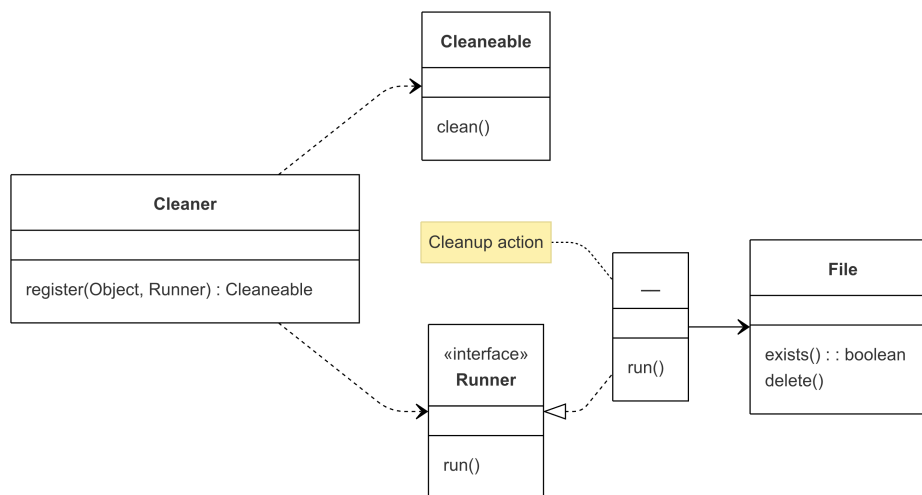


Figure 8.13. Class diagram of Cleaner API.

In the new implementation of `TempImageCleanable` of Figure 8.14, the code that deletes the temporary file is placed in the cleaning action rather than in `close`. The `close` method, in turn, invokes the cleaner via the `Cleanable` reference. This approach has a beneficial effect: **when the `clean` method is called on the `Cleanable` object, the associated cleaning action is removed from the cleaner and will no longer be executed.** As a result, if an already closed `TempImageCleanable` instance becomes unreachable, it can be immediately reclaimed from the heap by GC without undergoing the cleaning process.

Figure 8.14. TempImageCleanable implementing cleanup logic using a Cleaner.

```

1  class TempImageCleanable(url: String) : Closeable {
2      companion object { val cleaner: Cleaner = Cleaner.create() }
3
4      private val cleanable = cleaner.register(this, object : Runnable {
5          private val file = File(url.substringAfterLast('/'))
6          override fun run() {
7              println("Try deleting file...")
8              if(file.exists()) { file.delete() }
9          }
10     })
11     override fun close() {
12         cleanable.clean()
13     }
14     ... // the rest remains the same as TempImage

```

Now, if we repeat the test of [Figure 8.12](#) with the `also` scope function replaced by `use` and `TempImage` replaced by `TempImageCleanable`, the message "Try deleting file..." will appear only once from the call to `close()`. This is because the *cleaning action* will no longer be executed after the `close` method is called.

The Java documentation for the Cleaner API advises caution when implementing a cleaning action in a lambda expression or anonymous class because:

easily will capture the object reference, by referring to fields of the object being cleaned, preventing the object from becoming phantom reachable

For example, in `TempImageCleanable`, if we suppress the duplicated property declared in the *cleaning action* (line 6 of [Figure 8.14](#)):

```
private val file = File(url.substringAfterLast('/'))
```

and, instead capture the `file` property from the enclosing class, it appears to be a viable optimization that behaves the same way. However, in that case, the inner class will capture a reference to the enclosing instance, which will prevent it from being garbage collected.

•

Bibliography

- [Binstock, 2022] Binstock, Andrew. “*Understanding the constant pool inside a Java class file*”, Oracle Java Magazine, February, 2022
- [Box, 2003] Box, D., & Sells, C. “*Essential. Net: the common language runtime (Vol. 1)*”, Addison-Wesley Professional, 2003
- [Bruneton, 2002] Bruneton, Eric, Romain Lenglet, and Thierry Coupaye. “*ASM: a code manipulation tool to implement adaptable systems*”, *Adaptable and extensible component systems* 30.19, 2002
- [Evans, 2023] Evans, Ben “*The performance implications of Java reflection*”, Oracle Java Magazine, Apr. 2023
- [Fowler, 2012] Fowler, Martin. “*Patterns of enterprise application architecture*”, Addison-Wesley, 2012
- [Fowler, 2015] Fowler, Martin. “*Collection Pipeline*”, <https://martinfowler.com/articles/collection-pipeline/>, 2015
- [Gamma, 1993] Gamma, Erich, et al. “*Design patterns: Abstraction and reuse of object-oriented design*”, ECOOP’93—Object-Oriented Programming: 7th European Conference Kaiserslautern, Germany, July 26–30, 1993 Proceedings 7. Springer Berlin Heidelberg, 1993
- [Goetz, 2012] Goetz, Brian. “*Translation of lambda expressions*”, <https://cr.openjdk.org/~briangoetz/lambda/lambda-translation.html>, Apr. 2012
- [Knuth, 1974] Knuth, Donald E. “*Structured programming with go to statements*”, *ACM Computing Surveys (CSUR)* 6.4, 261-301, 1974
- [Moskala, 2022] Moskala, Marcin. “*Kotlin Essentials*”, Leanpub, 2022
- [Ungar, 1984] Ungar, David. “*Generation Scavenging: A non-disruptive high performance storage reclamation algorithm*”. In *Proceedings of the first ACM SIGSOFT/SIGPLAN software engineering symposium on Practical software development environments (SDE 1)*, 157–167. <https://doi.org/10.1145/800020.808261>, 1984